國立台灣海洋大學

通訊與導航工程學系

專題研究報告

指導教授:莊季高教授

不同風速下校園風場數值模擬與實體量測對比分析

學生：林哲宇、蘇意為 撰

中華民國 114 年 11 月

摘要

本研究旨在解決國立臺灣海洋大學理工科系在無人機飛行與應用研究中，對校園微氣候風場數據缺乏經驗證、可靠數值的實務痛點。透過整合數值模擬與高機動性實測，本專題目標為評估 OpenFOAM 搭配 紊流模型在不同風速工況下的預測可靠度，並提供一套實用的模擬基準。

在方法論上，我們使用 Blender 和 blosm 插件建立校園建築群的精確幾何模型，並將 yPlus 值處於極高範圍的挑戰，視為硬體限制下的工程取捨。實測部分則採用四軸旋翼無人機搭載高精度感測器，針對 2 級、 3 級、4 級風三種工況，各進行了三次獨立的重複測量，以確保數據的統計嚴謹性與可重複性。

結果顯示： 經由空間對齊與 RMSE 相關係數的定量對比，OpenFOAM 模型在中度風速下表現最佳。其中，3 級風工況的模擬數值與實際測量的趨勢相近，證明了模型在宏觀流場趨勢預測上的可參考性。然而，在 2 級風工況下，模擬數值顯著增高，這主要歸因於 RANS 模型的固有局限性以及近壁面 yPlus 極限的影響。

本研究結論： 模型在中度風速下具有顯著的工程應用價值。本專題成果為海洋大學理工科系提供了一套經實測校準的 CFD 模擬操作基準，直接降低了無人機飛行優化所面臨的風場未知影響。

關鍵詞：OpenFOAM、CFD、無人機量測、$k-\omega\ SST$ 、RMSE、yPlus

目錄

摘要 2

目錄 3

圖目錄 5

表目錄 8

第一章 緒論 9

1.1 研究動機 9

1.2 研究目的 10

1.3 研究材料及方法 10

1.3.1 主要元件規格及其用途: *13*

第二章 系統架構設計 18

2.1 系統架構設計 18

2.1.1 感測層 *19*

2.1.2 通訊層 *19*

2.1.3 主控層 *21*

2.1.4 顯示層 *22*

2.2 系統流程 23

2.2.1 系統啟動與模組初始化 *23*

2.2.2 Modbus RTU 查詢流程 *23*

2.2.3 感測器回應資料解析 *23*

2.2.4 JSON 打包與 USB Serial 資料傳輸 *24*

2.2.5 Raspberry Pi 端資料接收與緩衝機制 *24*

2.2.6 每分鐘平均化與資料紀錄 *24*

2.2.7 Flask API 資料提供流程 *25*

2.2.9 系統流程總結 *25*

2.3 系統實作 26

2.3.1 硬體實作 *27*

2.3.2 軟體實作 *29*

2.4 系統測試與驗證 40

2.4.1 RS-485 通訊測試 *40*

2.4.2 ESP-32S JSON 封裝與 USB 資料傳輸測試 41
2.4.3 後端伺服器與資料處理功能驗證 41
2.4.4 CSV 資料寫入測試 42
2.4.5 前端顯示功能測試 43
第三章 研究方法與實驗設計 45
3.1 校園 CFD 建模與數值設定 45
3.1.1 校園幾何模型建立 45
3.1.2 數值網格生成與品質控制 46
3.1.3 OpenFOAM 數值模型與邊界條件 48
3.1.4 邊界條件設定與多工況設計 49
3.2 無人機實測系統與數據採集 49
3.2.1 無人機平台與感測器集成 50
3.2.2 實驗設計流程及地點選擇 50
3.2.3 飛行路徑與數據採集策略 51
3.3 數據處理與定量對比方法 53
3.3.1 無人機數據前處理與平均化 53
3.3.2 定量誤差指標計算 54
第四章 實驗結果與討論 55
4.1 CFD 數值模擬結果展示 55
4.1.1 三級風下各點位風場數值 55
4.2 無人機實測數據整理與處理 73
4.2.1 實測數據品質與可重複性分析 73
4.3 模擬與實測數據的定量對比 77
4.4 實驗結果之檢討與建議 82
第五章 結論 83
第六章 參考資料 84
附錄 86

圖目錄

圖 1：研究方法 11
圖 2:四軸旋翼無人機(含各元件) 11
圖 3:超聲波風速風向儀 11
圖 4:FRSKY 無線遙控接收器、HOLYBRO TELEMETRY RADIO 數傳模組(一對) 12
圖 5:無刷直流馬達（EAGLEPOWER LA4312 KV350） × 4 12
圖 6:PIXHAWK 4 MINI 飛行控制器 12
圖 7:PIXHAWK 4 GPS 模組 12
圖 8:NODEMCU-ESP-32S 開發板 12
圖 9:RS-485 通訊模組 12
圖 10:RASPBERRY PI 5 13
圖 11:MINI POWER HUB(分電板) 13
(圖 12:系統架構流程圖) 18
(圖 13:硬體接線電力與資料關係圖) 21
(圖 14:完整資料處理流程) 26
(圖 15:EAGLEPOWER LA4312 KV350 無刷馬達安裝) 27
(圖 16:ESC 與電線配置示意) 28
(圖 17:飛控安裝示意) 28
(圖 18:前端網頁模擬圖) 40
(圖 19:RASPBERRY PI 5 啟動 SERVER.PY) 42
(圖 20:確實建立 CSV 檔以及每分鐘的平均數據) 43
(圖 21:雷射測距儀) 45
(圖 22:海洋大學的校區模擬模型) 46
(圖 23:模擬輸出的部分內容) 47
(圖 24:YPLUS 輸出數值) 47
(圖 25:實測點位-工學院外圍) 52
(圖 26: 實測點位-延平大樓旁) 52
(圖 27: 實測點位-生科院旁) 53
(圖 28:延平大樓旁風速圖) 55
(圖 29:工學院旁風速圖) 56
(圖 30:生科院旁風速圖) 56
(圖 31:延平大樓旁風速圖) 57

(圖 32:工學院旁風速圖).....57
(圖 33:生科院旁風速圖).....58
(圖 34:延平大樓旁風速圖).....58
(圖 35:工學院旁風速圖).....59
(圖 36:生科院旁風速圖).....59
(圖 37:延平大樓旁風速圖).....60
(圖 38:工學院旁風速圖).....60
(圖 39:生科院旁風速圖).....61
(圖 40:延平大樓旁風速圖).....61
(圖 41:工學院旁風速圖).....62
(圖 42:生科院旁風速圖).....62
(圖 43: 延平大樓旁風速圖).....63
(圖 44:工學院旁風速圖).....63
(圖 45:生科院旁風速圖).....64
(圖 46: 延平大樓旁風速圖).....64
(圖 47:工學院旁風速圖).....65
(圖 48:生科院旁風速圖).....65
(圖 49: 延平大樓旁風速圖).....66
(圖 50:工學院旁風速圖).....66
(圖 51:生科院旁風速圖).....67
(圖 52:延平大樓旁風速圖).....67
(圖 53:工學院旁風速圖).....68
(圖 54:生科院旁風速圖).....68
(圖 55:延平大樓旁風速圖).....69
(圖 56:工學院旁風速圖).....69
(圖 57:生科院旁風速圖).....70
(圖 58:延平大樓旁風速圖).....70
(圖 59:工學院旁風速圖).....71
(圖 60:生科院旁風速圖).....71
(圖 61:延平大樓旁風速圖).....72
(圖 62:工學院旁風速圖).....72
(圖 63:生科院旁風速圖).....73
(圖 64:延平大樓旁及工學院實測照片).....74

(圖 65:生科院實測照片)....75
(圖 66:2 級風速圖與標準差)....76
(圖 67:3 級風速圖與標準差)....76
(圖 68:4 級風速圖與標準差)....77
(圖 69:延平大樓旁風速數據對比)....78
(圖 70:工學院旁風速數據對比)....78
(圖 71:生科院旁風速數據對比)....79
(圖 72:延平大樓旁風速數據對比)....79
(圖 73:工學院旁風速數據對比)....80
(圖 74:生科院旁風速數據對比)....80
(圖 75:延平大樓旁風速數據對比)....81
(圖 76:工學院旁風速數據對比)....81
(圖 77:生科院旁風速數據對比)....82

表目錄

(表 1：風速計規格) 19
(表 2：三種風級工況的 OPENFOAM 入口邊界條件) 49
(表 3：2 級風數據表) 73
(表 4：3 級風數據表) 74
(表 5：4 級風數據表) 74

第一章 緒論

1.1 研究動機

在挑選專題時，我們思考了許久遲遲想不到要做什麼主題，在指導教授的建議下，我們找了碩班的學長聊聊，在因緣際會之下我們得知了，原來風場的不同會造成無人機電池的使用時間，也會影響到 PID 的參數，於是我們就想知道有沒有什麼辦法，可以不用每次都去測量風速就能知道實際的風場數值，於是學長跟我們介紹了一個開源軟體 OPENFOAM，它可以很好的模擬及呈現假設風流進來會有什麼樣的變化。

但是我們遇到了一個問題，數值模擬的結果並不一定是完全跟現實情況相符的，需要有實際測量的數據來佐證，模擬結果也可能因為模型設定或邊界條件的不同而出現誤差，特別是在建築物密集、氣流變化劇烈的邊界層區域中誤差會變得更為嚴重。因此，我們這項研究的主要目的，就是要了解「模擬」與「實際」之間的差距。

我們決定使用 Blender 來建立校園建築的立體模型，接著利用先前提到的 OpenFOAM 進行風場數值模擬，並加入無人機進行風速及風向的測量。透過這樣的方式，我們能得到更加接近現實情況的數據，也能以多風速、多次重複的實測數據來驗證模擬的準確性，進一步了解 OpenFOAM 在模擬實際校園環境中的可靠度。最後，我們希望能得到一套經過實測的可信的模擬基準，讓未來進行相關研究時，能把 OpenFOAM 這個程式做為可以信任的模擬軟體，使未來的研究增加一個可以選擇或參考的方式。

由於海洋大學的校園鄰近海邊，風場變化快速而且複雜，這讓準確的風場資料顯得特別重要。許多研究領域，像是無人機飛行控制、風能發電設備的部署、大氣污染擴散模擬，都需要比較精確地風速與風向資訊。當風速從二級（輕風）上升到四級（和風）時，氣流的分離位置、渦流形成以及紊流強度也都會有明顯的變化，我們希望能透過這個專題，利用多種風速條件下的實測數據來驗證數值模型，來驗證這種風場模擬方法是不是可信的，為未來的研究提供依據。

1.2 研究目的

本研究的目的是結合數值模擬與高精確度無人機的實地測量，來評量 OpenFOAM 在模擬海洋大學校園內特定建築群間，風在不同速度下流動時的準確性，並建立一個可供研究參考的基礎。

首先會使用 Blender 軟體，依照實際尺寸建立校園的完整立體建築模型，作為後續 OpenFOAM 模擬的基礎資料。接著在 OpenFOAM 軟體上設定適當的紊流模型與網格策略，設定 2 級、3 級與 4 級風速三種不同情境進行定常流場模擬，取得校園內風速與風向的立體分布結果。

為了驗證模擬的可靠度，我們選定了幾棟建築物，在建築物周圍利用無人機在實地實測。無人機會在選定的建築物周圍以 10 公尺為一個級距，在三種風速條件下各進行三次測量，利用搭載的風向風速器收集實際的風場數據，透過樹莓派獲得即時數據，確保結果具有統計代表性。

在資料分析時，我們會將模擬結果與三次實測的平均值進行空間上的對應，並透過 均方根誤差（RMSE）、平均絕對誤差（MAE） 以及 相關係數（R） 來計算並且以數值來呈現兩者的差異性與一致性。

最後，根據比較結果，我們將探討模型在不同風速下的誤差來源，並提出改進 OpenFOAM 模型參數與操作流程的具體建議。希望能將這些成果作為海洋大學理工相關科系的學生們進行風場或微氣候模擬時的參考。

1.3 研究材料及方法

本研究之研究流程(圖 1)及所使用之主要材料與設備含各項主要元件(圖 2~圖 11):

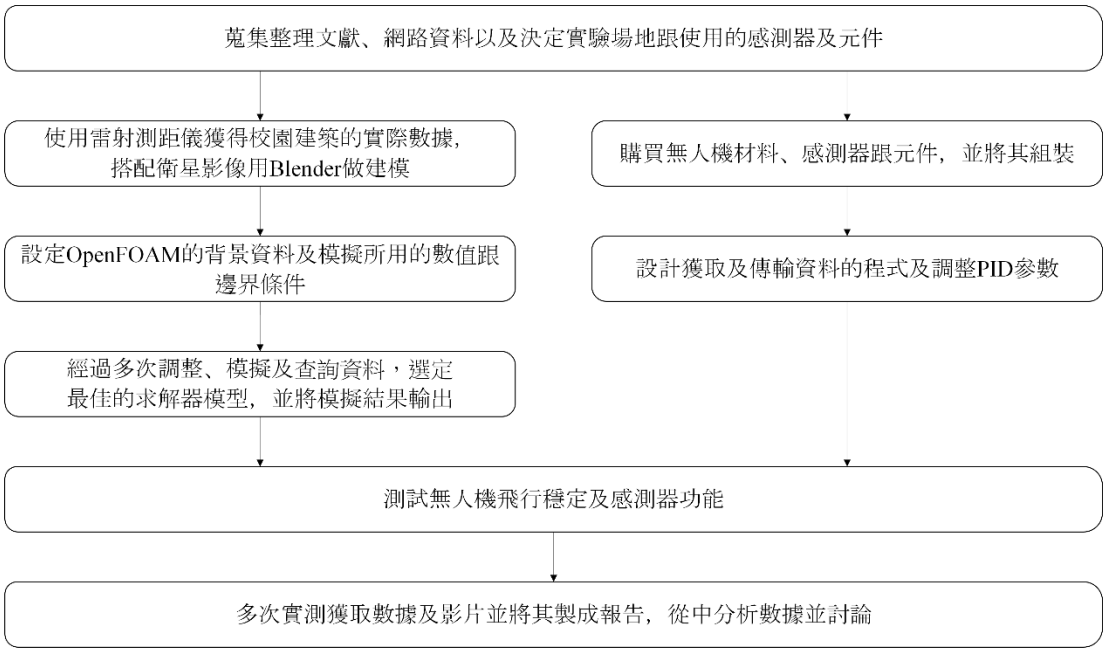


圖 1：研究方法



圖 2:四軸旋翼無人機(含各元件)



圖 3:超聲波風速風向儀



圖 4:FrSky 無線遙控接收器、Holybro Telemetry Radio 數傳模組(一對)



圖 5:無刷直流馬達（Eaglepower LA4312 KV350） × 4



圖 6:Pixhawk 4 Mini 飛行控制器



圖 7:Pixhawk 4 GPS 模組



圖 8:NODEMCU-ESP-32S 開發板



圖 9:RS-485 通訊模組



圖 10:Raspberry Pi 5

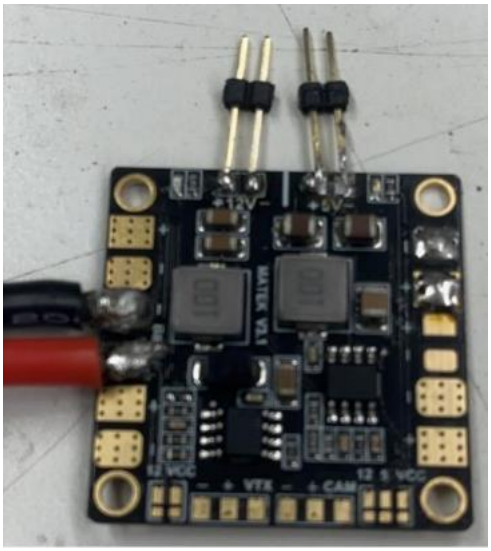


圖 11:Mini Power Hub(分電板)

1.3.1 主要元件規格及其用途:

ESP-32S

1. **參數規格[2]**
   - 模組型號：NodeMCU-32
   - 尺寸：25.4mm(W)*48.3mm(H)±0.2mm
   - 封裝：DIP-38(2.54 間距標準排針)
   - SPI FLASH：默認 32 Mbits
   - 支持接口：UART/SPI/SDIO/I2C/PWM/I2S/IR/ADC/DAC
   - 頻率範圍：24002483.5λMHz
   - 串口速率：支持 300~4608000bps,默認 115200bps
   - 藍牙：藍牙 4.2BR/EDR 和 BLE 標準
   - SPI Flash：默認 32Mbit,最大支持 128Mbit
   - 工作溫度：-20℃C70℃C
   - 存儲環境：-40℃~125℃，<90%RH
   - 供電範圍：
   - Micro USB 供電電壓 4.75V5.25V,推薦 5.0V
   - 供電電壓 3.0V~3.6V,供電電流>500mA,推薦 3.3V
2. **用途**
   - 作為通訊層核心微控制器。
   - 透過 MAX485 模組與超音波風速風向儀進行 Modbus RTU 通訊。
   - 定期以 RS-485 發送查詢指令並接收風速／風向資料。
   - 將解析後的資料封裝為 JSON 字串經 UART（115200 bps）傳送

至 Raspberry Pi 5。

- 作為感測器與主控層之間的資料「轉換節點」。

RS-485 模組

1. **參數規格[3]**
   - 晶片具有限擺率驅動程式，減少 EMI，並減少由不恰當的終端匹配線纜引起的信號反射，從而使無差錯信號傳輸率高達 500Kbps。
   - 訊號形式：差動訊號（A / B 線）
   - 邏輯電平：TTL（與 ESP-32S UART 電平相容）
   - 具有 1/4 單位負載的接收器輸入阻抗，支援多機通訊，允許接在不超過 128 個設備的匯流排上
   - 工業級晶片工作溫度可以達到-40℃至+85℃。
   - 具有電源和 RXD、TXD 信號指示燈
   - 支援 3.3V 和 5V 信號控制
2. **用途**

本研究採用的 RS-485 轉換模組以 MAX485 晶片為核心，用於將超音波風速風向儀所輸出的 RS-485 差動訊號轉換為 ESP-32S 可讀取之 UART 電平訊號。由於超音波風速風向儀採用 Modbus RTU 協定，且使用 半雙工 RS-485 作為實體層，因此必須透過 MAX485 進行電氣層介面轉換與方向控制，才能與 ESP-32S 正確通訊。

MAX485 模組之於本系統的用途如下：

(1) **進行 RS-485 → UART 電氣層轉換（感測器 → ESP-32S）**

- 超音波風速風向儀以 RS-485 差動訊號輸出量測資料，ESP-32S 僅支援 UART（TTL 電平）
- MAX485 負責將 RS-485 A/B 差動訊號轉換為單端 UART 訊號（RO → ESP-32S RX）
- 同時將 ESP-32S 的 UART 傳送訊號（TX → DI）轉換為 RS-485 差動訊號（A/B）
- 確保感測資料能在兩種不同電氣標準間正確傳輸

(2) **提供 UART → RS-485 的查詢指令（ESP-32S → 風速儀）**

ESP-32S 程式碼會主動發送：

node.readHoldingRegisters(0x0001, 1);

此指令會：

- 由 ESP-32S 透過 UART TX 輸出查詢封包
- 經 MAX485 的 DI 腳位轉成 RS-485 差動訊號（A/B）
- 傳送到風速儀（Slave）

因此 MAX485 是 ESP-32S 能進行 Modbus 查詢的必要模組。

(3) **處理半雙工通訊的方向切換（DE/RE）**

RS-485 以半雙工方式運作，需要於「傳送模式」與「接收模式」

間切換。

MAX485 的 DE/RE 腳位提供此切換功能：

- DE/RE＝HIGH → 進入傳送模式（主站查詢風速儀）
- DE/RE＝LOW → 進入接收模式（等待回應封包）

此機制確保 Modbus RTU 的查詢與回應不互相衝突，是本研究通訊流程能穩定運作的關鍵。

(4) **提供差動訊號之抗干擾能力，適用於無人機工作環境**

- RS-485 差動訊號具備高抗 EMI 能力
- 可有效抑制無人機上馬達、ESC、高電流線路所產生的噪聲
- 即使在振動與動態電磁干擾環境中，仍能維持穩定、可靠的資料傳輸

(5) **作為感測層與通訊層之間的訊號穩定化橋接模組**

MAX485 負責整合：

- 風速風向儀（RS-485）
- ESP-32S（UART）

確保感測資料以正確的電氣格式傳輸至通訊層，

並為後續資料解析與數據封裝提供穩定來源。

Raspberry Pi 5

1. **參數規格[4]**
   - 處理器：Broadcom BCM2712，64 位元四核心 Arm Cortex-A76，時脈 2.4 GHz，內建加密擴充，每核心 512 KB L2 快取，2 MB 共用 L3 快取。
   - 圖形處理器：VideoCore VII GPU，支援 OpenGL ES 3.1 與 Vulkan 1.2，具 800 MHz 時脈，並內建 4Kp60 HEVC 解碼能力。
   - 顯示輸出：雙 HDMI，可支援 4Kp60 螢幕顯示，並具備 HDR 支援。
   - 記憶體：LPDDR4X-4267 SDRAM，提供 2GB, 4GB, 8GB 和 16GB 容量選項。
   - 網路無線功能：雙頻 802.11ac Wi-Fi（Wi-Fi 5），以及藍牙 5.0／低功耗藍牙（BLE）。
   - 儲存與擴充性：
   - microSD 卡插槽，支援高速 SDR104 模式（效能為前代的兩倍）。
   - 提供單條 PCIe 2.0 ×1 插槽，可搭配 M.2 HAT 或其他轉接板，連接高頻寬外設。
   - USB 接口：2 × USB 3.0（同時支援 5 Gbps 傳輸），及 2 × USB 2.0。
   - 以太網路與 PoE 支援：千兆乙太網路（Gigabit Ethernet），支援

PoE+（需另購 PoE+ HAT）。

- MIPI 接口：2 × 四通道（4-lane）MIPI 攝影機／顯示器收發器，帶寬為先前兩條 1 Gbps 接口的三倍，可支援任意兩個攝影機或顯示設備組合。
- 電源：5V/5A DC，透過 USB-C 提供，支援 Power Delivery。
- GPIO：標準 Raspberry Pi 40-pin 接頭（向後相容）。
- 其他功能：內建實時時鐘（RTC，需外部電池供電）與電源開關按鈕

2. **用途**

Raspberry Pi 5 在本研究中作為 主控層（Processing Layer）

負責 ESP-32S 傳回資料的接收、資料處理、資料記錄與視覺化網頁服務之提供，是整體系統中最核心的資料管理平台。

(1) **作為主控層資料處理核心**

- 以 USB-to-Serial 介面（115200 bps）接收 ESP-32S 所傳送的 JSON 格式風速／風向資料
- 使用 Python（pySerial）解析每筆資料並存入記憶體緩衝區
- 執行資料有效性檢查（防空讀取、不完整 JSON 濾除）
- 將「最新數據」存放於系統共用變數以供網頁查詢

(2) **執行資料統計與長期儲存**

Raspberry Pi 5 負責以背景執行緒定期處理感測資料：

- 每分鐘計算一次平均風速與平均風向
- 自動建立每日 CSV 檔案（依日期命名）
- 儲存完整的時間戳記、平均風速、平均風向資料
- 作為風場長期監測及後續分析之數據來源

(3) **建立即時資料伺服器（Flask Web Server）**

Raspberry Pi 5 使用 Flask 建立本研究的前端服務：

- /data：提供最新風速（m/s）與風向（°）之 JSON API
- /：提供視覺化網頁（Chart.js）
- 允許其他裝置透過同網域 IP 直接瀏覽即時資料（例如手機、筆電）

此架構讓使用者可即時查看無人機載測風場之即時資訊。

(4) **提供資料整合與系統管理功能**

- 自動偵測本機 IP，顯示系統網址以供快速瀏覽
- 負責管理背景執行緒（感測器輪詢、資料儲存）
- 整合感測層與顯示層的所有資料流程
- 作為整個無人機風場量測系統的主要運算平台

(5) **作為感測層與顯示層之間的資料橋接節點**

Raspberry Pi 5 扮演資料中樞（Data Hub）角色，負責：

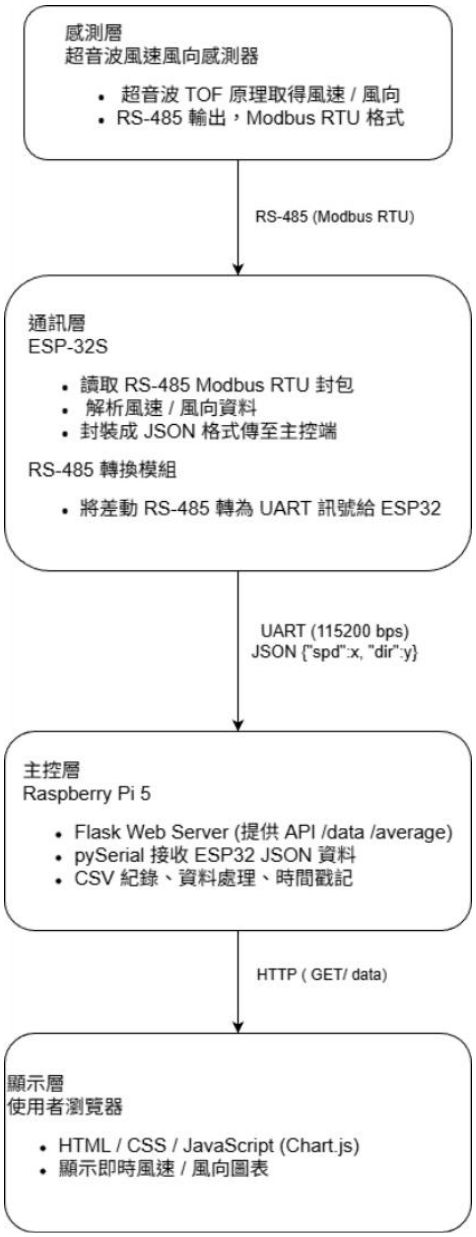
- 接收來自 ESP-32S（通訊層）的風速／風向資料
- 將資料整理為標準化格式
- 轉換為網路可讀格式（JSON）
- 提供瀏覽器端即時視覺化

使整個系統能從無人機端一直到使用者端保持資料同步與一致性。

第二章 系統架構設計

2.1 系統架構設計

本研究所建置之「無人機風場即時量測系統」由感測層、通訊層、主控層及顯示層組成，其整體架構如(圖 12) 所示。系統架構旨在於無人機飛行過程中持續獲取風速與風向資料、即時傳輸至地端運算平台，並透過網頁介面進行視覺化呈現。



(圖 12:系統架構流程圖)

2.1.1 感測層

感測層使用超音波風速風向感測器 進行環境風場量測，由一顆 4000 mAh 6S 的電池供電。

該感測器採用 超音波飛行時間差（Time of Flight, ToF） 原理，能同時提供瞬時風速（m/s）與風向（°）資訊，並以 RS-485 介面輸出 Modbus RTU 格式之資料。

感測器主要規格如下[1]

項目	規格內容
量測範圍	風速：0–60 m/s；風向：0–359°
量測精度	風速：±3%；風向：±3°
分辨率	風速：0.1 m/s；風向：1°
感測器尺寸	直徑 50 mm；高度 50 mm
重量	56 g
輸出介面	RS-485
通訊協定	Modbus RTU / ASCII
波特率	4800–19200 bps（可設定）
電源需求	VDC：5-30V； 15mA(12V)
操作溫度/濕度	−40°C ～ +70°C；0–100 %
工作海拔	0–4000 m
安裝方式	飛行器頂部立住安裝或底部吊裝
外殼材質	ABS/黑色

(表 1：風速計規格)

2.1.2 通訊層

通訊層負責將感測層取得之風速與風向資料可靠地傳輸至主控層，是本系統中確保量測資料穩定性與正確性的核心架構。本研究之通訊層由 RS-485 轉換模組（MAX485） 與 ESP-32S 微控制器 所組成，分別負責差動訊號之電氣轉換、Modbus RTU 通訊、資料解析與重新封裝等功能。RS-485 模組的供電來源跟風速儀是同一顆電池供電，由 Mini Power Hub 分電分成 12V 與 5 V 使用。

本研究使用之超音波風速風向儀以 RS-485 差動式訊號並遵循 Modbus RTU 協定輸出資料。由於 ESP-32S 僅支援 UART（TTL 電平），故需透過 MAX485 模組將 RS-485 差動訊號轉換為 UART 單端訊號，使微控制器能正確讀取感測器之回應封包。

ESP-32S 於通訊過程中擔任 Modbus RTU 主站（Master），定期發送 Function Code 0x03（Read Holding Registers） 之查詢指令，以讀取風速與風向所屬之寄存器資料。查詢前透過控制 MAX485 之 DE/RE 腳位切換至傳送模式，待封包送出後則切換回接收模式以讀取感測器回應資料。此方向切換機制為 RS-485 半雙工通訊之必要條件，可確保查詢與回應封包不互相干擾。

ESP-32S 接收到感測器回應後，會解析寄存器內容，並將風速（m/s）與風向（°）轉換為浮點數格式。解析完成後，ESP-32S 會將資料封裝為 JSON 字串，並透過模組內建之 USB-to-Serial（USB CDC）介面，以 115200 bps 傳送至 Raspberry Pi 5，由主控層進行後續資料處理與儲存。

之所以選擇 RS-485 作為感測層與通訊層之間的資料傳輸介面，基於以下原因：

1. **較強的抗干擾能力**

   RS-485 採差動訊號傳輸，具備強大的 EMI 抑制能力，特別適用於無人機環境中因馬達、電子調速器（ESC）與高電流電源所造成之電磁雜訊。

2. **可靠之長距離與工控等級通訊能力**

   RS-485 可支援數十至數百公尺的傳輸距離，即使本研究中線長不長，其高可靠特性仍可確保資料穩定性。

3. **符合 Modbus RTU 之工業標準實體層**

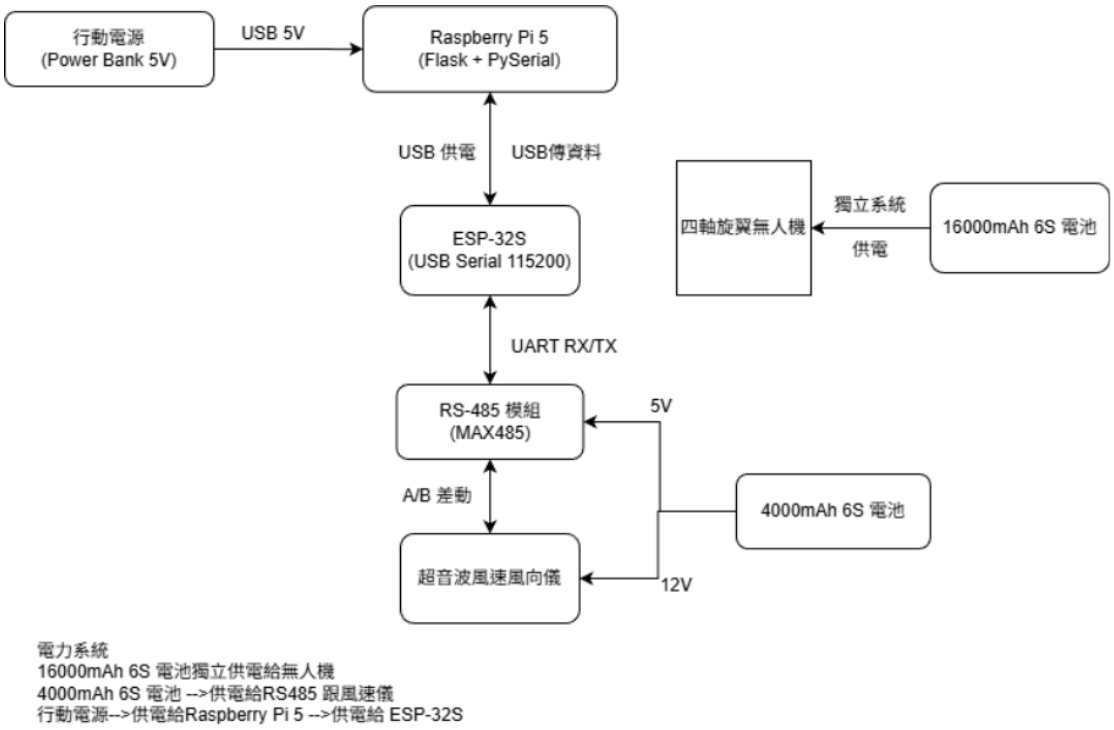
   風速風向儀採 Modbus RTU 作為通訊協定，而 RS-485 為 Modbus RTU 最常見之實體層，有利於減少協定實作複雜度並提高系統穩定性。

4. **降低地電位差與提升電氣安全性**

   差動訊號具備較低電壓擺幅，可有效降低地環路問題並提升系統整體可靠度。

綜合以上設計，本研究通訊層採用「RS-485（Sensor → ESP-32S）＋ USB Serial（ESP-32S → Raspberry Pi 5）」的雙層資料傳輸架構，兼具工控等級穩定性與即時資料更新能力，確保風速與風向量測資料能可靠傳遞至主控層。

下圖(圖 13)展示超音波風速風向儀、RS-485 轉換模組（MAX485）、ESP-32S 與 Raspberry Pi 5 之間的電源分配、資料傳遞方向與實體接線關係。無人機主電池分別供電給 RS-485 模組與風速儀；行動電源提供 Raspberry Pi 運算電力，而 ESP-32S 透過 USB 與 Raspberry Pi 建立序列通訊並同時獲得 5V 電力。資料自風速儀以 Modbus RTU（RS-485）傳至 ESP-32S，再以 USB Serial 傳至 Raspberry Pi 進行解析與視覺化。



(圖 13:硬體接線電力與資料關係圖)

2.1.3 主控層

主控層由 Raspberry Pi 5 所構成，樹莓派由一顆行動電源供電，負責接收通訊層所傳遞之風速與風向資料，進行資料解析、緩衝、即時資料更新、統計計算與資料儲存，並提供資料存取之網路服務。此層整合序列通訊處理、資料管理與網頁伺服器功能，是整體系統資料整合與呈現的核心單元。

Raspberry Pi 5 透過其 USB 端口與 ESP-32S 連接，並以 USB-to-Serial（USB CDC）方式建立 115200 bps 的序列通訊連線。主控層使用 Python 之 pySerial 函式庫持續讀取 ESP-32S 所輸出的 JSON 格式資料，並進行解析與資料驗證。解析成功的資料會即時更新於系統共用變數 last_data 中，並同時推入採樣緩衝區以供後續統計運算使用。

為避免主程序阻塞與確保資料即時性，主控層採用背景執行緒（Thread）進行資料輪詢與存檔作業。背景執行緒以高頻率（50 Hz 輪詢）讀取序列埠資料，並每分鐘自動計算一次平均風速與平均風向。計算結果以 CSV 格式儲存，依日期自動建立檔案，使系統能進行長期量測紀錄與後續統計分析。

Raspberry Pi 5 並以 Flask Web Framework 建立網頁伺服器，提供資料查詢與前端顯示功能。伺服器提供以下 API：

- /data：回傳最新風速（m/s）與風向（°）之 JSON 資料
- /：提供前端視覺化網頁介面（HTML + JavaScript）

透過此設計，使用者可於瀏覽器輸入設定好之網址，<樹莓派 IP>:5000 即可即時查看無人機量測之風場資訊。主控層同時整合 IP 偵測功能，於啟動時自動顯示本機局域網路 IP，以便使用者快速連線查看資料。

整體而言，主控層負責處理資料接收、解析、平均化、儲存與即時對外服務等工作，並作為感測層與顯示層之間的資料中樞，使本風場量測系統能於無人機運作期間提供即時且穩定的風速風向資訊。

2.1.4 顯示層

顯示層負責將主控層處理後的風速與風向資料以視覺化方式呈現，使使用者能即時掌握無人機量測到之風場變化。本研究之顯示層採用基於網頁的呈現方式，主要由 Flask Web Server、HTML 前端介面與 Chart.js 圖表套件所組成。此層提供跨平台、免安裝、低延遲之資料觀測方式，可透過一般桌上型電腦、筆記型電腦或行動裝置進行查看。

顯示層與主控層之間透過 HTTP 標準介面進行資料交換。Raspberry Pi 5 於主控層中建立 /data API，以 JSON 格式傳送最新風速與風向資料。前端介面每隔固定時間（約 1 秒）向此 API 發送請求，取得最新風場資訊，並即時更新於圖表畫面。此方式不需額外安裝應用程式或驅動程式，僅需在瀏覽器中輸入<樹莓派 IP>:5000 即可使用。

前端視覺化部分由 HTML 與 JavaScript 建構，並使用 Chart.js 作為資料繪圖框架。透過 Chart.js，可將風速變化以折線圖方式即時呈現，而風向資料則可進行角度轉換後顯示於極座標或儀表式圖形。此視覺化介面可清楚呈現無人機飛行期間風場的瞬時變化情形，並有助於使用者判讀風況趨勢。

顯示層之整體運作流程如下：

1. 前端 JavaScript 定期向 Flask /data API 發送 AJAX 請求
2. 主控層回傳最新風速（m/s）與風向（°）資料
3. 前端取得資料後即時更新圖表
4. 使用者透過網頁介面查看風場變化

此架構具備以下優點：

- 跨平台性高：可於任何具備瀏覽器的裝置使用
- 即時回饋：資料更新頻率高，延遲極低
- 低成本且易維護：免安裝額外應用程式，部署方式簡單
- 擴充性佳：可新增歷史紀錄查詢、風向玫瑰圖或儀錶板等功能

綜合而言，顯示層提供使用者友善且具即時性的視覺化風場數據介面，使整體系統能有效呈現無人機量測過程中的風速與風向變化情形，並提升系統之可用性與分析能力。

2.2 系統流程

本節說明本研究風速風向量測系統自啟動、資料讀取、傳輸、解析、儲存與視覺化顯示之完整運作流程。系統由感測層、通訊層、主控層與顯示層協同運作，以確保風場數據能穩定、即時且連續產生。

2.2.1 系統啟動與模組初始化

當 ESP-32S 開機後，會首先初始化 USB Serial（供樹莓派接收）、UART2（用於 RS-485 / Modbus 通訊）、ModbusMaster 物件與 RS-485 方向控制腳位（DE/RE）。以下程式碼為 ESP-32S 啟動階段的初始化動作程式碼：

```
Serial.begin(115200);   // 與樹莓派通訊
Serial2.begin(9600, SERIAL_8E1, RXD2, TXD2);   // 與風速計通訊：9600, 8E1
node.begin(1, Serial2); // Modbus ID = 1
pinMode(RE_DE, OUTPUT);
digitalWrite(RE_DE, LOW); // 預設為接收狀態
```

說明：

上述初始化設定包含 USB Serial（115200 bps）、Modbus RTU 參數（9600 bps、8E1）、RS-485 方向控制腳位設定，確保 ESP-32S 能依序進行查詢與接收感測器回應。

2.2.2 Modbus RTU 查詢流程

ESP-32S 以 Modbus Master 身分每秒向超音波風速風向儀發送查詢命令（Function Code 0x03）。研究中讀取的寄存器包含風向（1 寄存器）與風速（2 寄存器）。以下為查詢指令的實際程式碼：

```
uint8_t result_dir = node.readHoldingRegisters(0x0001, 1);
uint8_t result_spd = node.readHoldingRegisters(0x0002, 2);
```

說明：

ESP-32S 傳送查詢時，MAX485 會切換至傳送模式；完成發送後切回接收模式，以符合 RS-485 半雙工通訊特性。

2.2.3 感測器回應資料解析

感測器回傳資料後，ESP-32S 需解析寄存器內容。其中風速以兩個 16-bit Word 表示，需組合為 32-bit 浮點數。以下程式碼負責資料解析：

```
uint16_t high_word = node.getResponseBuffer(0);
uint16_t low_word    = node.getResponseBuffer(1);


union {
  uint32_t i;
```

```
  float f;
} u;
u.i = ((uint32_t)low_word << 16) | high_word;
float spd = u.f;
```

說明：

此段程式碼將低位元組（low_word）與高位元組（high_word）組合成 IEEE 754 32-bit 浮點數，並解析出風速數值（m/s）。

2.2.4 JSON 打包與 USB Serial 資料傳輸

解析完成後，ESP-32S 會將資料封裝為 JSON 格式字串，並經 USB Serial 傳送至 Raspberry Pi 5。以下程式碼負責資料封裝與輸出：

```
Serial.printf("{\"spd\":%.2f,\"dir\":%.1f}\n", spd, dir);   // 標準 JSON
```

說明：

此 JSON 字串由 Raspberry Pi 端之 pySerial 解析，可直接作為前端資料來源，格式統一且易於網頁處理。

2.2.5 Raspberry Pi 端資料接收與緩衝機制

Raspberry Pi 會以非阻塞方式持續讀取 ESP-32S 輸出的 JSON 資料。程式會解析 JSON 並更新最新資料與緩衝區，如下所示：

```
line = ser.readline().decode("utf-8").strip()
d = json.loads(line)
last_data = {"spd": float(d["spd"]), "dir": float(d["dir"])}
samples.append(last_data)
```

說明：

last_data 用於即時顯示；samples 作為 60 秒緩衝區，用於後續平均化計算。

2.2.6 每分鐘平均化與資料紀錄

系統每分鐘自動計算平均風速與風向，並寫入 CSV 作為實驗紀錄。以下為平均化與儲存功能：

```
avg_spd = sum(d["spd"] for d in samples) / len(samples)
avg_dir = sum(d["dir"] for d in samples) / len(samples)


with open(filename, "a") as f:
    w = csv.writer(f)
    w.writerow([timestamp, avg_spd, avg_dir])
```

說明：

此機制可產生包含時間戳記的日誌檔案，供後續分析與比對使用。

2.2.7 Flask API 資料提供流程

Raspberry Pi 以 Flask 建立 API，前端透過 AJAX 取得最新資料。以下為 /data 路由之程式碼：

```
@app.route("/data")
def data():
    return jsonify(last_data)
```

說明：

此 API 提供前端每秒即時更新 Chart.js 所需之資料來源。

2.2.8 前端資料顯示與 Chart.js 更新流程

前端以 JavaScript 每秒向 /data 發送請求，更新圖表。例如：

```
setInterval(function() {
    fetch("/data")
        .then(r => r.json())
        .then(d => updateChart(d));
}, 1000);
```

說明：

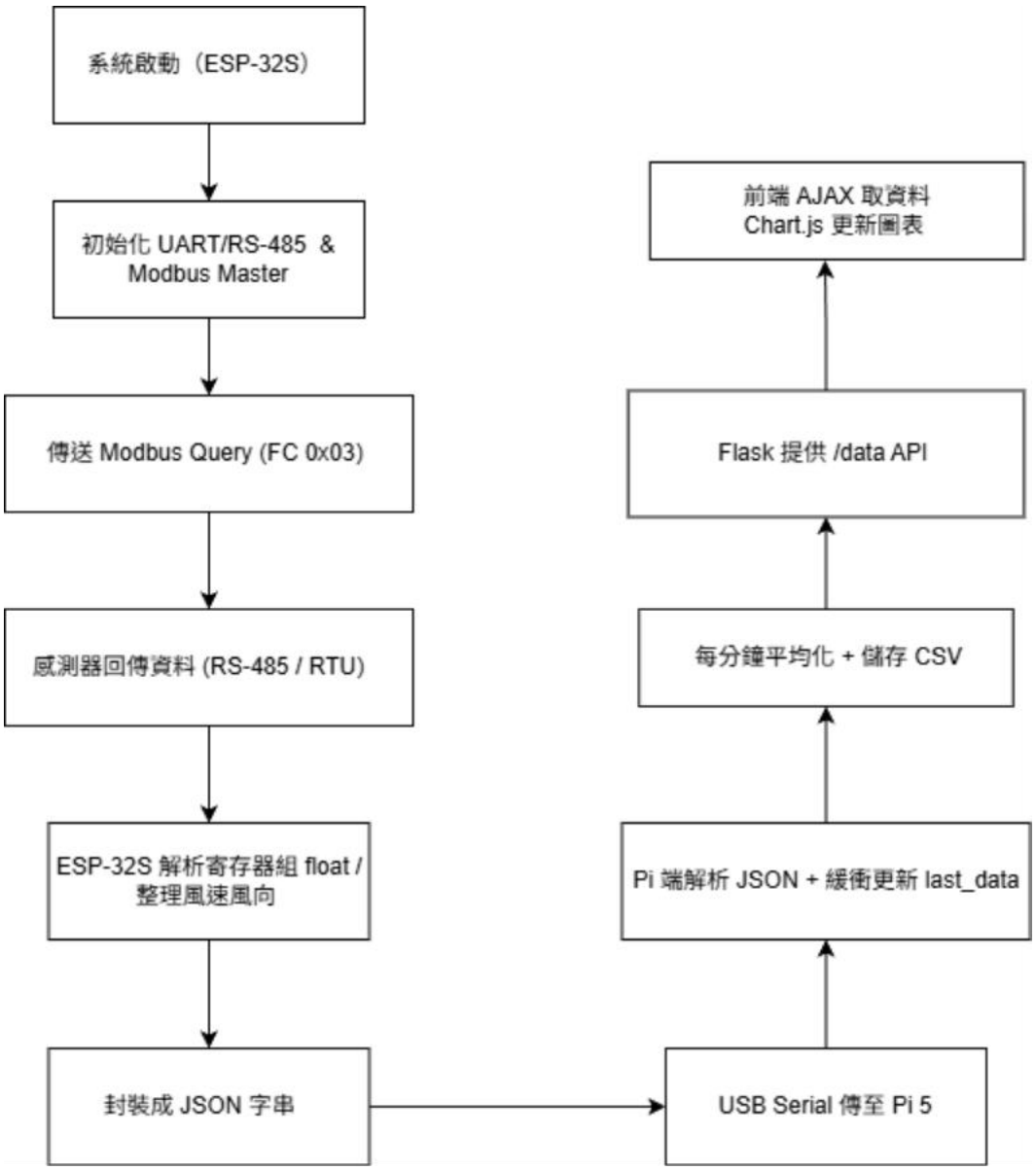
前端會根據最新資料更新折線圖或其他可視化介面，實現即時資料顯示功能。

2.2.9 系統流程總結

綜合上述步驟，本研究之資料流程由「感測 → 通訊 → 主控 → 顯示」構成完整閉環。流程圖可視化呈現於圖 3-X，包含：

- Modbus RTU 查詢
- RS-485 半雙工資料流
- ESP-32S JSON 打包
- USB Serial 傳輸
- Raspberry Pi 資料接收、緩衝與儲存
- Flask API 與前端即時呈現

以下流程圖(圖 14)說明系統自 ESP-32S 啟動後，依序完成 Modbus RTU 查詢、資料解析、JSON 打包、USB Serial 傳輸、Raspberry Pi 解析與儲存，以及透過 Flask 與前端網頁進行即時視覺化之整體資料處理流程。



(圖 14:完整資料處理流程)

2.3 系統實作

本節將說明本研究無人機風場量測系統之實際硬體整合與軟體架構實作流程。硬體部分涵蓋無人機平台、動力系統、感測器與通訊模組之組裝安裝；軟體部分則包含 ESP-32S 與 Raspberry Pi 5 之程式開發、資料解析與即時顯示功能。以下將分別介紹硬體與軟體實作內容。

2.3.1 硬體實作

本研究之風場量測系統建置於四軸旋翼無人機平台。整體硬體整合包含動力系統、飛控模組、ES P-32S 通訊模組、MAX485 RS-485 轉換模組、超音波風速風向儀與 Raspberry Pi 5。本節依照實際組裝流程，說明各硬體模組之安裝方式與整合結果。

(1) 機體與動力系統組裝

無人機採用碳纖材質機架搭配避震機臂，可降低馬達振動傳遞至機體中央艙位，提升飛控穩定性並避免干擾感測器量測。

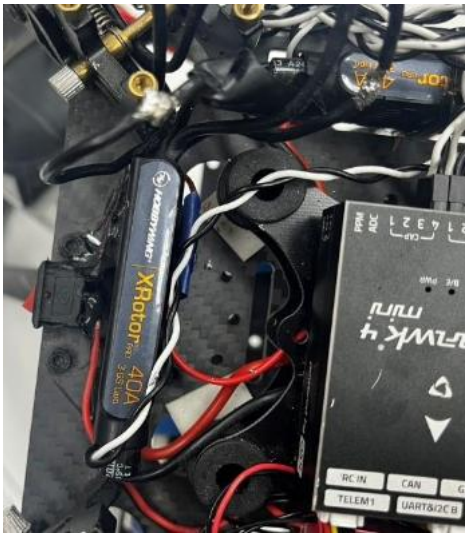
動力系統使用 EaglePower LA4312 KV350 無刷馬達，具備高扭力與低 KV 值特性，適合搭載額外量測模組並維持穩定升力。馬達固定於鋁合金機臂端部，以螺絲與止滑墊片固定，確保高速旋轉下之結構強度。



(圖 15:EaglePower LA4312 KV350 無刷馬達安裝)

(2) 電子調速器（ESC）配置

本研究採用 Hobbywing XRotor 40A 電子調速器（ESC） 作為無刷馬達之驅動器。ESC 安裝於機臂內側並以熱溶膠固定，三相線連接至馬達端，而電源線則連接至主電池分電板。此配置可降低阻力干涉，並確保 ESC 在通風環境下正常散熱。

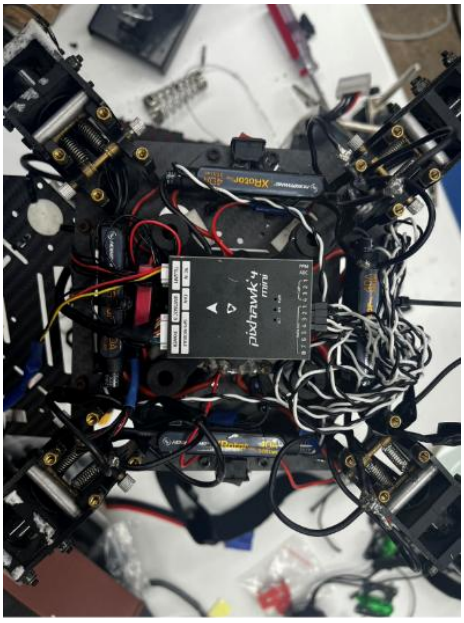


(圖 16:ESC 與電線配置示意)

(3) 飛控模組安裝

無人機使用 Pixhawk 4 Mini 作為主要飛控模組，安裝於中央減震板上，以減震球隔離結構震動，使 IMU 能穩定運作。Pixhawk 與四組 ESC 以 PWM 訊號線連接，同時外接：

- GPS/Compass
- Telemetry 數傳（433 MHz）
- RC 接收器



(圖 17:飛控安裝示意)

(4) 電源分配與供電架構

本研究之無人機風場量測系統採用三組獨立電源，以提升飛行安全性與資料量測穩定性。三組電源分別供應動力系統、感測系統與資料處理模組，各自負責不同子系統，以避免負載干擾與電氣雜訊影響。

主動力電池（約 16000 mAh Li-Po）

主動力電池為無人機的主要能源，提供高電流供應給四組 ESC 與無刷馬達，同時透過飛控電源模組（或 ESC BEC）為 Pixhawk 4 Mini 提供穩定電壓。此電池負責飛行控制與動力輸出，是整體飛行的核心電源來源。

感測專用電池（約 4000 mAh Li-Po）

本研究另配置獨立 4000 mAh 鋰電池供應：

- 超音波風速風向儀
- MAX485 RS-485 轉換模組
- 相關訊號調理電路

目的在於避免感測器受到主動力電池電壓變動與 ESC 電磁噪聲（EMI）干擾，確保 Modbus RTU 通訊穩定性，提高風速與風向量測可靠度。

行動電源

Raspberry Pi 5 由行動電源提供獨立 5V 來源，使其不受主動力電池電壓下降影響，避免飛行過程中樹莓派因低壓而自動重啟。ESP-32S 透過 USB 與 Raspberry Pi 相連，同時進行資料傳輸與取得 5V 電源。

此架構具備以下優點：

- Raspberry Pi 運作更穩定
- ESP-32S 不受動力系統干擾
- 資料記錄不會因飛行電壓下降而中斷

2.3.2 軟體實作

本節說明本研究在 ESP-32S 與 Raspberry Pi 5 上之軟體實作，包括 Modbus RTU 讀取程式、JSON 資料封裝、USB 串列資料傳輸、後端伺服器建

立與前端視覺化介面。整個資料流程如圖 3-2 所示，由 ESP-32S 負責感測資料讀取與封裝，樹莓派負責資料解析、儲存與網頁顯示。

(1) ESP-32S 資料讀取與 JSON 封裝程式

ESP-32S 與超音波風速風向儀採用 RS-485 介面通訊，並以 Modbus RTU 協定讀取感測資料。本研究使用 UART2 以 9600 bps、8E1（8-bit data、Even parity、1 stop bit）與 MAX485 模組溝通。

ESP-32S 程式負責：

1. 初始化 ModbusMaster
2. 控制 RS-485 方向腳位（DE/RE）
3. 傳送查詢指令
4. 解析寄存器回傳值（風速浮點數、風向整數）
5. 封裝為 JSON
6. 透過 USB 序列埠傳送至 Raspberry Pi

下列為本研究使用之 ESP-32S 程式碼之關鍵部分

程式碼：

```
#include <ModbusMaster.h>


// ====== 物件與腳位設定 ======

ModbusMaster node;


#define RE_DE 4       // RS485 模組 DE/RE 腳位

#define TXD2 17       // ESP32 UART2 TX

#define RXD2 16       // ESP32 UART2 RX
```

```
// RS485 傳輸前/後的方向控制
void preTransmission()  { digitalWrite(RE_DE, HIGH); }
void postTransmission() { digitalWrite(RE_DE, LOW); }

void setup() {
  Serial.begin(115200);  // 與樹莓派通訊
  Serial2.begin(9600, SERIAL_8E1, RXD2, TXD2);  // 與風速計通訊：9600, 8E1
  node.begin(1, Serial2); // Modbus ID = 1

  pinMode(RE_DE, OUTPUT);
  digitalWrite(RE_DE, LOW); // 預設為接收狀態

  node.preTransmission(preTransmission);
  node.postTransmission(postTransmission);

  delay(1000);
  Serial.println("☑ ESP32 風速計啟動完成，開始傳送資料...");
}

void loop() {
  float spd = NAN;
  float dir = NAN;
```

```
// ======= 讀取風向（地址 0x0001，寄存器數量 1）======
uint8_t result_dir = node.readHoldingRegisters(0x0001, 1);
if (result_dir == node.ku8MBSuccess) {
  dir = node.getResponseBuffer(0);
}

// ======= 讀取風速（地址 0x0002，寄存器數量 2）======
uint8_t result_spd = node.readHoldingRegisters(0x0002, 2);
if (result_spd == node.ku8MBSuccess) {
  uint16_t high_word = node.getResponseBuffer(0);
  uint16_t low_word  = node.getResponseBuffer(1);

  // 正確順序：低位在前
  union {
    uint32_t i;
    float f;
  } u;
  u.i = ((uint32_t)low_word << 16) | high_word;
  spd = u.f;
}

// ======= 防呆與輸出 JSON ======
if (!isnan(spd) && !isnan(dir)) {
```

```
    Serial.printf("{\"spd\":%.2f,\"dir\":%.1f}\n", spd, dir);  // 標準 JSON
  } else {
    Serial.println("{\"spd\":0,\"dir\":0}");  // 若讀取失敗則回傳 0
  }

  delay(1000);  // 每秒輸出一次
}
```

功能說明：

- ModbusMaster 套件初始化：設定裝置位址、通訊介面與校驗方式
- RS-485 DE/RE 控制：確保發送與接收方向正確
- 寄存器讀取：0x0001 為風向；0x0002~0x0003 為浮點數風速
- 浮點數合併：Modbus 回傳兩組 16-bit，程式需重新組合成 float
- JSON 輸出：每秒產生 {"spd": x, "dir": y} 資料
- USB Serial 傳輸：ESP-32S 經由 USB 送至 Raspberry Pi 進行處理與顯示

(2) Raspberry Pi 5 串列接收與資料處理程式

Raspberry Pi 5 透過 pySerial 以 115200 bps 接收 ESP-32S 輸入之 JSON 字串。

後端程式負責：

1. 解析 JSON
2. 更新最新資料
3. 累積每分鐘的風速／風向樣本
4. 每分鐘計算平均值
5. 寫入 CSV
6. 提供 Flask API 供前端介面讀取

7.

Raspberry Pi 資料接收與處理程式（Python）

程式碼：

```
# ================================
# 風速風向 Flask Server（完整穩定版）
# ================================
from flask import Flask, render_template, jsonify
import serial, json, time, csv, os, threading, socket
from datetime import datetime

app = Flask(__name__)

# ---------- 串口設定 ----------
# 依你目前環境：ESP32 用 /dev/ttyUSB0，115200
# 若不同可改成 /dev/ttyACM0 或 Windows 的 COMx
SERIAL_PORT = os.environ.get("WIND_SERIAL", "/dev/ttyUSB0")
BAUD = 115200
ser = None

def _open_serial():
    global ser
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD, timeout=1)
```

```
            print(f"☑ Serial opened on {SERIAL_PORT} @ {BAUD}")
        except Exception as e:
            ser = None
            print(f"❌ Serial open failed: {e}")

_open_serial()

# ---------- 全域資料 ----------
last_data = {"spd": 0.0, "dir": 0.0}
samples = []
data_lock = threading.Lock()

# ---------- 工具：取得本機 IP ----------
def get_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except:
        return "127.0.0.1"
```

```
# ---------- 感測資料讀取（單筆） ----------
def read_one():
    global last_data, ser
    if ser is None or not ser.is_open:
        _open_serial()
        return

    try:
        line = ser.readline().decode("utf-8", errors="ignore").strip()
        if not line:
            return

        # 僅接受完整 JSON
        if not (line.startswith("{") and line.endswith("}")):
            # print("⚠ 忽略不完整資料:", line)
            return

        d = json.loads(line)
        if "spd" in d and "dir" in d:
            with data_lock:
                last_data = {"spd": float(d["spd"]), "dir": float(d["dir"])}
                samples.append(last_data)
        # else:
```

```
            #         print("⚠ JSON 缺少欄位:", d)

        except json.JSONDecodeError:
            # print("❌ JSON decode failed:", line)
            pass
        except Exception as e:
            print("❌ read_one error:", e)

# ---------- 每分鐘平均存檔 ----------
def save_average():
    global samples
    with data_lock:
        if not samples:
            return
        avg_spd = sum(d["spd"] for d in samples) / len(samples)
        avg_dir = sum(d["dir"] for d in samples) / len(samples)
        samples = []

    if not os.path.exists("data"):
        os.makedirs("data")
    filename = f"data/wind_{datetime.now().strftime('%Y-%m-%d')}.csv"
    write_header = not os.path.exists(filename)
```

```
    with open(filename, "a", newline="") as f:
        w = csv.writer(f)
        if write_header:
            w.writerow(["timestamp", "avg_speed(m/s)",
"avg_direction(deg)"])
        w.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                    round(avg_spd, 2), round(avg_dir, 1)])
    print(f"📦 Saved avg: {avg_spd:.2f} m/s, {avg_dir:.1f}°")


# ---------- 感測讀取背景執行緒 ----------
def sensor_loop():
    last_save = time.time()
    while True:
        read_one()
        if time.time() - last_save > 60:
            save_average()
            last_save = time.time()
        time.sleep(0.02)  # 50Hz 輕量輪詢


# ---------- Flask 路由 ----------
@app.route("/")
def index():
    return render_template("index.html")  # templates/index.html
```

```
@app.route("/data")
def data():
    with data_lock:
        d = dict(last_data)
    return jsonify(d)


# ---------- 入口 ----------
if __name__ == "__main__":
    ip = get_ip()
    print(f"🌐 Open http://{ip}:5000（你的 IP 是 {ip}，你提供的是 172.20.10.6 也可以）")

    # 啟動感測背景執行緒
    threading.Thread(target=sensor_loop, daemon=True).start()

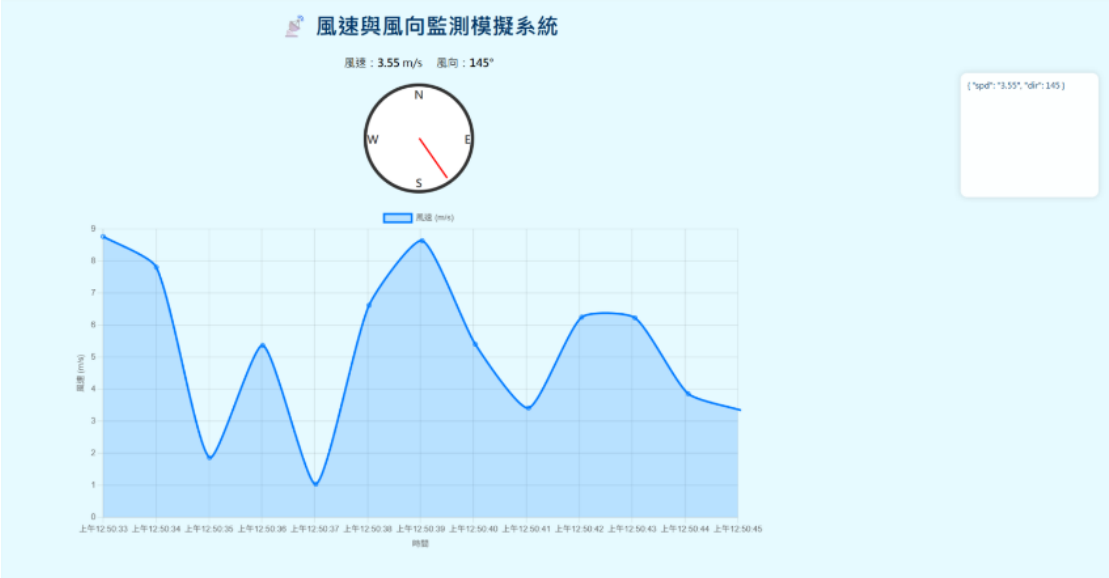
    # 啟動 Flask（對外可訪問）
    app.run(host="0.0.0.0", port=5000, debug=False)
```

（3）Flask 前端即時顯示介面

Raspberry Pi 5 執行 Flask 伺服器，前端網頁使用 JavaScript 以固定週期請求 /data API，並更新頁面上的風速（m/s）與風向（°）資訊。此設計可於無網路環境下直接透過樹莓派提供的 Wi-Fi 或區域 IP 進行監看。

前端網頁結合：

- AJAX 資料請求
- Canvas 或 SVG 即時指標顯示
- m/s 與 degree 數值更新
- 簡潔 UI



(圖 18:前端網頁模擬圖)

2.4 系統測試與驗證

本節針對本研究所建立之無人機風場量測系統進行功能性與穩定性測試，驗證 RS-485 通訊、感測資料讀取、USB 資料傳輸、後端伺服器運作及即時前端顯示功能是否符合需求。測試內容包含通訊層驗證、資料完整性驗證、系統穩定度測試以及實際量測功能驗證。

2.4.1 RS-485 通訊測試

為確認 ESP-32S 能正確與超音波風速風向儀進行 Modbus RTU 通訊，本研究於地面進行 RS-485 層級之初步驗證。
測試步驟如下：

1. 將 ESP-32S 與 MAX485 模組連接風速風向儀之 RS-485 A/B 端子
2. 設定 UART2 通訊參數為 9600 bps、8E1

3. 以 ModbusMaster 套件發送 Holding Register 查詢指令

4. 監看風向（0x0001）與風速（0x0002~0x0003）是否成功回傳

測試結果顯示 ESP-32S 能穩定取得感測器資料，且查詢周期為 1 秒時無封包遺失情形。此結果證實本系統之 RS-485 實體層與 Modbus RTU 通訊協定均能正常運作。

2.4.2 ESP-32S JSON 封裝與 USB 資料傳輸測試

確認 RS-485 通訊正常後，本研究進一步驗證 ESP-32S 將讀取結果封裝為 JSON 字串並透過 USB 傳送至 Raspberry Pi 之運作情況。

測試內容包含：

- JSON 格式完整性（是否每秒產生合法字串）
- USB 連線持續性（長時間運作是否會中斷）
- 樹莓派端 pySerial 解碼驗證

測試結果顯示：

- ESP-32S 能以固定頻率輸出格式正確的 JSON，如：
  { "spd": 1.24, "dir": 85.0 }
- USB 傳輸穩定，可連續運行多小時未出現斷線
- 樹莓派能完整接收並解析每筆資料

此結果證實 ESP-32S → USB → Raspberry Pi 之資料路徑運作正常。

2.4.3 後端伺服器與資料處理功能驗證

為驗證 Raspberry Pi 5 上 Flask 後端之資料處理流程，本研究進行以下測試：

（1）即時資料更新測試

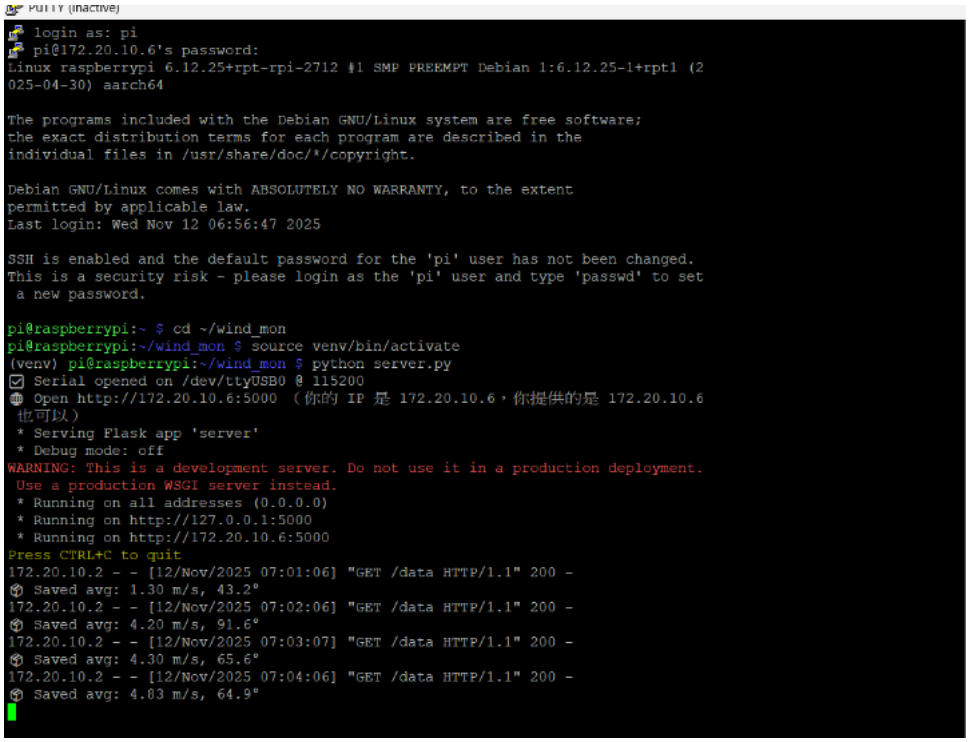
- 執行 python server.py 後，樹莓派成功啟動 Flask 伺服器
- 前端頁面於 /data API 每秒取得最新 JSON

- 當風速儀改變方向或風速時，前端可立即更新數值

（2）資料緩衝與平均化測試

- 樹莓派每 60 秒統計 samples 緩衝區
- 計算風速與風向平均值
- 每分鐘寫入 CSV

測試結果顯示 CSV 中之時間戳與平均值均與實際量測相符。



(圖 19:Raspberry Pi 5 啟動 server.py)

2.4.4 CSV 資料寫入測試

為確認資料可長期保存，本研究執行以下流程：

1. 連續運作系統 20 分鐘
2. 每分鐘生成一筆平均風速與風向資料
3. 開啟 data/wind_YYYY-MM-DD.csv 驗證格式

測試結果如下：

- CSV 檔案成功建立
- 格式正確（timestamp, avg_speed, avg_direction）
- 20 分鐘內共產生 20 筆平均資料
- 未發生寫入錯誤或檔案損毀

此結果驗證資料長期記錄功能可靠。

	A	B	C
1	timestamp	avg_speed(m/s)	avg_direction(deg)
2	2025/11/12 06:24	0.59	121.7
3	2025/11/12 06:25	0.39	241
4	2025/11/12 06:26	0.1	203.7
5	2025/11/12 06:27	0.29	300
6	2025/11/12 06:28	0.25	264.9
7	2025/11/12 06:29	0.51	247.9
8	2025/11/12 06:30	0.7	216.5
9	2025/11/12 06:31	1.09	186
10	2025/11/12 06:32	1.72	226.1
11	2025/11/12 06:33	2.47	214.9
12	2025/11/12 06:35	2.43	217.8
13	2025/11/12 06:36	2.59	221.5
14	2025/11/12 06:37	2.9	204.8
15	2025/11/12 06:38	3.62	107.5
16	2025/11/12 06:39	2.71	141
17	2025/11/12 06:40	4.49	67.3
18	2025/11/12 06:41	4.14	84.1
19	2025/11/12 06:42	4.09	84.8
20	2025/11/12 06:43	4.93	68.2
21	2025/11/12 06:44	4.69	106.3
22	2025/11/12 06:45	1.07	216.9
23	2025/11/12 06:46	0.47	143.5
24	2025/11/12 06:47	0	0
25	2025/11/12 06:48	0	0

(圖 20:確實建立 CSV 檔以及每分鐘的平均數據)

2.4.5 前端顯示功能測試

為確保整體系統能於飛行條件下長時間運作，本研究進行 30 分鐘連續地面運作測試：

- RS-485 通訊無封包遺失
- USB 無斷線
- Flask 仍能正常提供 API

- CSV 寫入正常、檔案未損毀
- ESP-32S 與 Raspberry Pi 無重新啟動情形

結果顯示本系統具備足夠穩定性，可支援長時間風場量測。

第三章 研究方法與實驗設計

3.1 校園 CFD 建模與數值設定

本小節旨在詳細闡述將實際校園建築群轉化為 CFD 可運算模型的整個流程。內容涵蓋了**幾何模型建立**的數據基礎與處理(Blender、blosm、實測數據整合)，網格生成的**品質控制**，以及 OpenFOAM 數值模型的**邊界條件與紊流參數** 設定。這些步驟是確保 CFD 模擬具備物理有效性和數值穩定性的前提。

3.1.1 校園幾何模型建立

本研究利用 **Blender 軟體**作為 CFD 幾何模型的前處理工具，其優勢在於能精確處理複雜的多邊形網格，並提供高度的自定義屬性 [7]。幾何模型建立流程結合了實地測量及衛星數據，具體流程如下:

1. 位置與間隔重建:

   整體校園建築群的相對位置與間隔，是透過開源 GitHub 配件程式（Add-on，blosm[8]）收集 GoogleMap 的衛星影像數據，並將其自動匯入 Blender 中進行初步的平面對齊與地貌重建。

2. 尺寸精度確立:

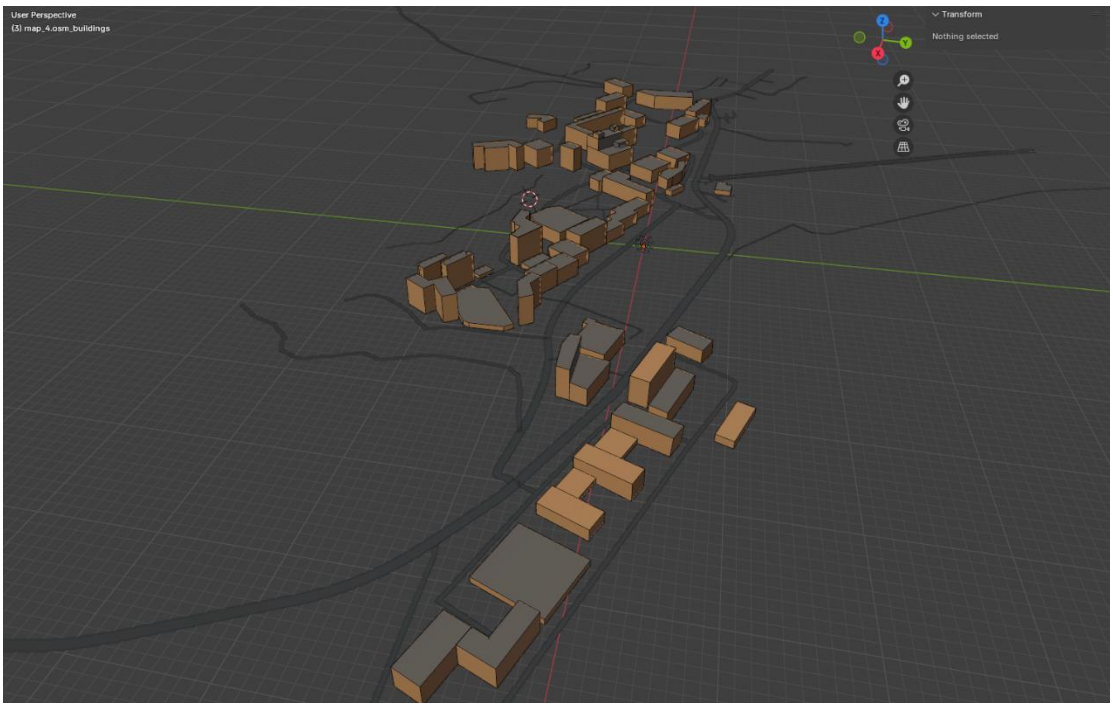
   雖然利用衛星數據提供了位置，但建築物實際的長、寬、高尺寸精度，則透過雷射測距儀進行實地量測 ，以確保模型在垂直方向上的尺寸準確性。



(圖 21:雷射測距儀)

3. 模型簡化:

由於 CFD 網格計算資源有限，我們基於模型簡化原則[9]。所有對流場影響極小的細節物件（如欄杆、植被群等）均被移除。並且利用 Blender 工具對所有建築物底座、屋頂邊緣進行修復和清理，以消除幾何瑕疵（如「非流形邊緣」），解決後續 OpenFOAM 網格生成時遇到的「爆點」問題[10]，確保輸出的 STL 檔案能有效的模擬風場(圖 22)。



(圖 22:海洋大學的校區模擬模型)

3.1.2 數值網格生成與品質控制

網格生成在 CFD 模擬中非常的重要，會直接影響到計算結果的精度和收斂速度[11]，我們採用了 OpenFOAM 中用來處理複雜幾何的網格生成器 snappyHexMesh 來進行六面體的網格生成。

(1.)網格生成策略

1.網格品質調整：

如先前所述，網格品質問題是本研究中最棘手的環節。 由於 Blender 輸出 STL 檔案中會潛藏非流形邊緣，容易導致網格生成器在邊界處難以生成有效單元，我們透過不斷的幾何修復與參數調整，才最終獲得了穩定的網格。

2. 網格規模與品質：

   最終採用的總網格單元數為 237600 個。這個數量是受到個人電腦計算資源限制與流場解析度的考量之後的選擇。

   此模型的網格的最大非正交性為 64.9977(圖 23)，遠低於 CFD 普遍接受的 70 上限[12]。這個數值能代表了幾何前處理是非常成功的，並為後續的數值收斂提供保障。

```
Checking geometry...
    Overall domain bounding box (-700 -700 -20) (950 500 100)
    Mesh has 3 geometric (non-empty/wedge) directions (1 1 1)
    Mesh has 3 solution (non-empty) directions (1 1 1)
    Boundary openness (4.77816e-18 -8.52569e-18 -6.89148e-17) OK.
    Max cell openness = 3.20909e-16 OK.
    Max aspect ratio = 14.8405 OK.
    Minimum face area = 0.243214. Maximum face area = 436.313.  Face area magnitudes OK.
    Min volume = 0.395118. Max volume = 2615.49.  Total volume = 2.36542e+08.  Cell volum
    **Mesh non-orthogonality Max: 64.9977 average: 15.5861**
    Non-orthogonality check OK.
```

(圖 23:模擬輸出的部分內容)

(2.)邊界層網格解析與處理

邊界層網格的解析是 CFD 模擬精度的核心。本研究採用 $k-\omega\,SST$ 紊流模型並結合標準壁函數進行計算，理論上要求壁面 y^+ 值需控制在 30 至 100 的範圍內，然而，受限於專題的計算硬體規格與總網格數量 237600 我們無法在建築物壁面部署足夠細緻的邊界層網格。在運行收斂模擬後，實際量測到的 y^+ 範圍如下：

- 最小值：63734.3
- 最大值：83639.1
- 平均值：71428.6

```
yPlus yPlus write:
    writing field yPlus
    patch bottom y+ : min = 63734.3, max = 83639.1, average = 71428.6
End
```

(圖 24:yPlus 輸出數值)

工程取捨與有效性：

顯然，這個 y^+範圍遠超過標準壁函數的適用上限。這證明了我們的 CFD 模型無法完整模擬建築物貼近壁面的黏滯流動。本研究將此視為在資源限制下，必須接受的工程取捨，我們將有限的網格資源集中於宏觀流場和建築物尾流區的捕捉，放棄了對近壁面摩擦力的精確計算。因此，本專題的模擬結果應被理解為：在簡化的近壁條件下，對校區宏觀氣流趨勢的評估。

重要性： 這個 y^+ 值將會被納入為**第**四章誤差分析中的誤差來源，用於解釋 CFD 模擬值與無人機實測值之間的部分偏差。

3.1.3 OpenFOAM 數值模型與邊界條件

本研究使用 windy 軟體所提供的氣象風速作為輸入，並採用 RANS(Reynolds-Averaged Navier-Stokes)方法進行數值模擬。此方法旨在計算流場的平均特性，以在有限的計算資源下提供穩定的工程評估結果。

(1.)求解器與紊流模型設定

1.求解器：

採用 OpenFOAM 中處理穩態不可壓縮流動的 simpleFoam 求解器。該求解器使用 SIMPLE(Semi-Implicit Method for Pressure-Linked Equations)算法處理壓力與速度的耦合。

2.紊流模型：

核心紊流模型選用$k-\omega\,SST$ (Shear Stress Transport)[13]。

3.優勢：

$k-\omega\,SST$ 模型透過混合函數，在邊界層內使用$k-\omega$模型以提升近壁面的精度，在自由流場中轉換為 $k-\varepsilon$ 模型以確保數值穩定性，是模擬建築物周圍流動分離(Flow Separation)和尾流區(Wake Region) 的最佳工程選擇之一 [14]

4.收斂判據：

數值收斂標準設定為所有物理量（速度 U、壓力 p、紊流動能 k、紊流耗散率ω）的殘差均收斂至10^{-4} 以下。此外，計算域出口的風速監測點變動幅度需小於 1% 時，才判定為穩定收斂。

3.1.4 邊界條件設定與多工況設計

本專題針對 2 級、3 級、4 級風三種工況進行模擬。入口的紊流動能(k)與紊流耗散率(ω)是根據紊流強度(I= 0.1)和混合長度 (L = 0.07) **公式**計算得出[15]。

邊界條件	參數	2 級風工況	3 級風工況	4 級風工況
速度(U)	$U_{ref}(m/s)$	3.0	5.0	7.0
紊流動能(k)	$k_{inlet}(m^2/s^2)$	0.135	0.375	0.735
紊流耗散率(ω)	$\omega_{inlet}(1/s)$	59.390	97.940	137.116
壓力(p)	出/入口條件	零梯度	零梯度	零梯度
壁面(wall)	條件	標準壁函數	標準壁函數	標準壁函數

(表 2：三種風級工況的 OpenFOAM 入口邊界條件)

(1) 壁面與其他邊界條件

1. **壁面條件：**

   建築物與地面壁面採用**無滑移條件(No-slip)**，並結合**標準壁函數**(Standard Wall Function)來處理近壁紊流。我們必須指出，由於 y^+值超出理想範圍，這項設定是本專題在有限的**資源限制下**對近壁面精度限制的工程取捨。

2. **其他邊界：**

   出口採用**零梯度壓力條件**(Zero Gradient)，計算域頂部及側面則採用對稱或滑移壁面條件。

3.2 無人機實測系統與數據採集

本研究的實測部分採用無人機作為高機動性的移動氣象平台，針對 CFD 模擬區域進行數據採集。

3.2.1 無人機平台與感測器集成

實測部分採用無人機作為高機動性的移動氣象平台。我們選擇 **四軸旋翼無人機**作為載具，其優勢是固定翼無人機難以取代的：

(1.)定點懸停與精確定位：

四軸旋翼具備卓越的垂直起降與定點懸停能力。這項優勢是滿足本專題對 CFD 網格中的特定空間坐標進行長時間(60 秒)穩定採集，以實現時間平均化處理的關鍵前提。

(2.)複雜環境下的機動性：

載具擁有高機動性與精確定位能力，使得我們能夠在建築物周圍獲取風場的數據。

(3.)集成設計與擾流處理：

為確保數據真實性，感測器於無人機中央處，用垂直的方式安裝，以最大程度地讓感測器遠離螺旋槳產生的下洗氣流。所有採集數據以每秒的實時風速做一分鐘的總平均，並結合機載 GPS/IMU 數據，用於後續的飛行姿態校正。

3.2.2 實驗設計流程及地點選擇

本研究為了確認 CFD 模型可靠程度且獲得具有代表性的驗證數據。我們的實驗著重於較常遇到的風速工況與數據的統計信賴度。

(1.)實測工況設定：

實測地點選定於延平大樓、生命科學園區、工學院外圍草皮，這些區域涵蓋了全校的範圍來驗證校園整體模擬真實性。我們依據 CFD 模擬的邊界條件，鎖定三種典型風級進行實測，以檢驗模型在不同紊流強度下的性能：

- 輕風工況 (2 級風)：$U_{ref} \approx 3.0\, m/s$
- 微風工況 (3 級風)：$U_{ref} \approx 5.0\, m/s$

- 和風工況 (4 級風)：$U_{ref} \approx 8.0\, m/s$

(2.)重複性設計：

為了確保數據的統計嚴謹性，針對每一種風級，我們各執行三次獨立的重複測量（共 9 組實驗）。這種設計讓我們能夠計算出實驗之間的最終標準差 σ_{final}，這是證明實測數據**可重複性**的關鍵指標[16]。

3.2.3 飛行路徑與數據採集策略

(1.)測量區域與目的：

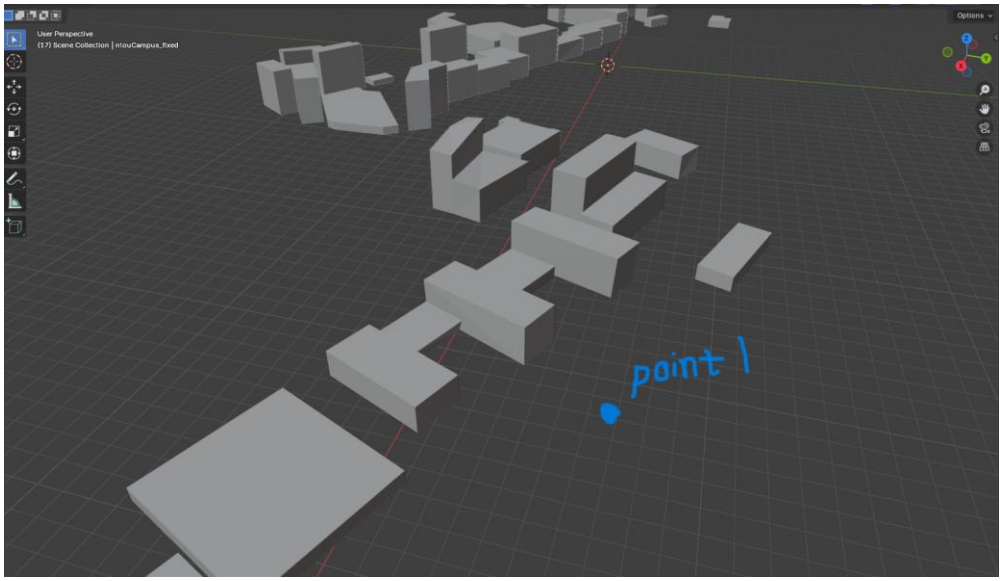
飛行路徑的設計旨在最大化採集數據的驗證價值。我們在相同風速風向下，在三個相對下容易收集的區域位置收集數據，以保證數值的可參考性。

(2.)採集模式：

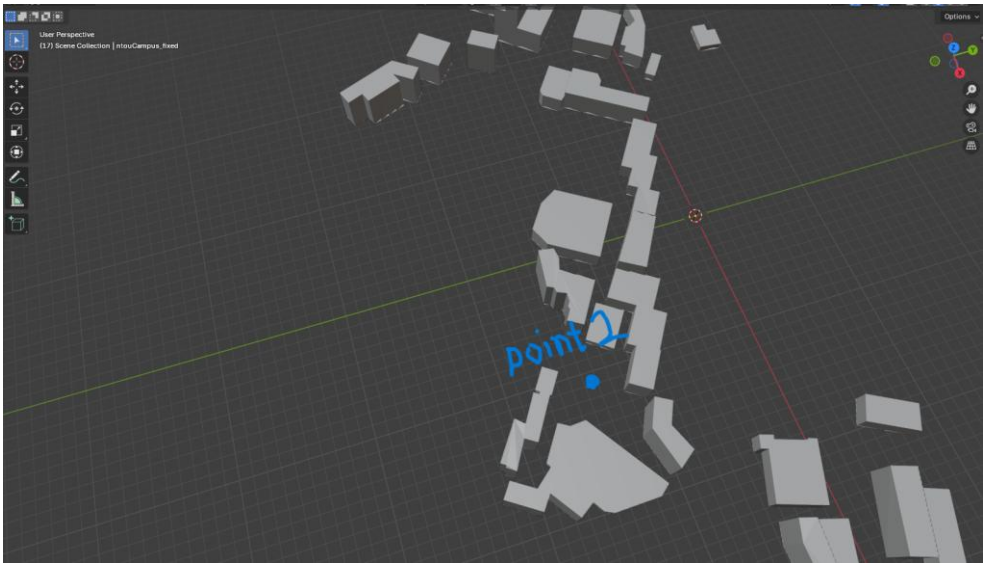
數據採集採用**垂直剖面懸停**模式。在每個測量點位(10m、20m、30m、40m)，無人機將定點懸停(60s)進行高頻數據採集。此 60 秒的採樣長度足以進行**時間平均化**處理，以對應 RANS 模型計算出的定常平均流場[18]。

(3.)飛行點位圖：

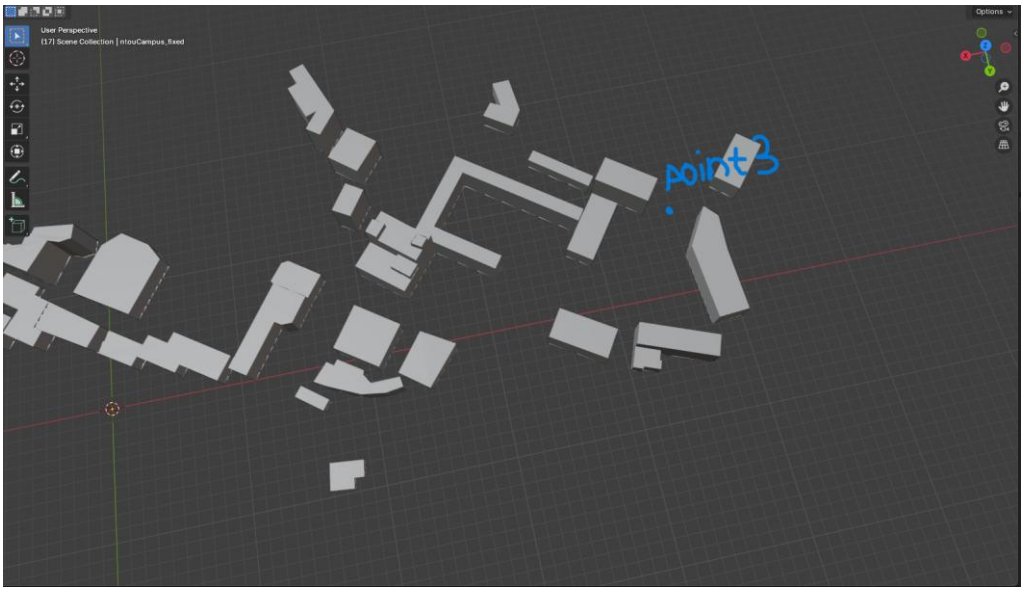
實測點位的分布將呈現於下圖中(圖 25~圖 27)。



(圖 25:實測點位-工學院外圍)



(圖 26: 實測點位-延平大樓旁)



(圖 27: 實測點位-生科院旁)

3.3 數據處理與定量對比方法

本專題的驗證流程核心在於確保 CFD 模擬的定常平均流場，能與無人機實測的瞬態數據進行**嚴謹且有效的定量對比**。數據處理與對比分為以下三個階段：

3.3.1 無人機數據前處理與平均化

為將瞬態實測數據轉化為可與 RANS 模型對比的平均值，我們採用了平均化步驟：

(1.)數據對時間平均：

首先，對所有採集到的數據進行**姿態角校正**，消除無人機自身微小姿態變化帶來的誤差。接著，計算每個點位 60 秒**懸停**期間的平均風速(v_i)與瞬時波動(σ_i)，其中σ_i反映了該點風場的紊流強度。

(2.) 實驗重複平均：

針對同一風級狀況下的**三次獨立實驗結果**(v_1、v_2、v_3)，我們計算其最終平均風速(v_{final})，作為與 CFD 結果對比的最終實測值。同時計算這三個平均值之間的

最終標準差，這個是我們證明實驗設計可重複性的關鍵指標，將用於在圖表中繪製誤差圖。

3.3.2 定量誤差指標計算

數據對齊後，我們將採用以下兩種標準指標，對每個風級工況的 CFD 預測精度進行定量評估：

(1.) 均方根誤差(RMSE)：

用於量化模擬風速v_{sim}與實測平均值 v_{final}之間的平均偏差。

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(V_{sim,i} - V_{final,i}\right)^2} \text{[式-1]}$$

(2.)相關係數(R) ：

採用 Pearson 相關係數，評估模擬數據與實測數據在變化趨勢上的一致。

$$R = \frac{\sum_{i=1}^{N}\left(V_{sim,i}-V_{sim}\right)\left(V_{final,i}-V_{final}\right)}{\sqrt{\sum_{i=1}^{N}\left(V_{sim,i}-V_{sim}\right)^2 \sum_{i=1}^{N}\left(V_{final,i}-V_{final}\right)^2}} \text{[式-2]}$$

第四章 實驗結果與討論

4.1 CFD 數值模擬結果展示

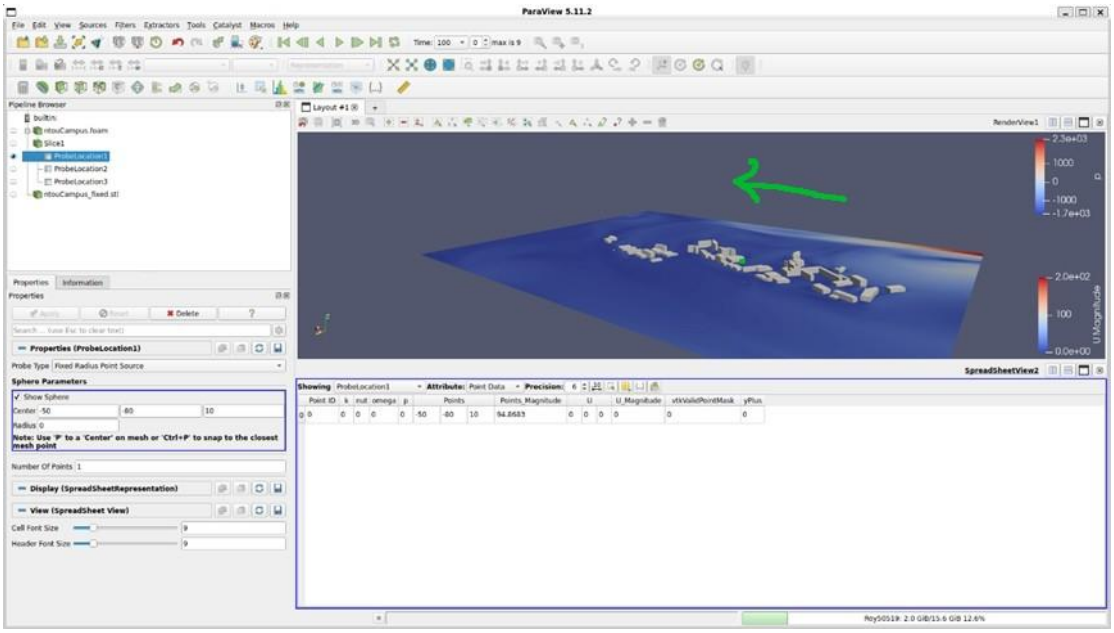
本小節旨在呈現 OpenFOAM 數值模型在**三種典型風級工況**下所計算出的宏觀流場結構。這些模擬結果是本專題進行無人機實測數據對比的**理論基準**。所有結果均基於 **3.1 節**所定義的計算參數和 **3.3 節**所設定的入口邊界條件。我們將首先從宏觀角度解析風場的關鍵特徵，例如迴流區、加速區與流線分佈，為後續 4.4 節的實驗結論及建議奠定基礎。

4.1.1 三級風下各點位風場數值

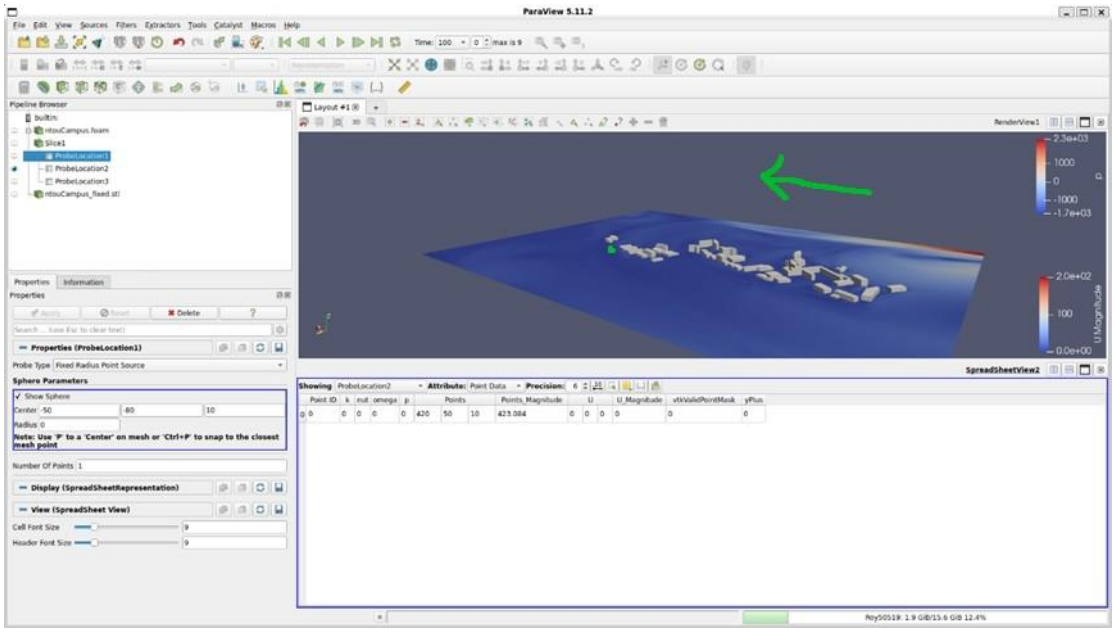
三種風速下各點位風速圖與實測點數值如下圖所示:

(1.)2 級風(3m/s):

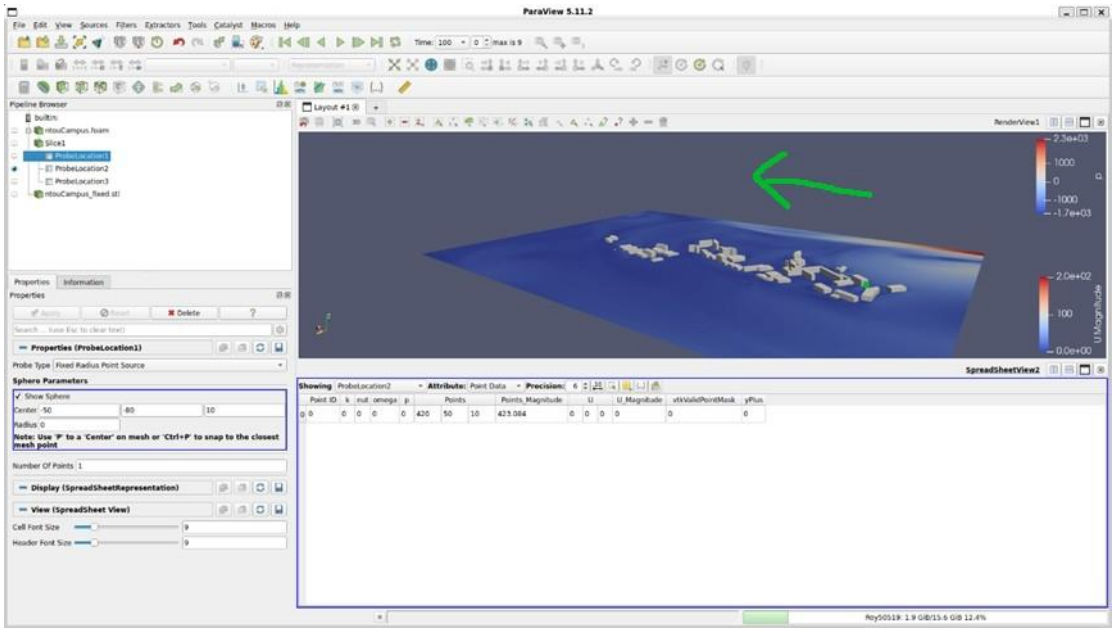
1.10 公尺



(圖 28:延平大樓旁風速圖)

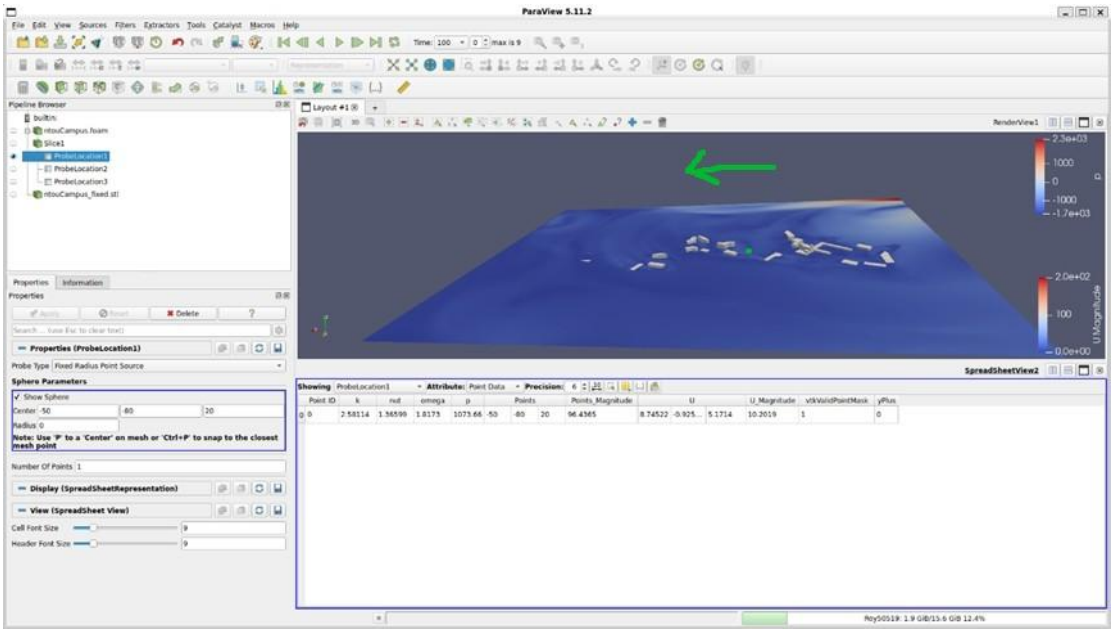


(圖 29:工學院旁風速圖)

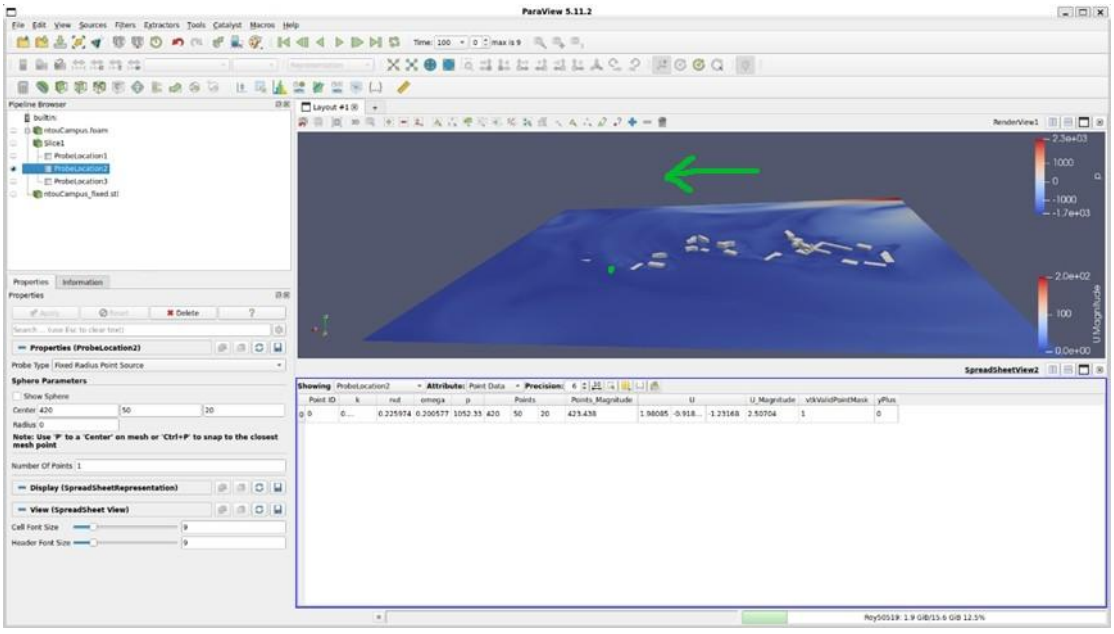


(圖 30:生科院旁風速圖)

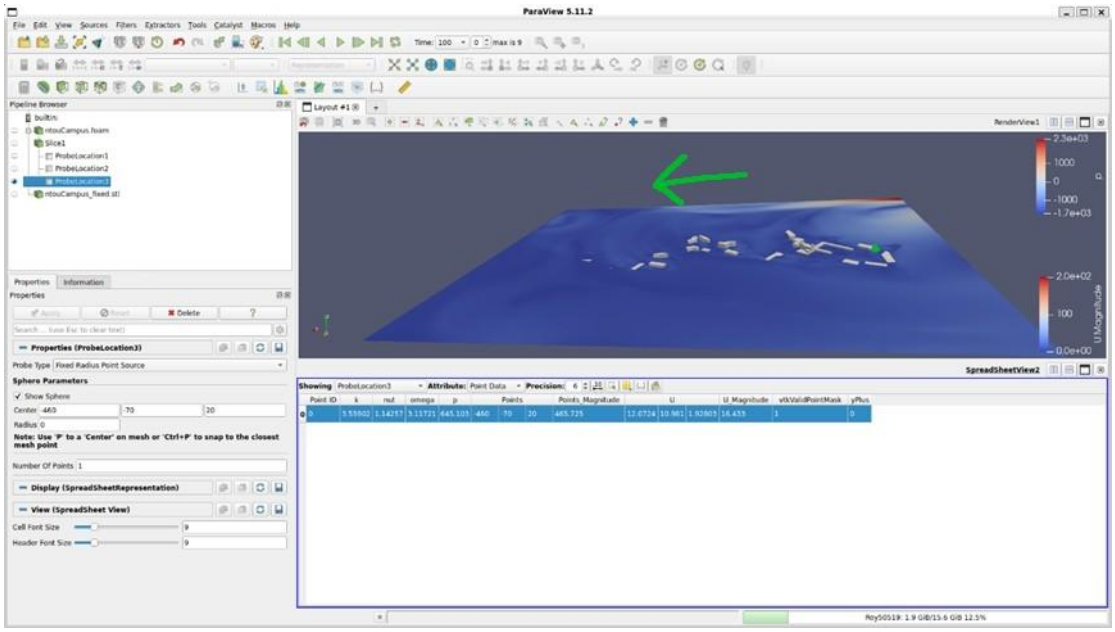
2. 20 公尺高:



(圖 31:延平大樓旁風速圖)

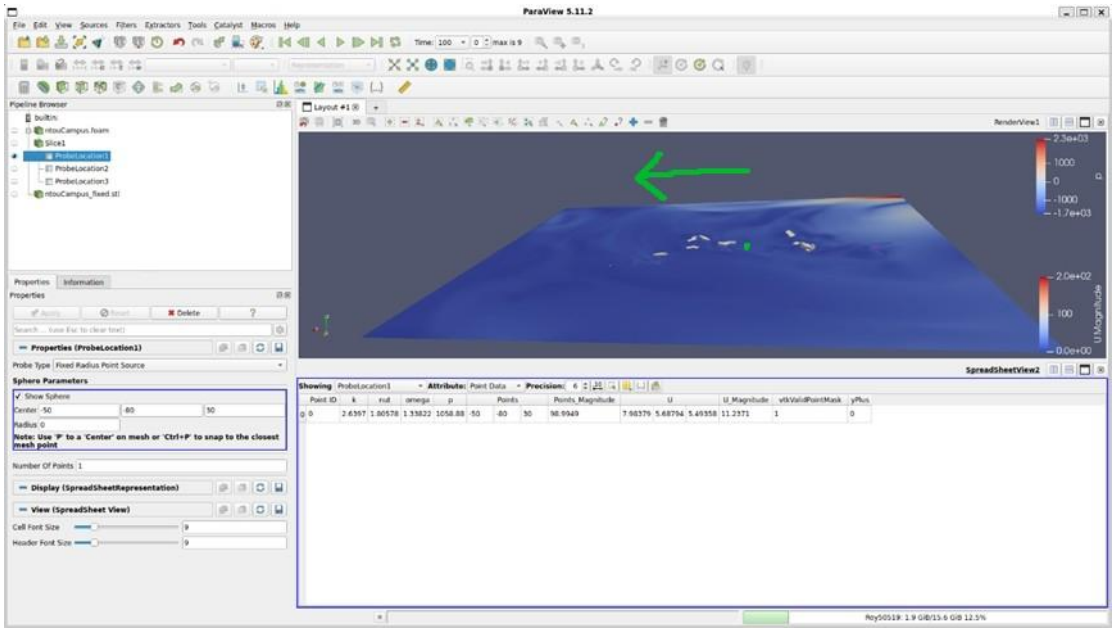


(圖 32:工學院旁風速圖)

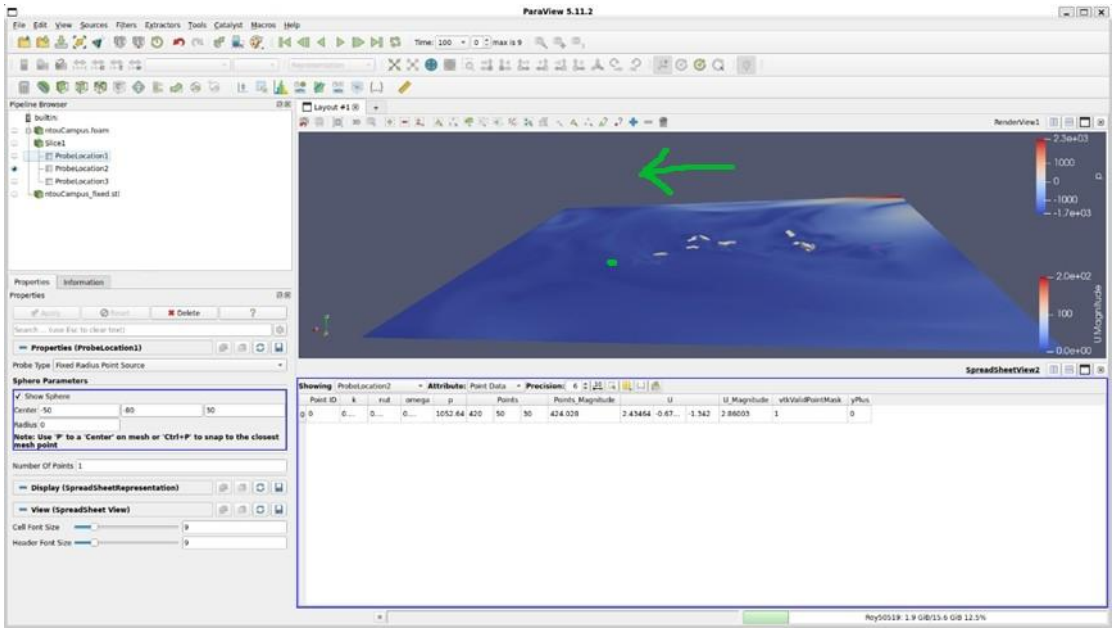


(圖 33:生科院旁風速圖)

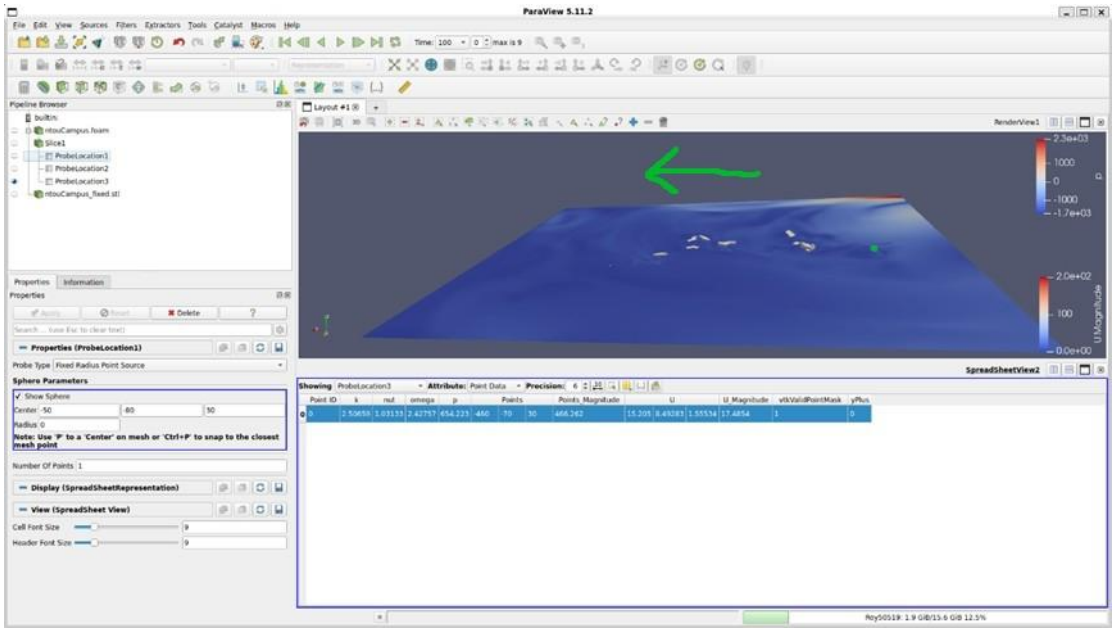
3. 30 公尺高:



(圖 34:延平大樓旁風速圖)

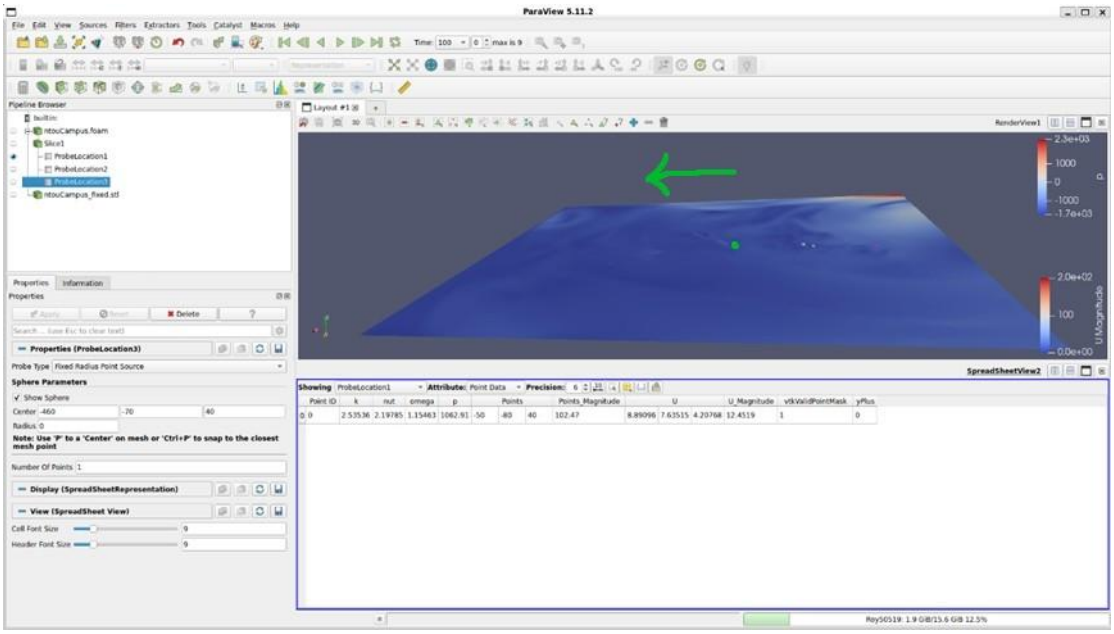


(圖 35:工學院旁風速圖)

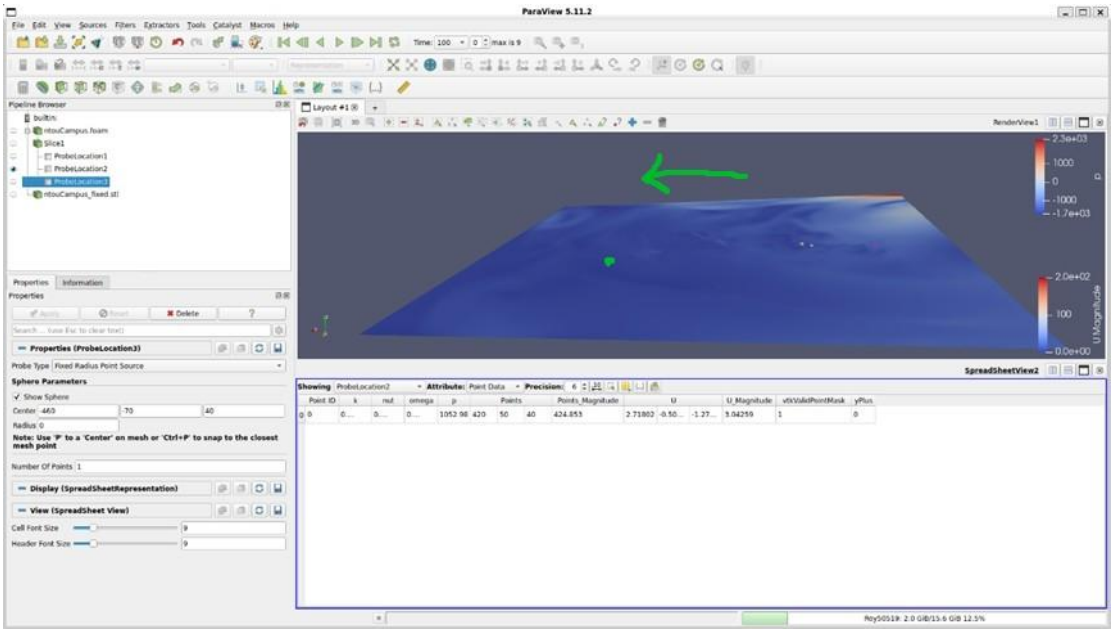


(圖 36:生科院旁風速圖)

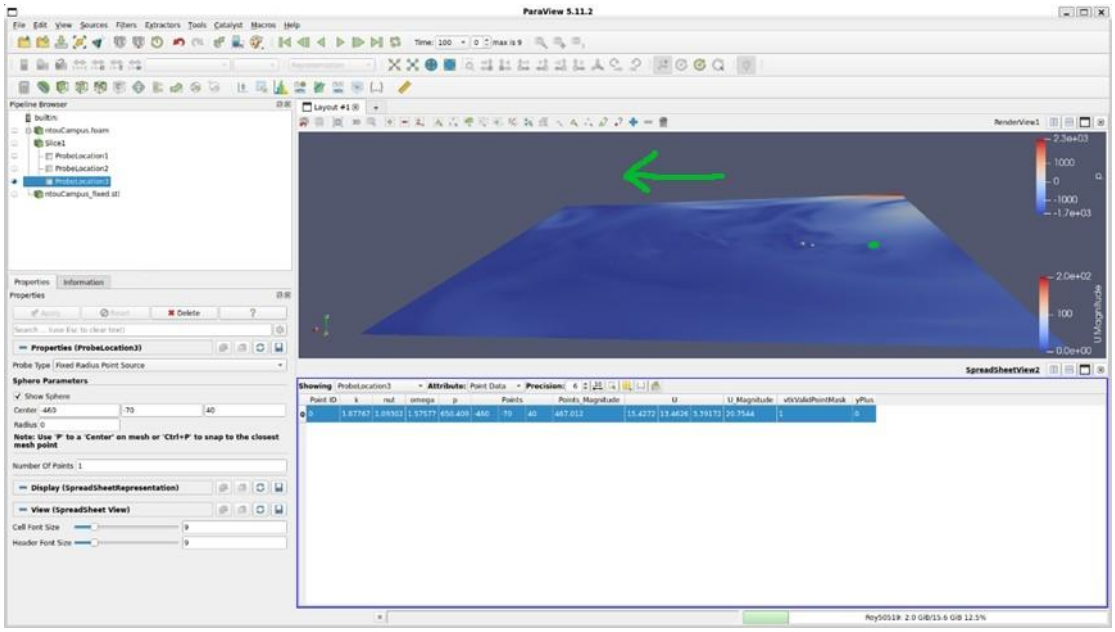
4. 40 公尺高:



(圖 37:延平大樓旁風速圖)



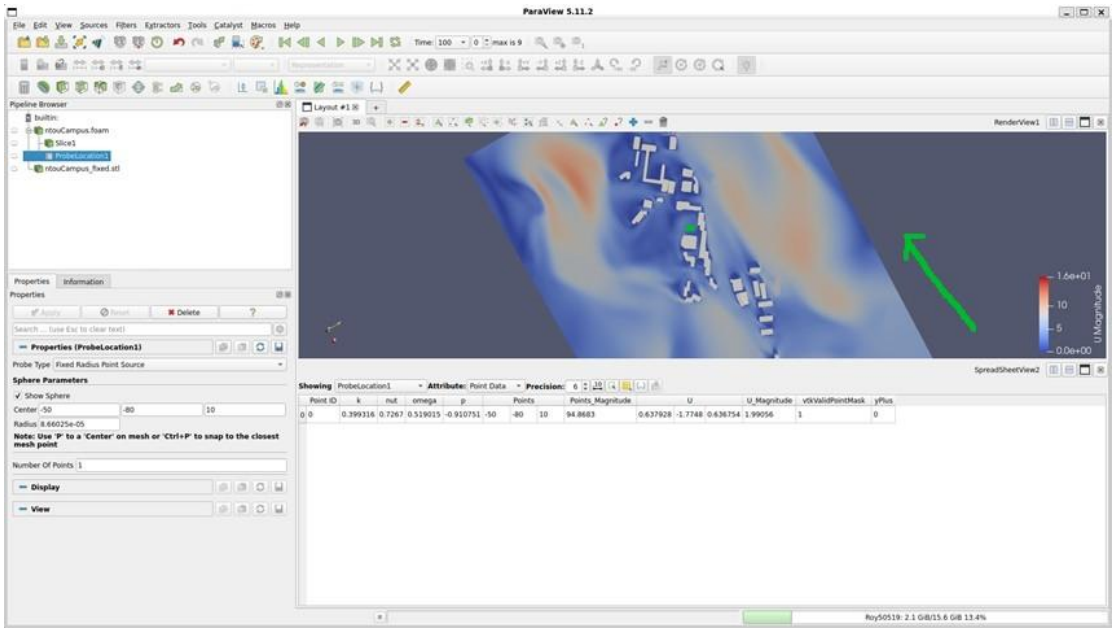
(圖 38:工學院旁風速圖)



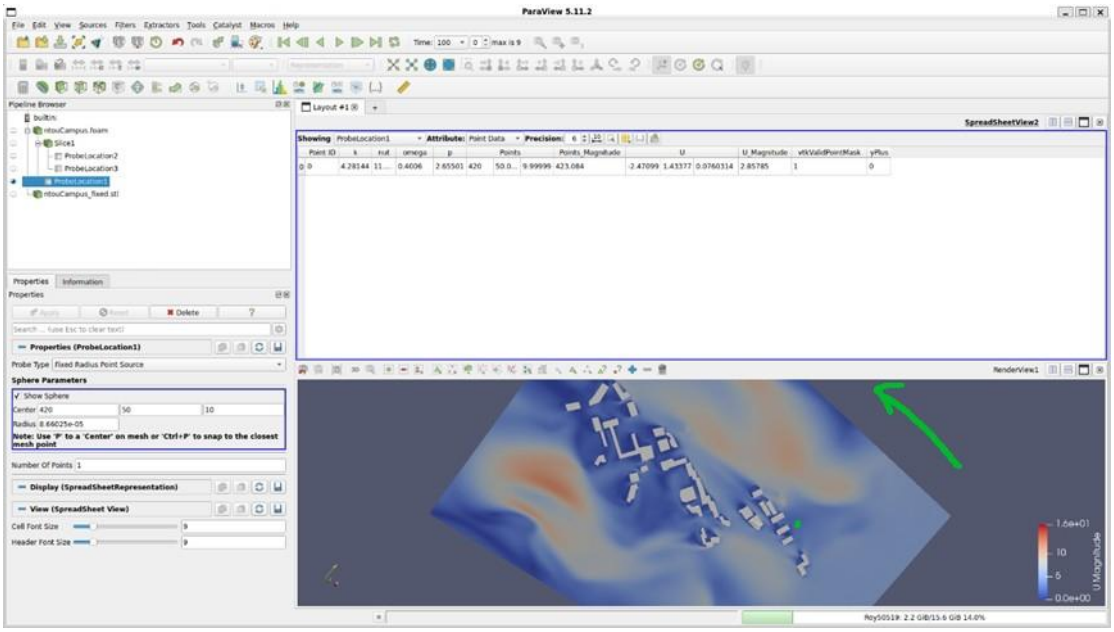
(圖 39:生科院旁風速圖)

(2.)3 級風(5m/s):

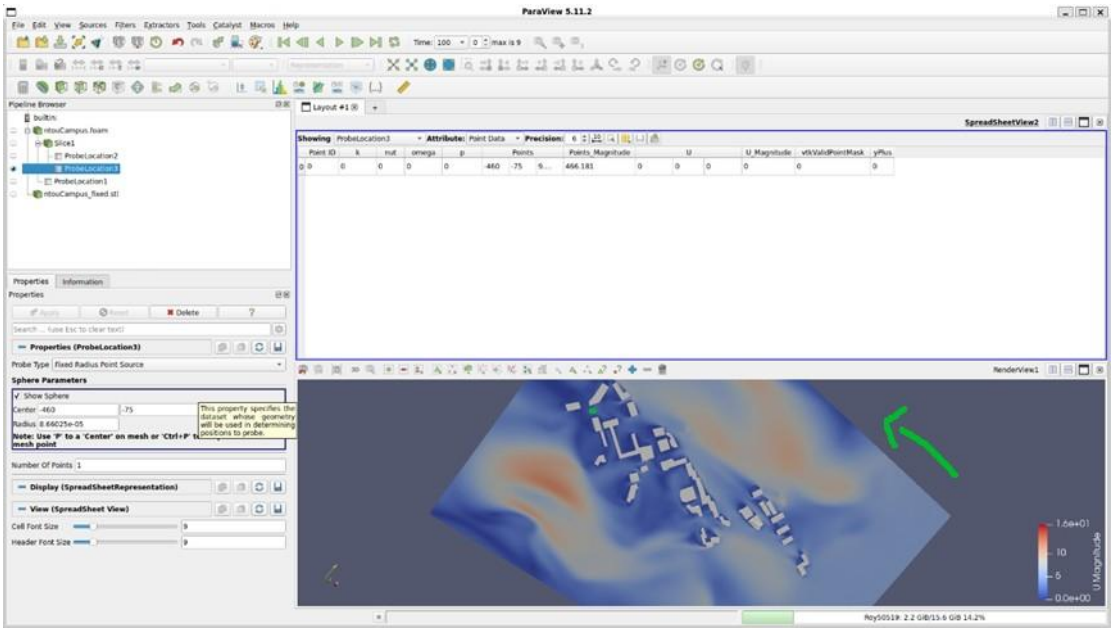
1. 10 公尺高:



(圖 40:延平大樓旁風速圖)

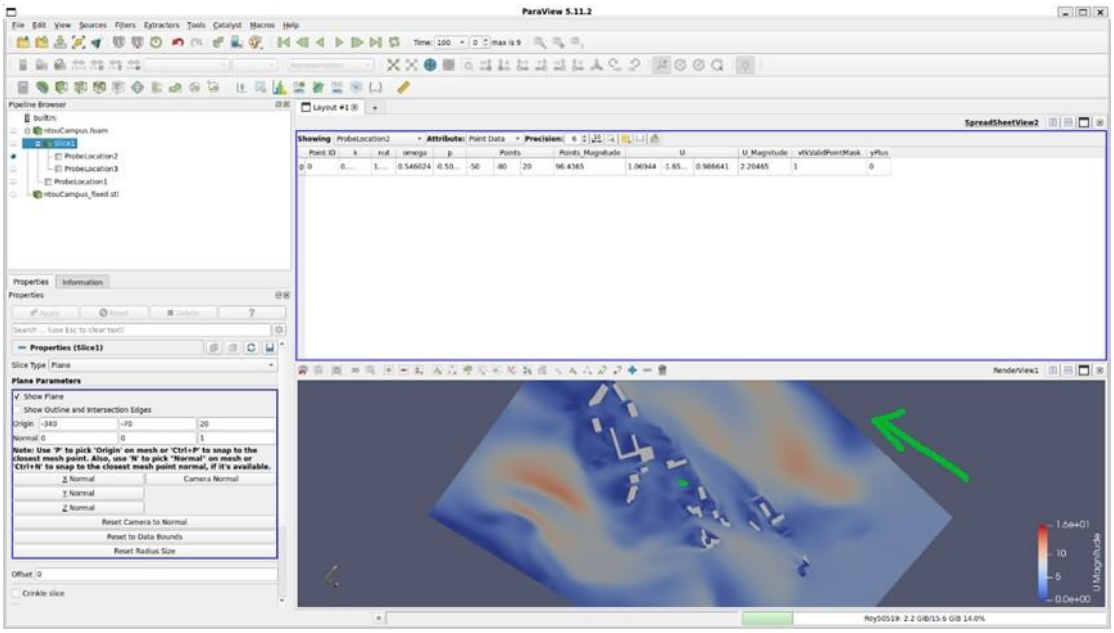


(圖 41:工學院旁風速圖)

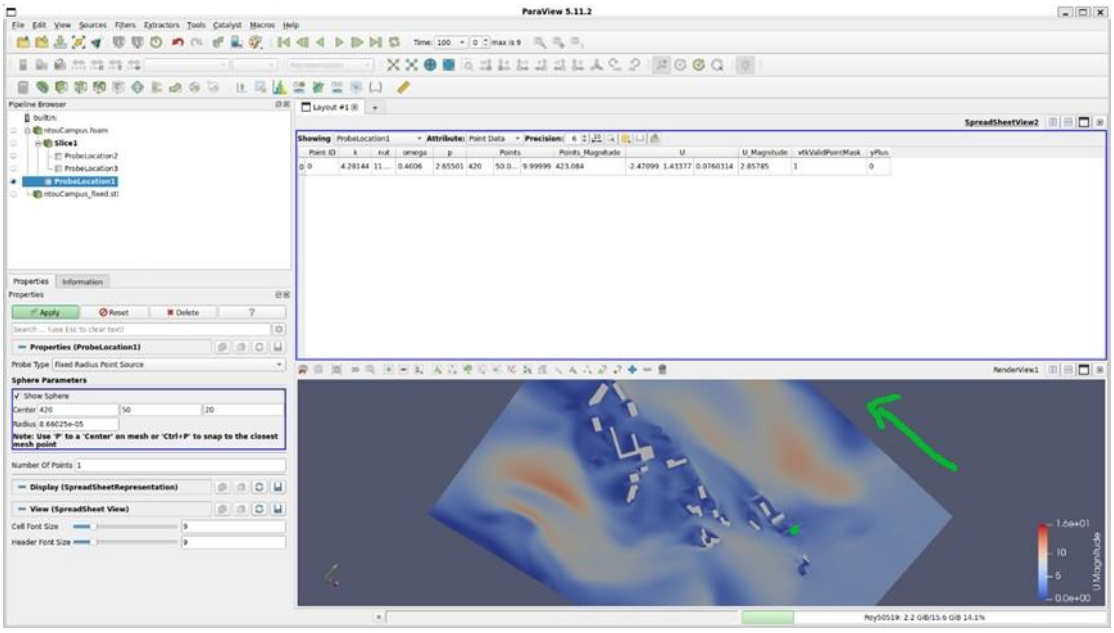


(圖 42:生科院旁風速圖)

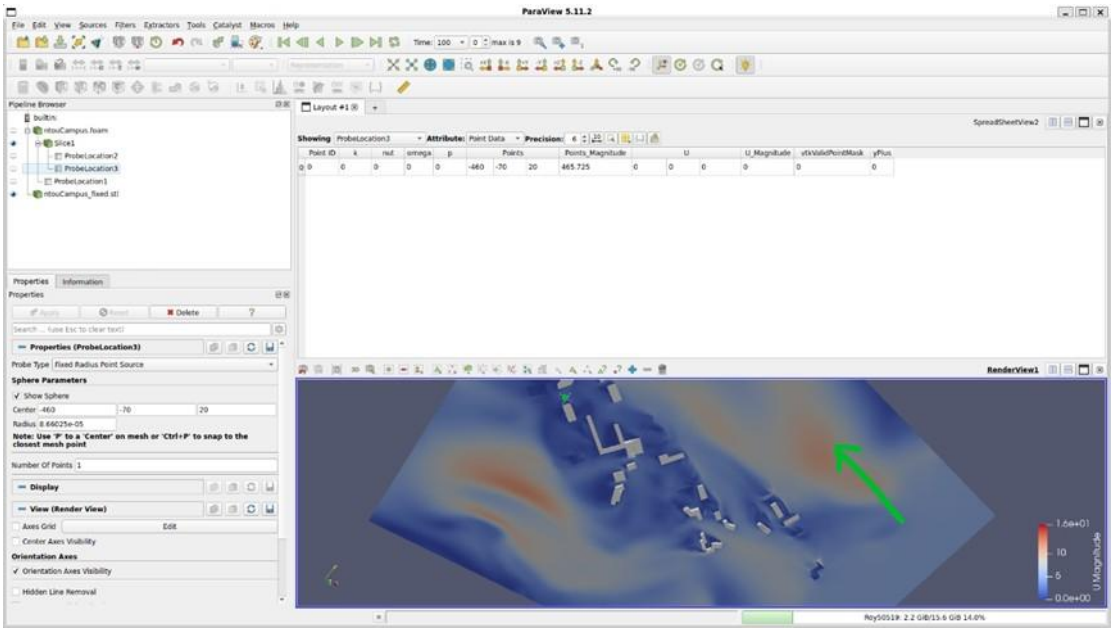
2. 20 公尺高 ：



(圖 43: 延平大樓旁風速圖)

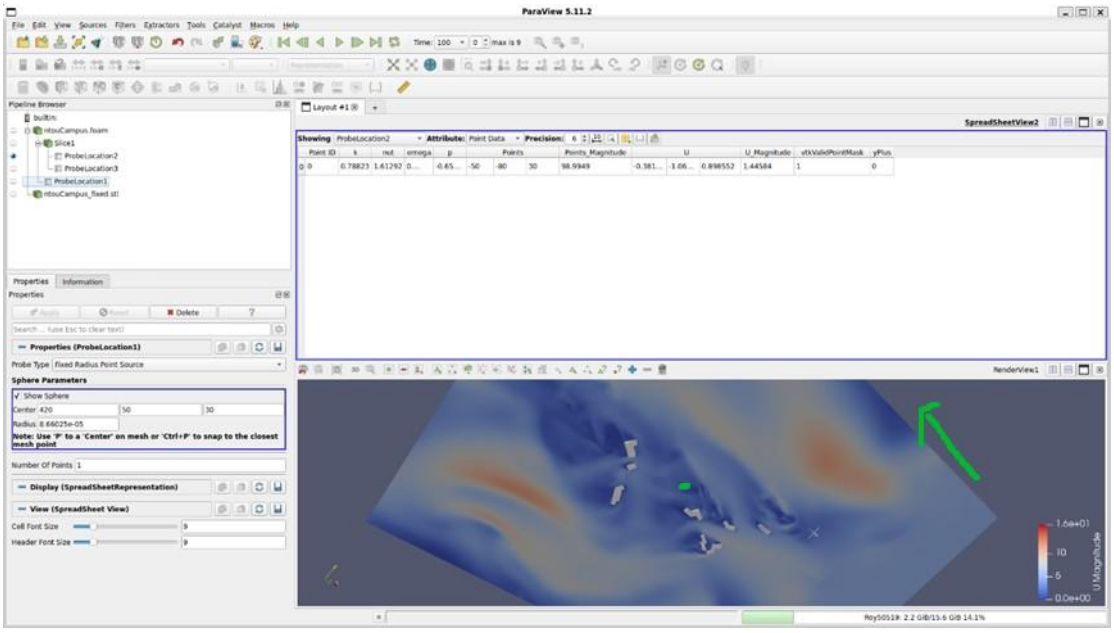


(圖 44:工學院旁風速圖)

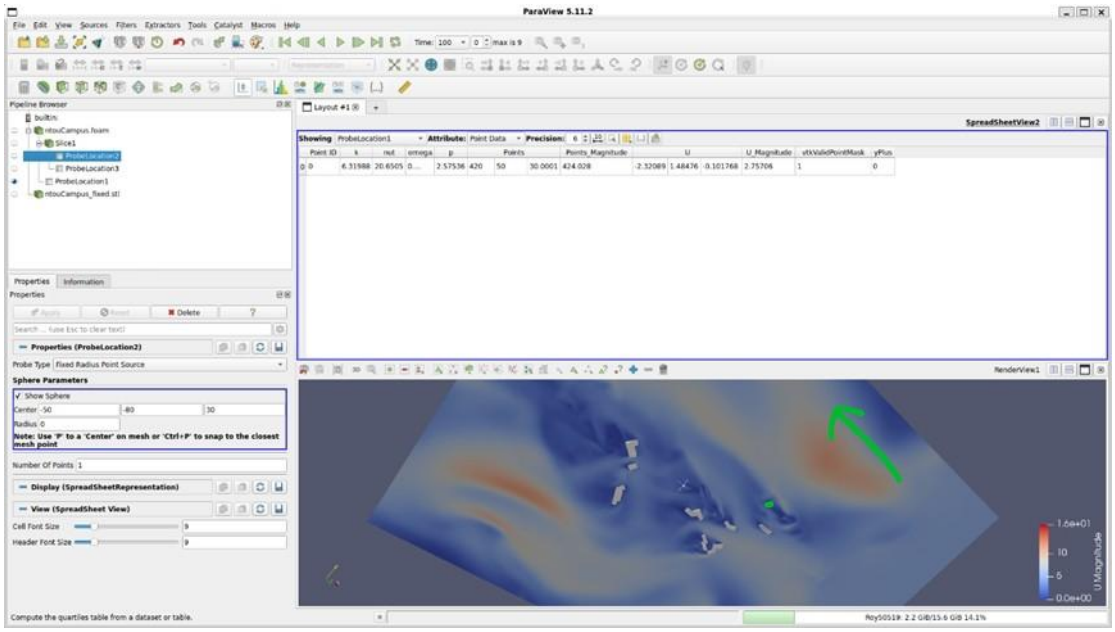


(圖 45:生科院旁風速圖)

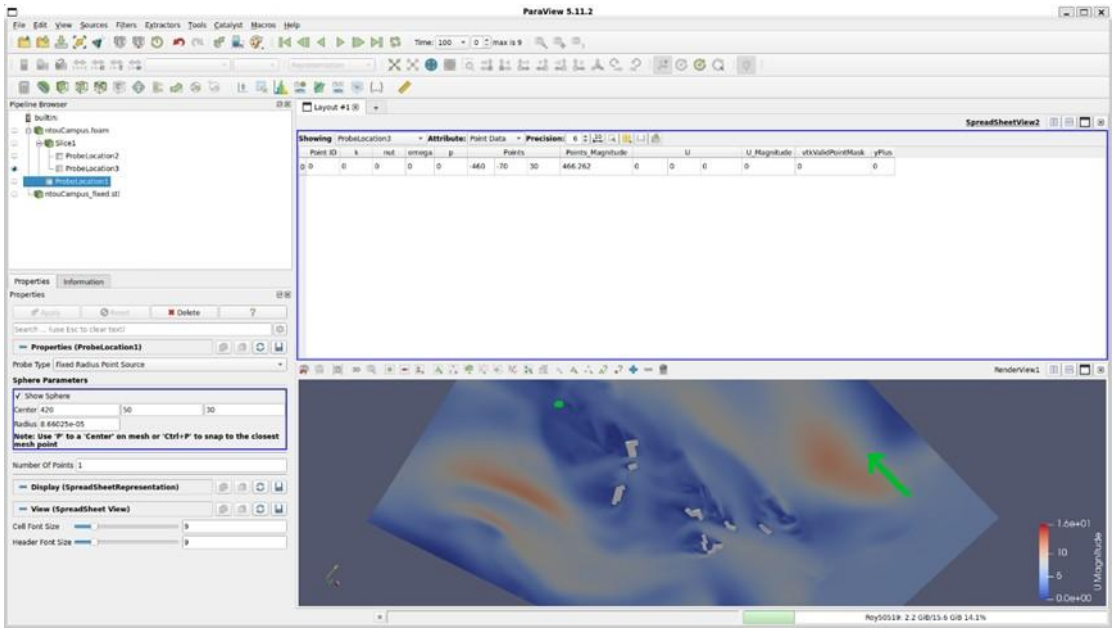
3. 30 公尺高 ：



(圖 46: 延平大樓旁風速圖)

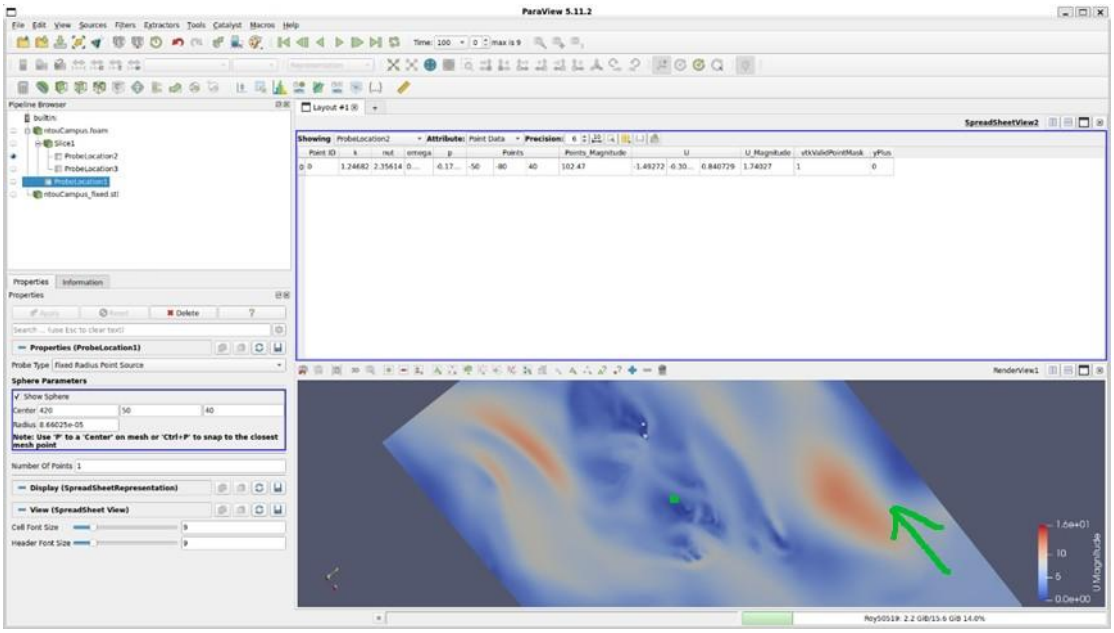


(圖 47:工學院旁風速圖)

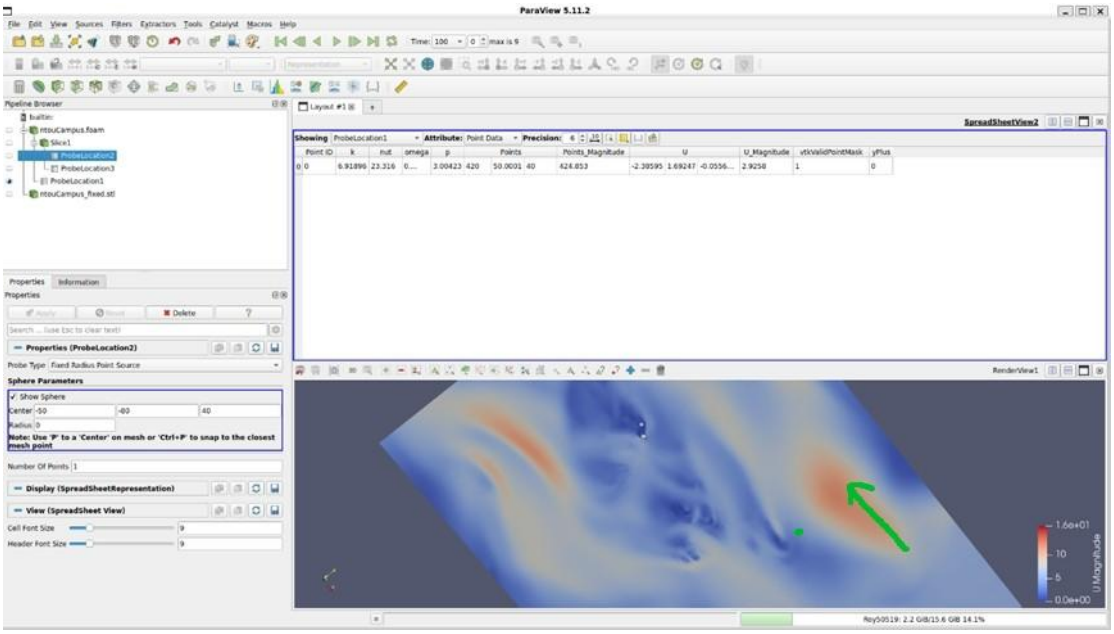


(圖 48:生科院旁風速圖)

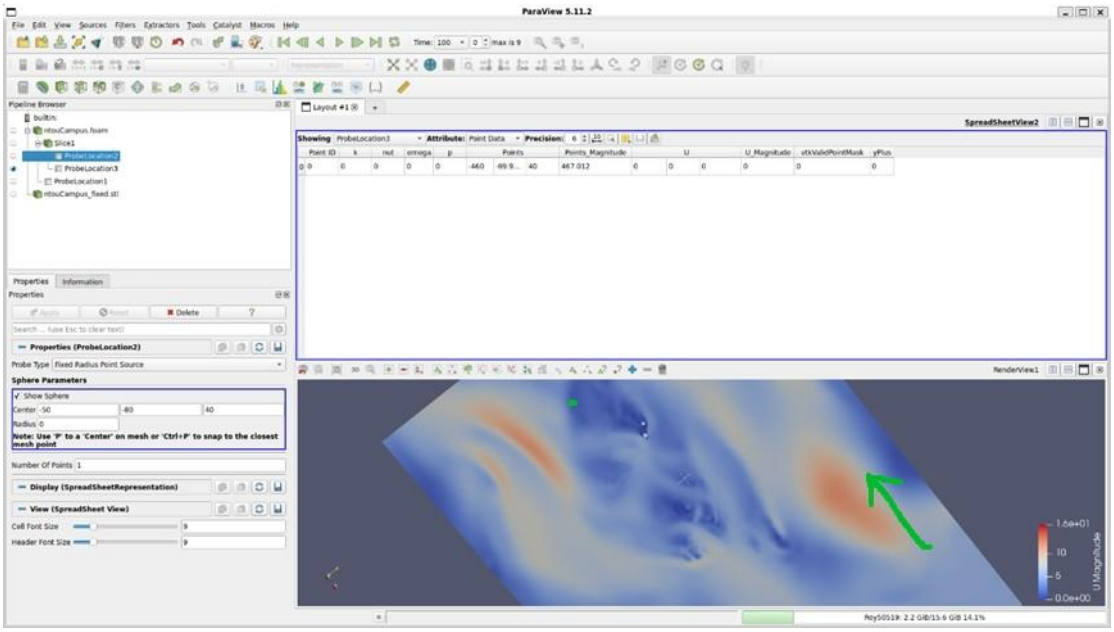
4. 40 公尺高 ：



(圖 49: 延平大樓旁風速圖)



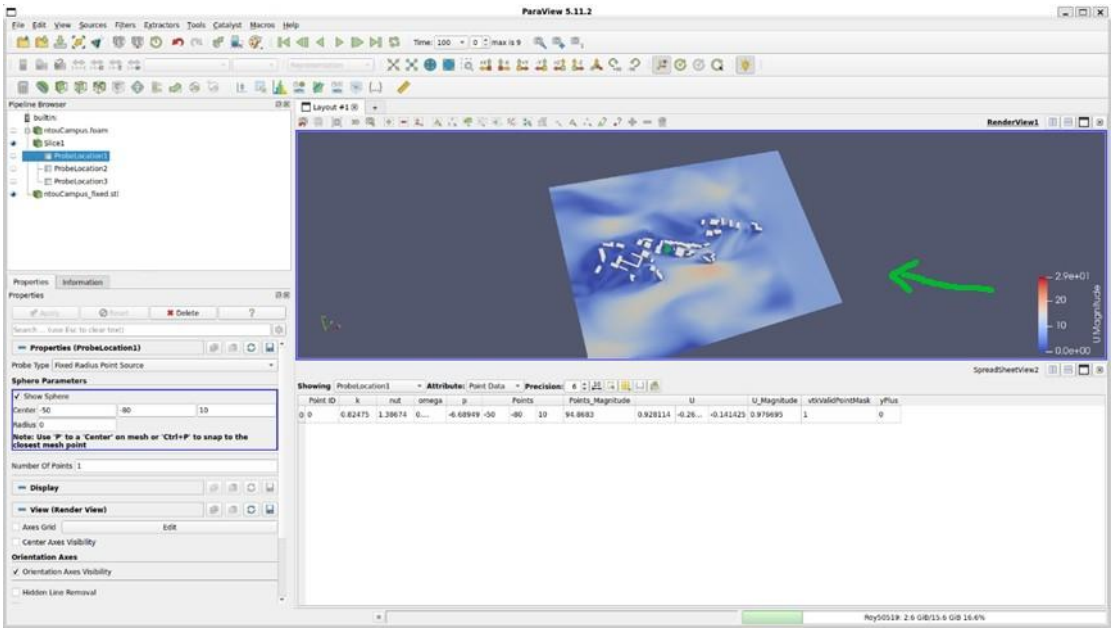
(圖 50:工學院旁風速圖)



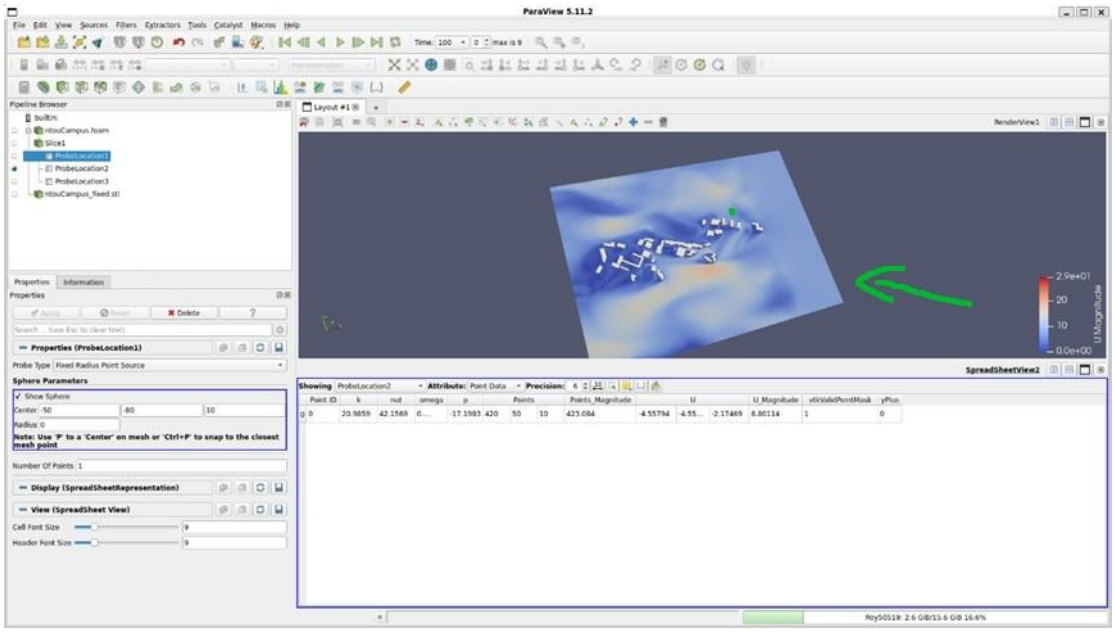
(圖 51:生科院旁風速圖)

(3.)4 級風(8m/s)

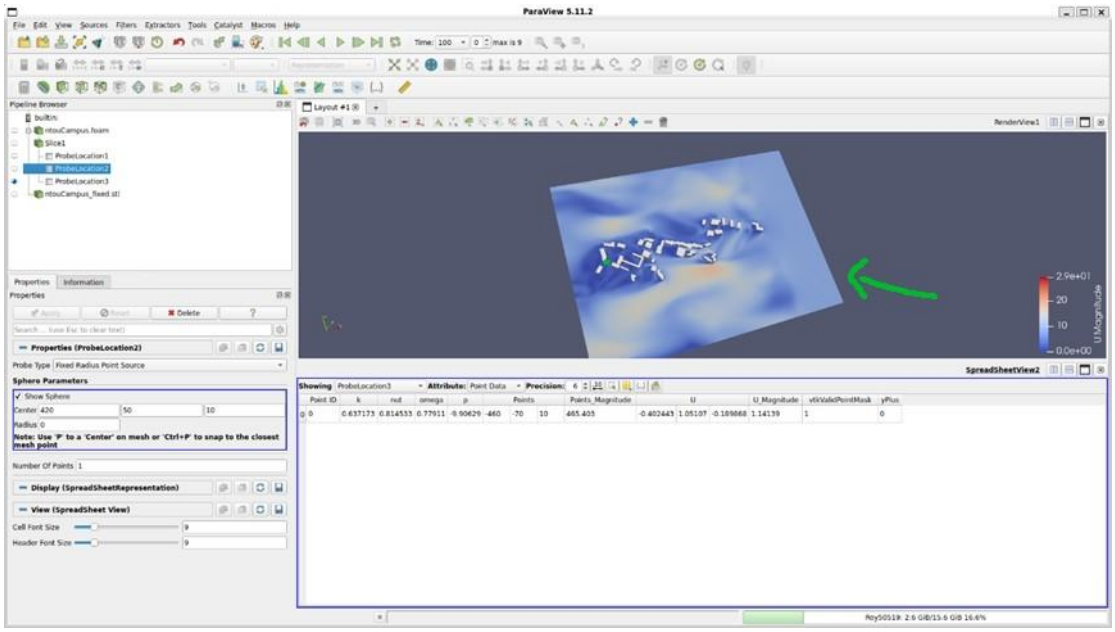
1. 10 公尺高:



(圖 52:延平大樓旁風速圖)

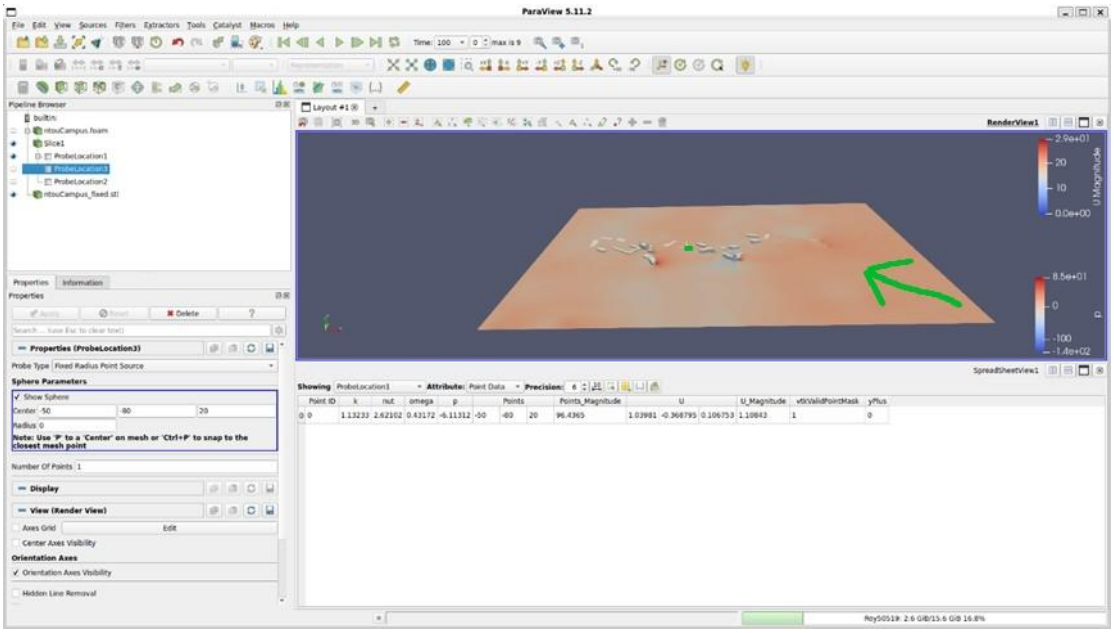


(圖 53:工學院旁風速圖)

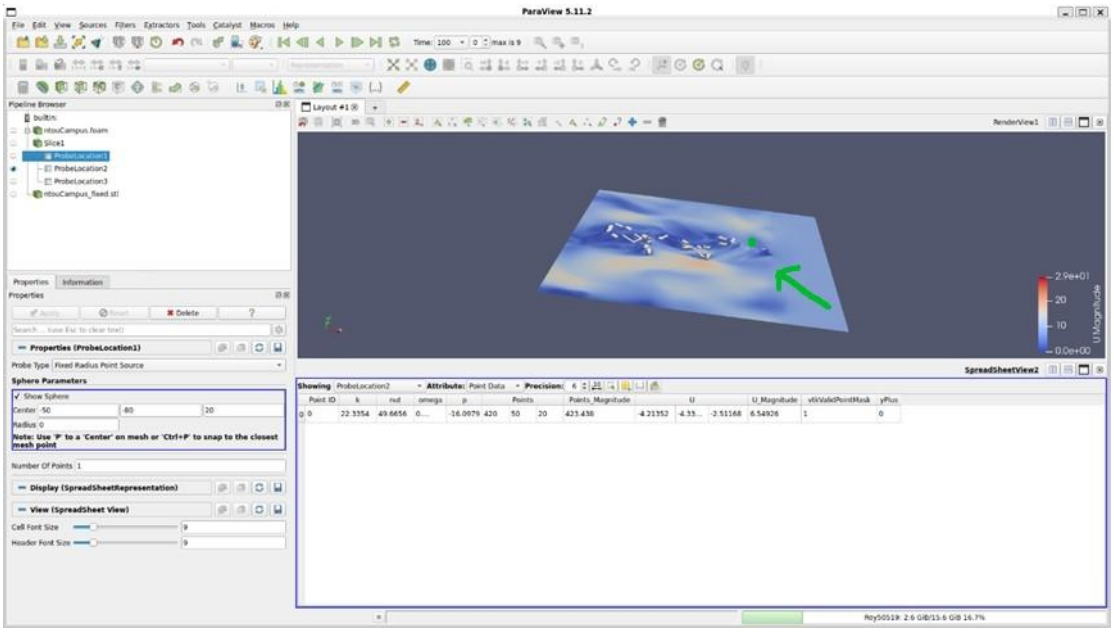


(圖 54:生科院旁風速圖)

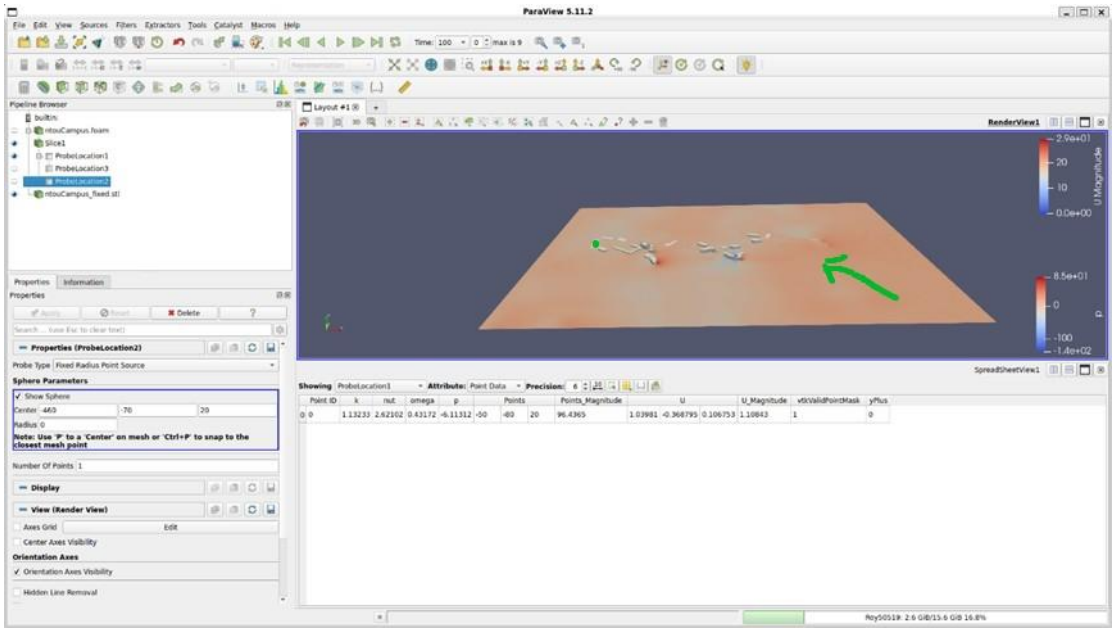
2. 20 公尺高



(圖 55:延平大樓旁風速圖)

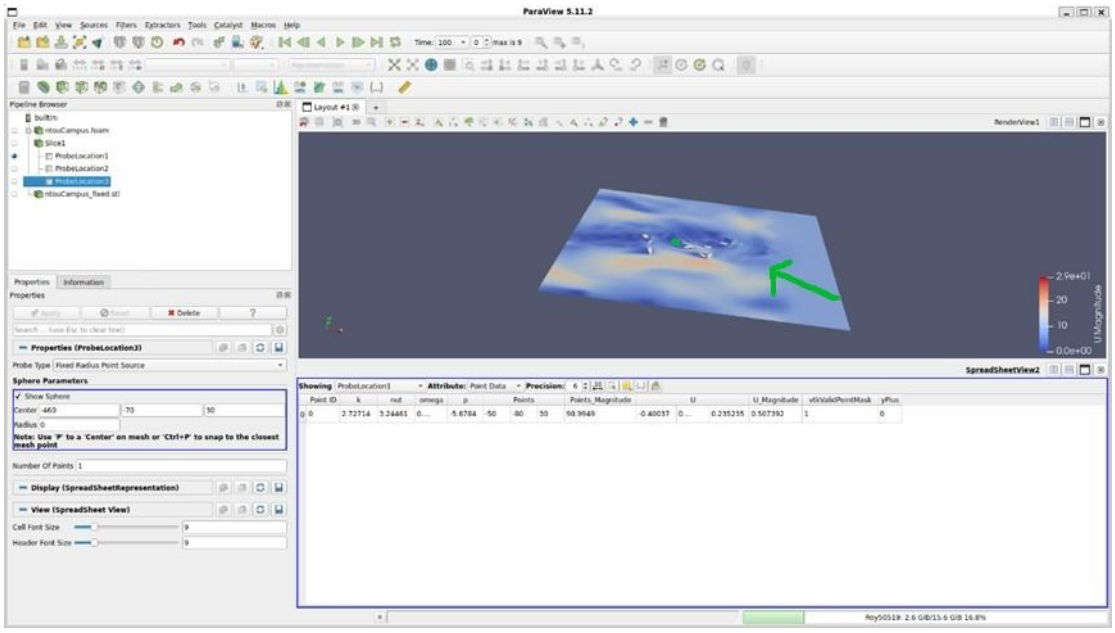


(圖 56:工學院旁風速圖)

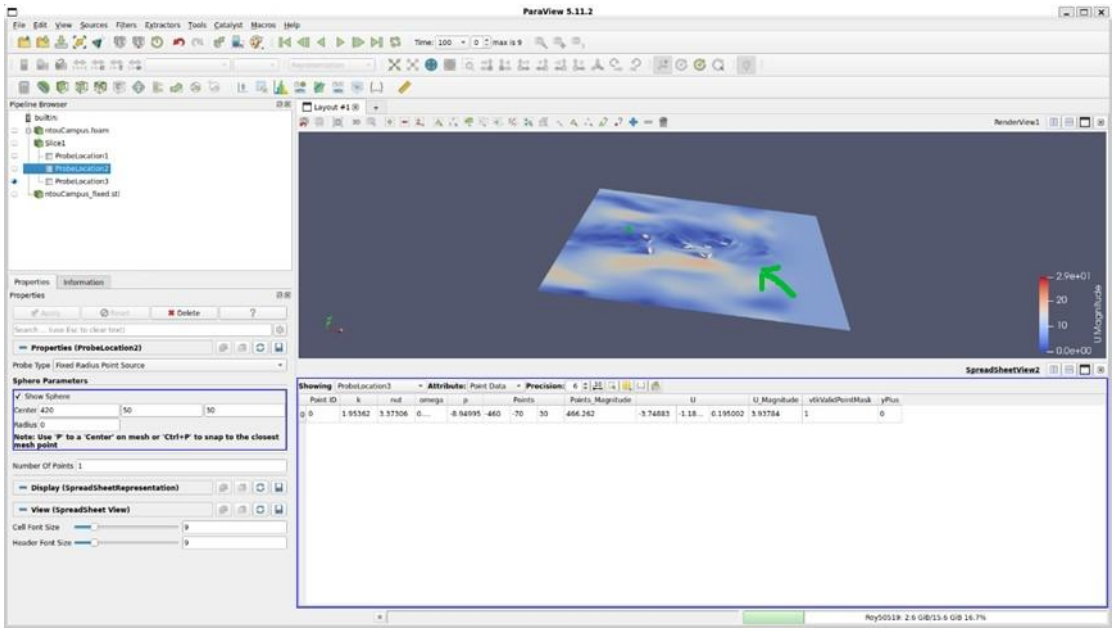


(圖 57:生科院旁風速圖)

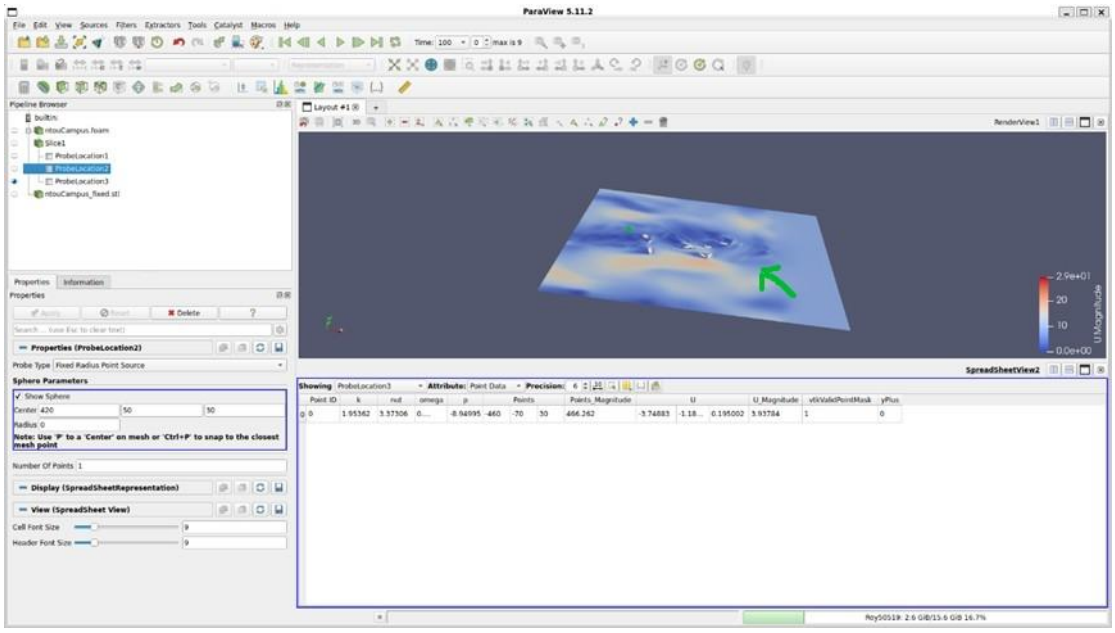
3. 30 公尺高:



(圖 58:延平大樓旁風速圖)

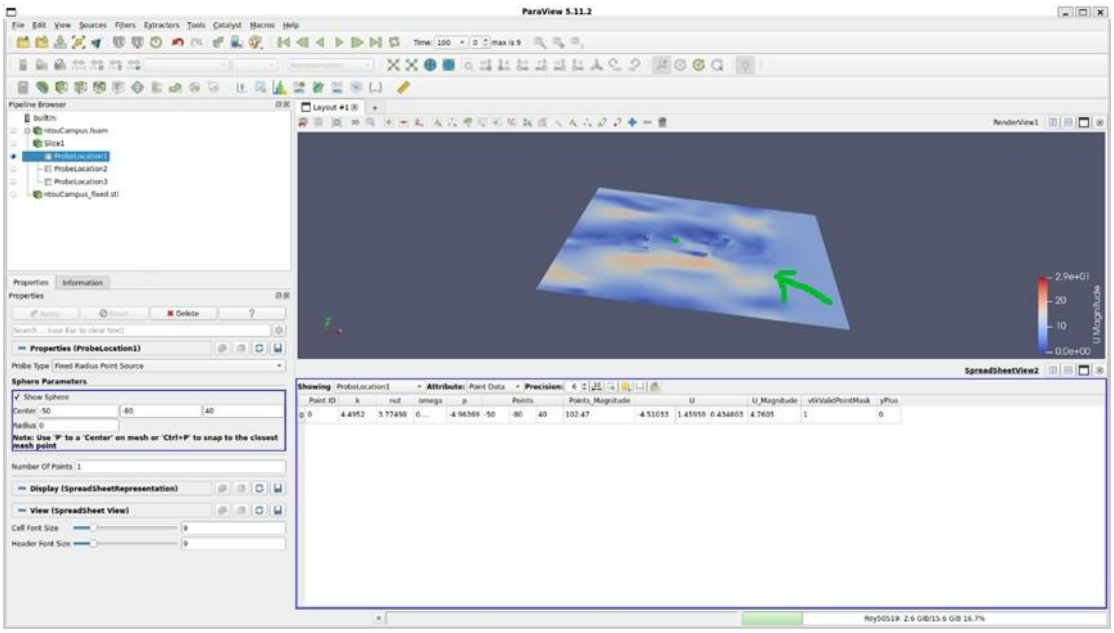


(圖 59:工學院旁風速圖)

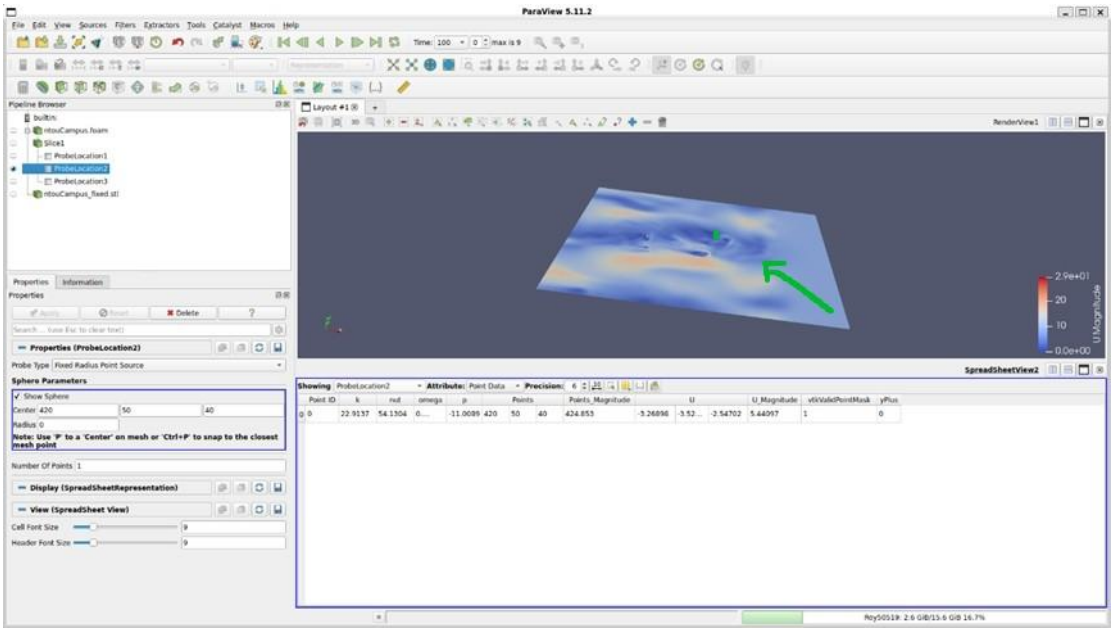


(圖 60:生科院旁風速圖)

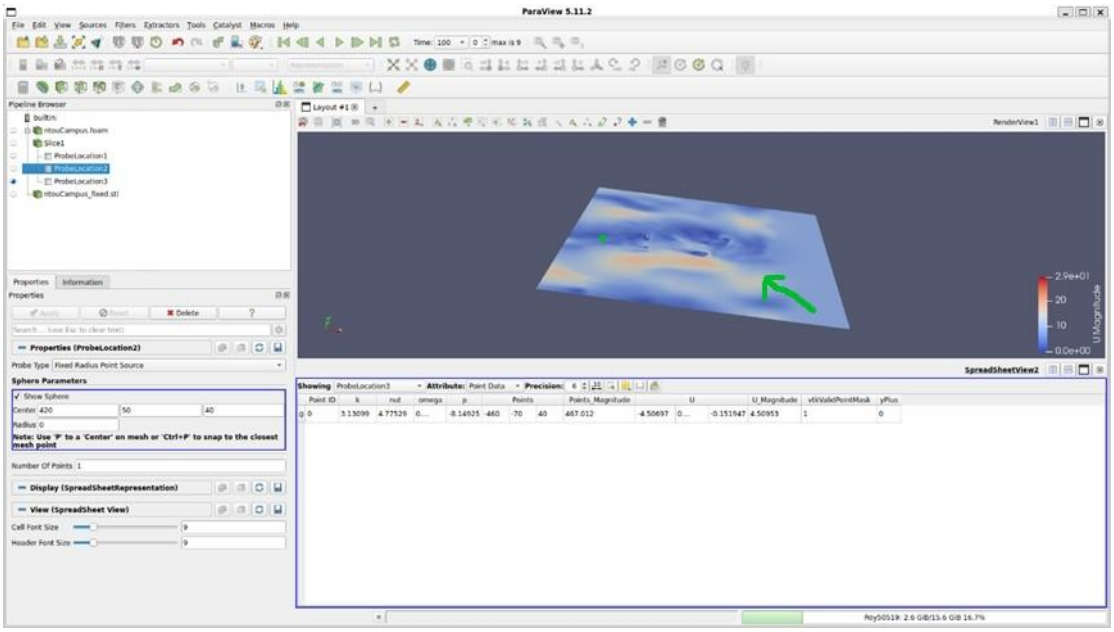
4.40 公尺高:



(圖 61:延平大樓旁風速圖)



(圖 62:工學院旁風速圖)



(圖 63:生科院旁風速圖)

4.2 無人機實測數據整理與處理

本小節旨在展示經過 3.3 節中平均化處理後的實測數據品質，並分析三種工況下的風場特性，以確立數據的統計信賴度。

4.2.1 實測數據品質與可重複性分析

以下是以無人機收集到的最終平均風速與標準差(表 3-表 5):

(1.) 數據彙總:

1. 2 級風(東北風):

風速及最終標準差	延平大樓旁	工學院旁	生科院旁
10 公尺	1.28 m/s、0.265	3.11 m/s、0.270	0.77 m/s、0.215
20 公尺	1.55 m/s、0.485	3.57 m/s、0.471	1.89 m/s、0.221
30 公尺	2.37 m/s、0.326	4.23 m/s、0.358	2.69 m/s、0.217
40 公尺	2.54 m/s、0.257	5.21 m/s、0.488	2.58 m/s、0.173

(表 3：2 級風數據表)

2. 3 級風(北風):

風速及最終標準差	延平大樓旁	工學院旁	生科院旁

10 公尺	2.47 m/s、0.248	4.23 m/s、0.293	1.3 m/s、0.177
20 公尺	2.71 m/s、0.316	5.12 m/s、0.341	4.1 m/s、0.214
30 公尺	4.14 m/s、0.233	6.57 m/s、0.313	4.2 m/s、0.238
40 公尺	4.93 m/s、0.264	5.98 m/s、0.426	4.83 m/s、0.220

(表 4：3 級風數據表)

3. 4 級風(東風):

風速及最終標準差	延平大樓旁	工學院旁	生科院旁
10 公尺	3.22 m/s、0.213	5.05 m/s、0.271	2.1 m/s、0.134
20 公尺	4.31 m/s、0.247	7.64 m/s、0.318	5.23 m/s、0.158
30 公尺	5.92 m/s、0.311	9.03 m/s、0.429	6.34m/s、0.193
40 公尺	7.58 m/s、0.286	7.66 m/s、0.367	7.22 m/s、0.241

(表 5：4 級風數據表)

4.實地實測照:



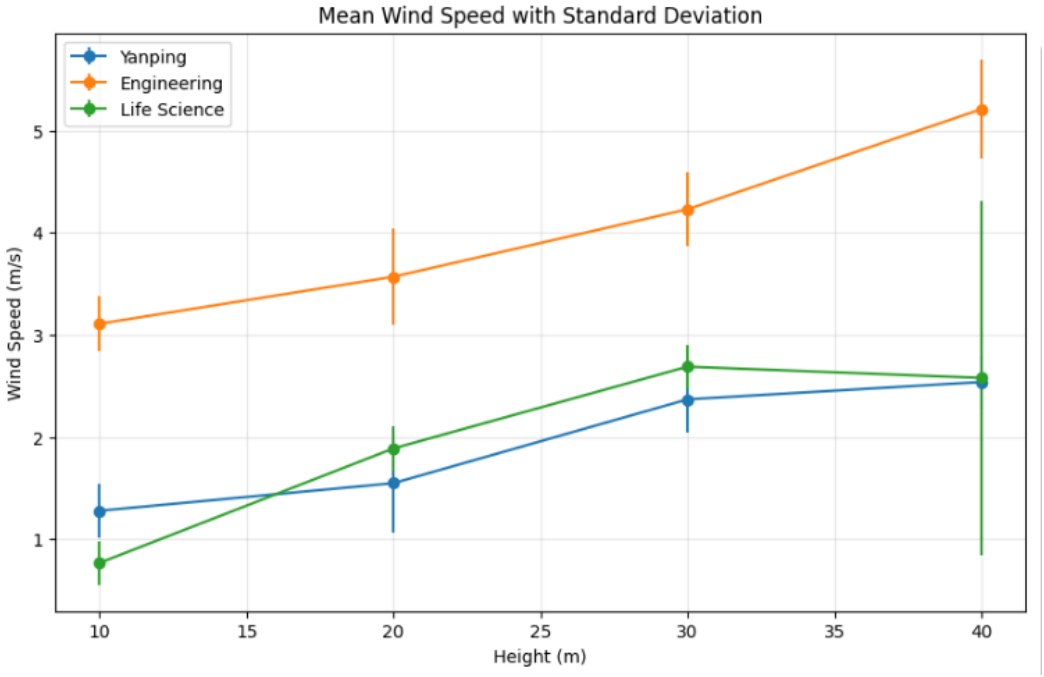
(圖 64:延平大樓旁及工學院實測照片)



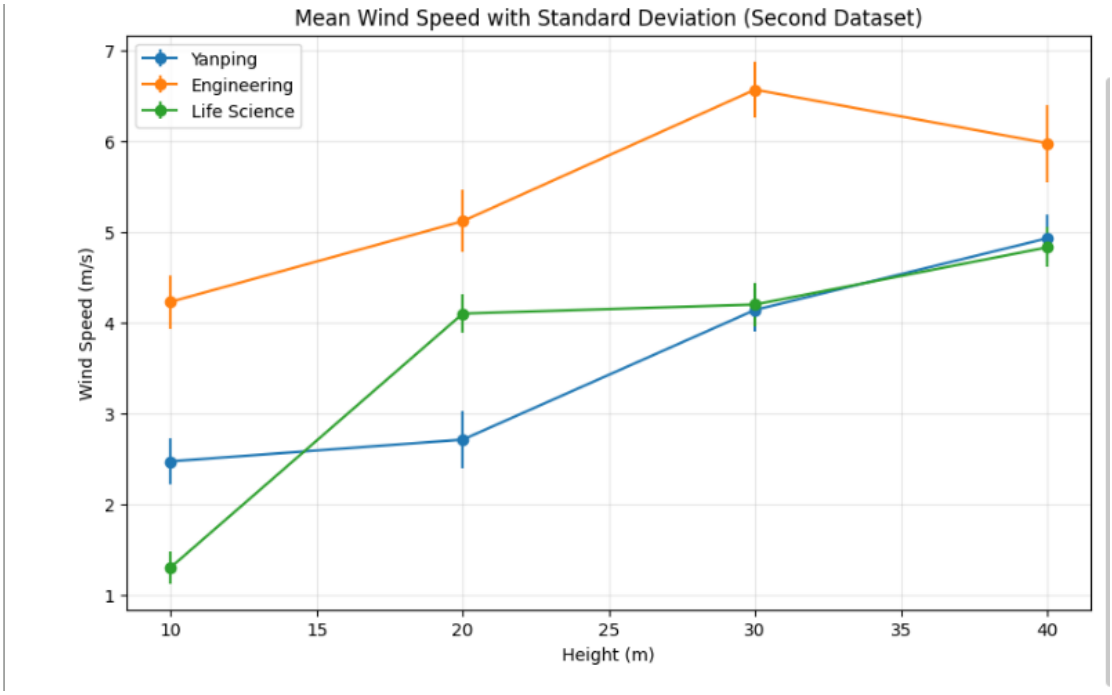
(圖 65:生科院實測照片)

(2.) 實測風場特性分析：

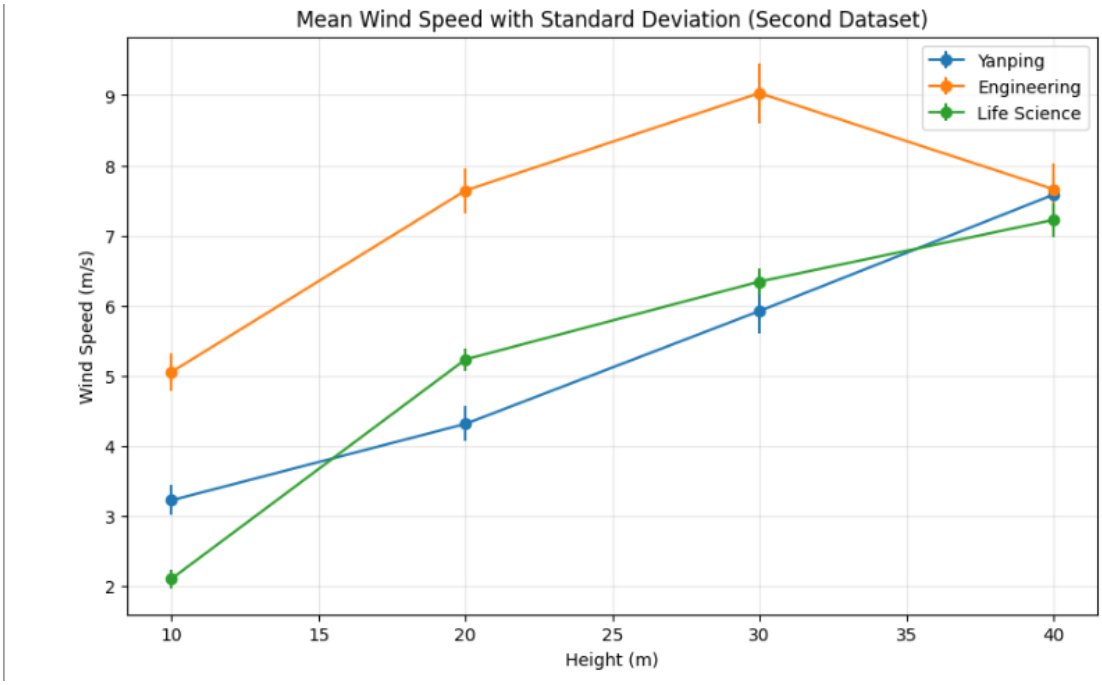
- 數據分析顯示，最終標準差隨著風速工況上升呈現**明顯增長趨勢**。這項趨勢與大氣邊界層中紊流強度隨風速增強而增加的物理特性一致。這證明了無人機感測器系統有效地捕捉了風場的**自然波動 (紊流)**。
- 垂直剖面趨勢：



(圖 66:2 級風速圖與標準差)



(圖 67:3 級風速圖與標準差)

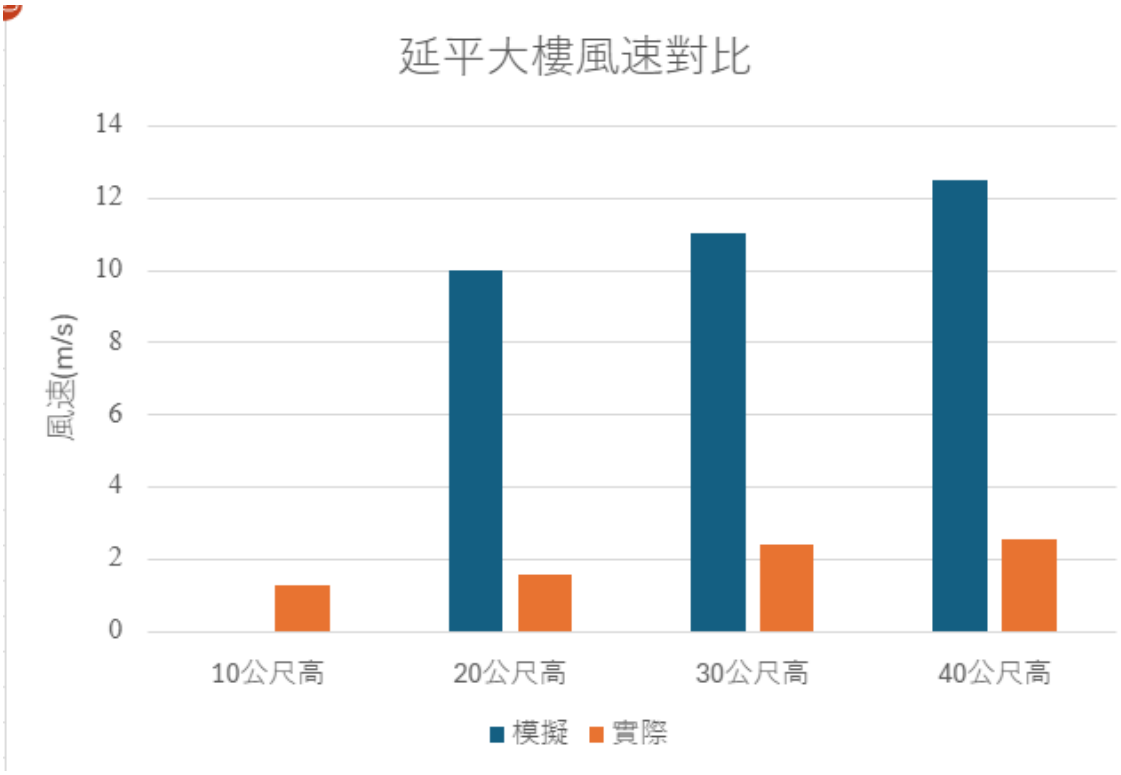


(圖 68:4 級風速圖與標準差)

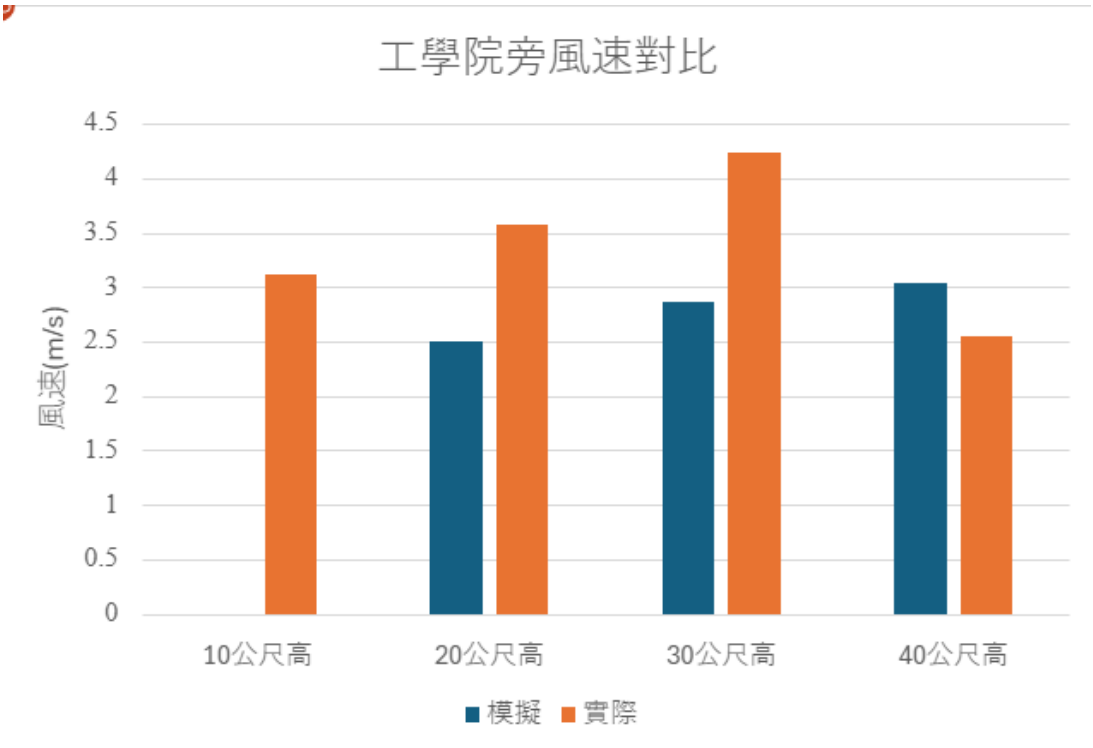
4.3 模擬與實測數據的定量對比

本小節旨在透過定量誤差指標 RMSE 與 R，驗證 OpenFOAM 搭配$k-\omega$ *SST* 模型在 2 級、3 級、 4 級風三種工況下的預測精度。所有實測數據均經過 3.3 節所述的平均化處理。

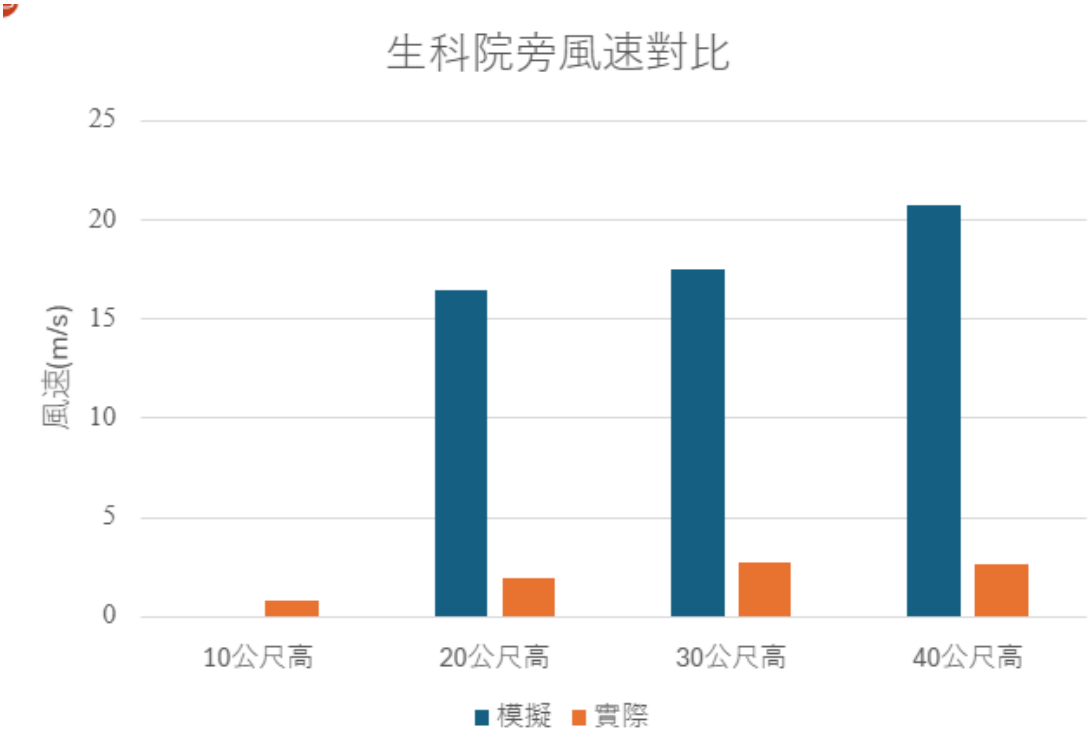
(1.)2 級風:



(圖 69:延平大樓旁風速數據對比)

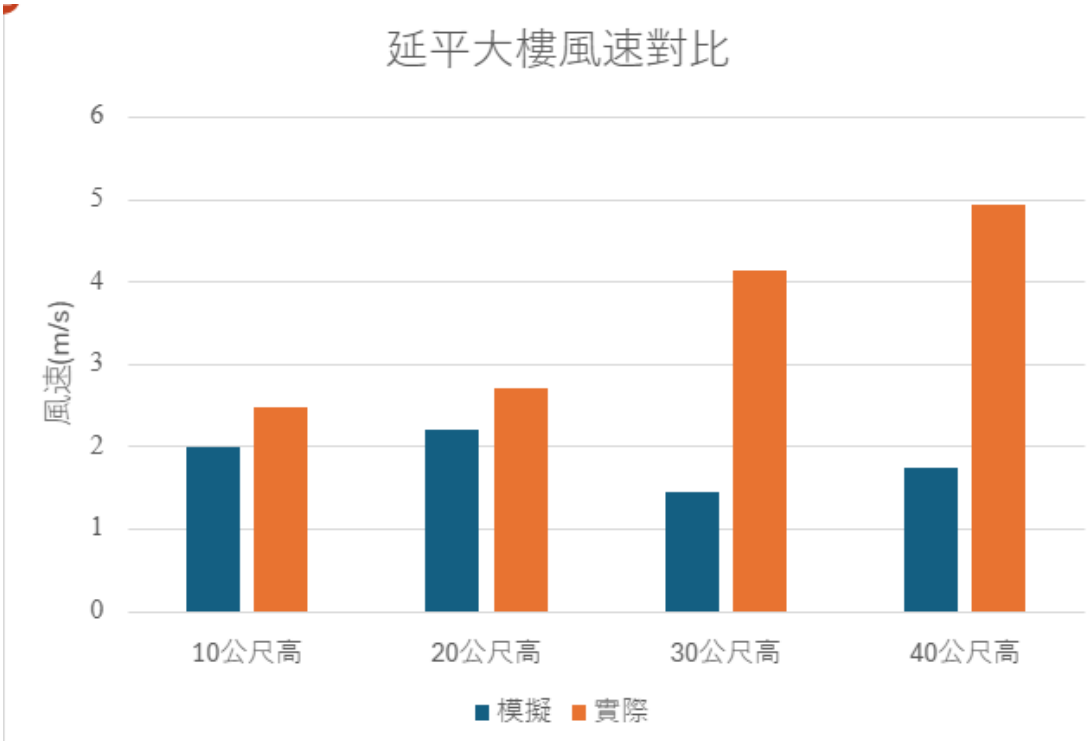


(圖 70:工學院旁風速數據對比)

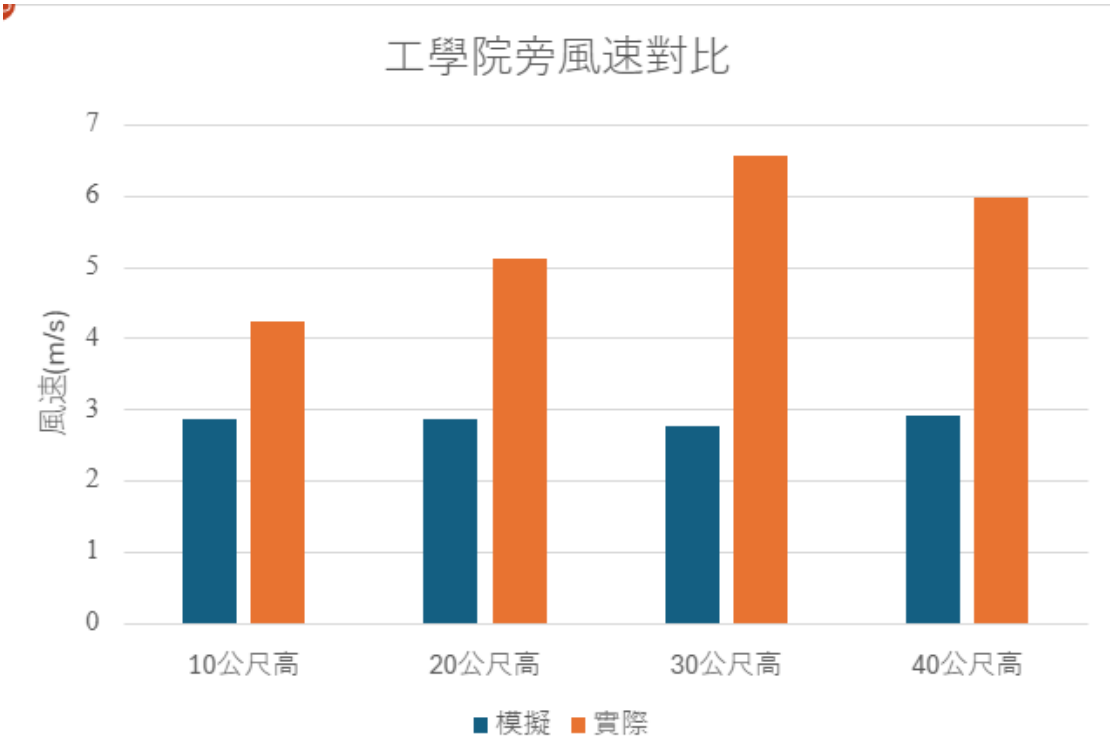


(圖 71:生科院旁風速數據對比)

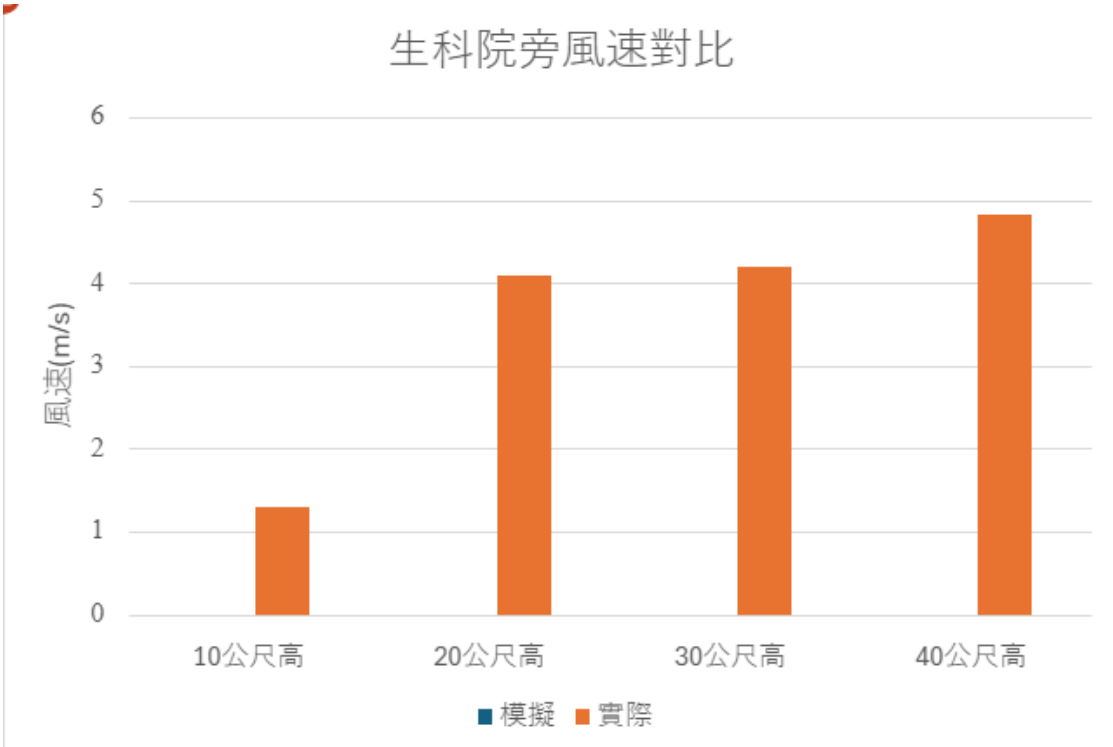
(2.)3 級風:



(圖 72:延平大樓旁風速數據對比)

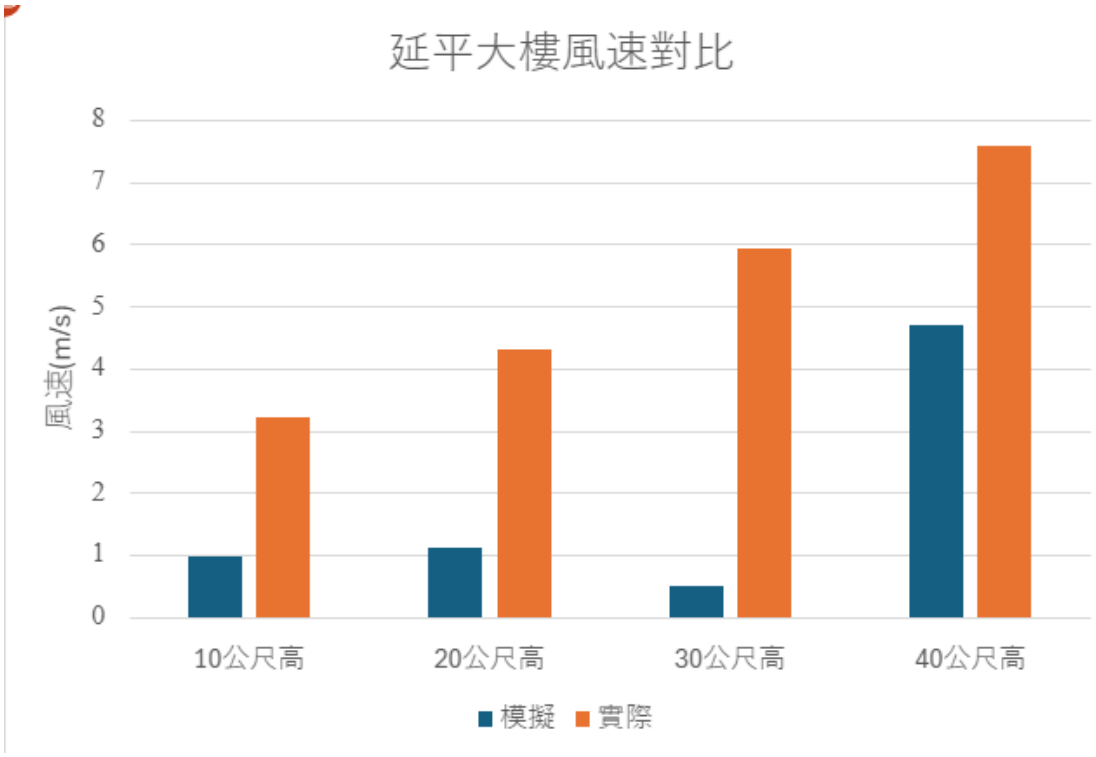


(圖 73:工學院旁風速數據對比)

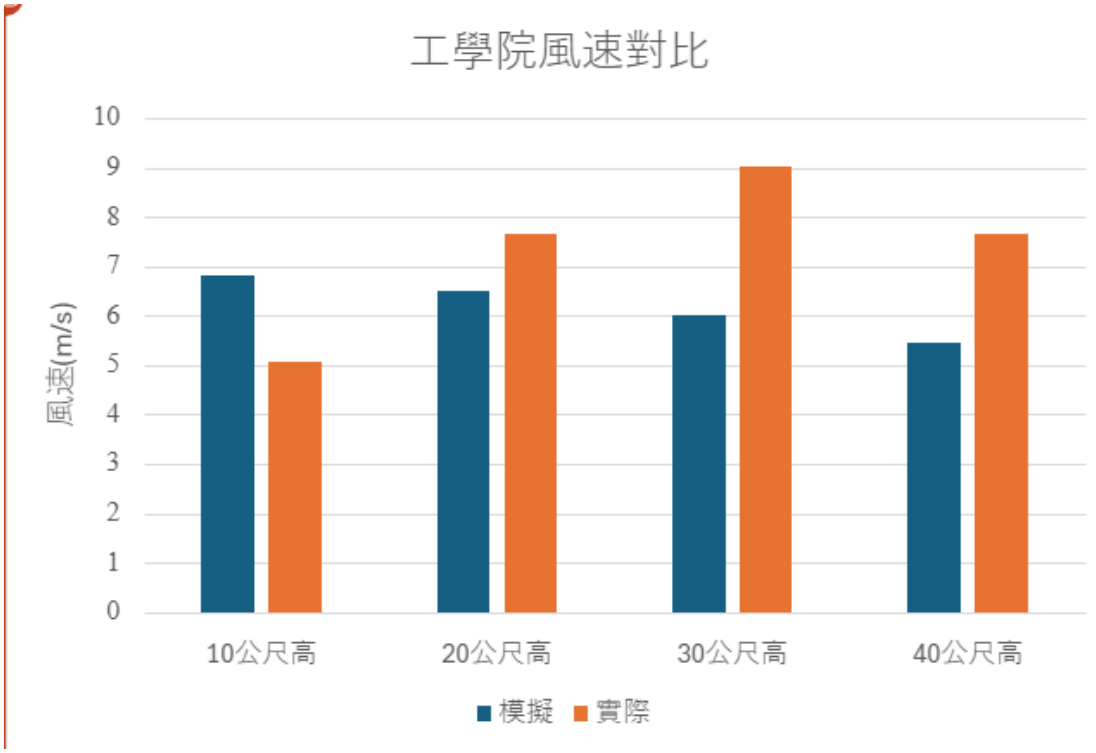


(圖 74:生科院旁風速數據對比)

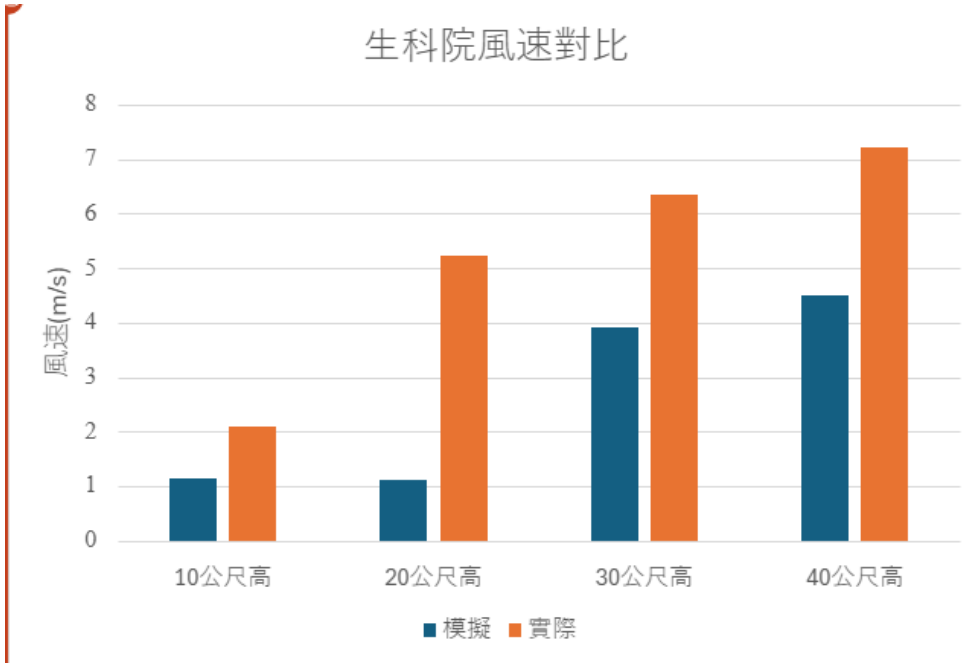
(3.)4 級風:



(圖 75:延平大樓旁風速數據對比)



(圖 76:工學院旁風速數據對比)



(圖 77:生科院旁風速數據對比)

4.4 實驗結果之檢討與建議

實驗結果發現，雖然 CFD 模擬與實測數據整體趨勢一致，但在建築群密集區域及低風速條件下出現明顯偏差，因此有必要檢討模擬模型、測量方法及相關假設，並提出後續改進方向。

(1.) 模擬方面:

因為硬體設備限制，紊流模型的 yPlus 數值相當高，導致在建築群周圍模擬出的風速跟實際的差距不小，並且簡化周圍環境也容易導致局部風速偏差擴大，未來在模擬時可以選購更好的設備進行模擬計算以解決這個問題。

(2.) 實測方面:

由於時間關係，本實驗無法獲得更多的實測樣本，無論是地點抑或是次數都偏少，樣本過少可能導致無法很好的反應真實結果，並且 Pixhawk 4 GPS 模組本身只帶有一個氣壓計，導致內建的氣壓計毀壞之後無人機會不受控制，甚至墜毀，未來再做實測時建議能用新型的 Pixhawk 6 GPS 模組，其本身內建兩個氣壓計，可以更好的避免無人毀壞問題。

第五章 結論

本實驗結合 Blender 幾何建模、OpenFOAM 數值模擬與無人機多工況實測，旨在驗證 CFD 模擬模型與 Windy 軟體的風速預測精確度。結果顯示，CFD 模擬在初始風速過小時會有極大的偏差，不過整體趨勢與實測風速高度分布相符，但整體數值會小於實際結果，並且在建築物群之中時，數值會下降的更多，整體平均偏差落在 30%-40%，我們猜測是由於硬體限制導致模擬無法在建築周圍用紊流模型很好的模擬整體風速、風向狀況，數據表明 CFD 模擬模型對建築物周邊風速的預測較為困難，但可以得知周圍風向流動，而高度越高時實際數值會越來越接近 Windy 軟體所預測的風速，整體而言，CFD 模型可以提供預測跟模擬，不過要注意風速數值的偏差，而 Windy 軟體則適合高空(40 公尺以上)或空地的風速估算，兩者可配合使用，以提升精確度。

第六章 參考資料

[1] 超音波風速風向儀用戶使用手冊(紙本)

[2] TAIWANIOT 網站，ESP-32S 規格資料：
https://www.taiwaniot.com.tw/product/nodemcu-esp-32s/?srsltid=AfmBOoo_hMrojUPosTF0vkJ2QTVEYjIizKfUIb4y2Y8FMPnDgSCwE-ck

[3]TAIWANIOT 網站，RS-485 模組規格資料：

https://www.taiwaniot.com.tw/product/%E5%96%AE%E7%89%87%E6%A9%9F-ttl%E8%BD%89rs485%E6%A8%A1%E7%B5%84-485%E8%BD%89%E4%B8%B2%E5%8F%A3uart%E9%9B%BB%E5%B9%B3%E4%BA%92%E8%BD%89/?srsltid=AfmBOorHN2OmkhHFwERAewhXJ8movpUTr7Pff5R-LPJm9NSfo53xiOTQ

[4]TAIWANIOT 網站，Raspberry Pi 5 規格資料：

https://www.taiwaniot.com.tw/product/raspberry-pi-5-model-b-4g/

[5]ArduPilot 官方網站，各種校正條參說明：

https://ardupilot.org/copter/index.html

[6]ESP-32S 使用教學：

https://blog.jmaker.com.tw/arduino-ide-esp32/

[7] Segersson D. A tutorial to urban wind flow using OpenFOAM. In: Proceedings of CFD with OpenSource Software. 2017. p. 1-28. (佐證 Blender 軟體優勢與 OpenFOAM 城市流動應用)

[8] vvoovv. blosm - Blender OpenStreetMap [computer software]. Available from: https://github.com/vvoovv/blosm. Accessed 2024 Nov 17. (佐證衛星數據獲取插件 blosm 的使用)

[9] Blocken B, Gualtieri C. A review of pollutant dispersion around isolated buildings in the urban environment: CFD modelling and wind tunnel experiments. Environ Pollut. 2009;157(10):2965-2977. (佐證模型簡化原則)

[10] Simularge Blog. Preparing CAD Geometry for CFD Simulations: Essential Steps and Best Practices. Simularge; 2025. Available from: https://www.simularge.com/blog/preparing-cad-geometry-for-cfd-simulations-essential-steps-and-best-practices. Accessed 2024 Nov 17. (佐證幾何修復消除非流形邊緣，解決網格爆點問題)

[11] Tominaga Y, Mochida A. CFD prediction of pedestrian wind environment around buildings: a review and practical recommendations. J Wind Eng Ind Aerodyn. 2011;99(10):349-361. (佐證網格生成的重要性，以及 CFD 最佳實踐規範)

[12] Richards DJ, Stansby PK. Computational fluid dynamics prediction of pedestrian-level winds: a review of best practice and validation methods. J Wind Eng Ind Aerodyn. 2018;176:216-228. (佐證網格品質指標，如最大非正交性的接受標準)

[13] Menter FR. Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications. AIAA Journal. 1994;32(8):1598-1605. ($k-\omega\,SST$ 模型原始論文，佐證其優勢)

[14] Blocken B. Computational Fluid Dynamics (CFD) for urban physics: A review. J Wind Eng Ind Aerodyn. 2015;147:1-35. (佐證 $k-\varepsilon\,SST$ 等其他紊流模型在城市 CFD 應用中的優勢與限制，以支持 $k-\omega\,SST$ 的選擇)

[15] Pope SB. Turbulent Flows. 1st ed. Cambridge University Press; 2000. (紊流理論經典專著，佐證紊流動能 k 和耗散率 ω 的初始值公式計算)

[16] Taylor JR. An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements. 2nd ed. University Science Books; 1997. (佐證最終標準差作為實驗可重複性的統計指標)

附錄

1. ESP-32S 開發板內的程式碼(使用 Arduino IDE 進行程式撰寫及燒錄)

完整程式碼：

```
#include <ModbusMaster.h>

// ====== 物件與腳位設定 ======
ModbusMaster node;

#define RE_DE 4      // RS485 模組 DE/RE 腳位
#define TXD2 17      // ESP32 UART2 TX
#define RXD2 16      // ESP32 UART2 RX

// RS485 傳輸前/後的方向控制
void preTransmission()  { digitalWrite(RE_DE, HIGH); }
void postTransmission() { digitalWrite(RE_DE, LOW); }

void setup() {
  Serial.begin(115200);  // 與樹莓派通訊
  Serial2.begin(9600, SERIAL_8E1, RXD2, TXD2);  // 與風速計通訊：9600, 8E1
  node.begin(1, Serial2); // Modbus ID = 1

  pinMode(RE_DE, OUTPUT);
  digitalWrite(RE_DE, LOW); // 預設為接收狀態

  node.preTransmission(preTransmission);
  node.postTransmission(postTransmission);
```

```
  delay(1000);
  Serial.println("☑ ESP32 風速計啟動完成，開始傳送資料...");
}

void loop() {
  float spd = NAN;
  float dir = NAN;

  // ====== 讀取風向（地址 0x0001, 寄存器數量 1）======
  uint8_t result_dir = node.readHoldingRegisters(0x0001, 1);
  if (result_dir == node.ku8MBSuccess) {
    dir = node.getResponseBuffer(0);
  }

  // ====== 讀取風速（地址 0x0002, 寄存器數量 2）======
  uint8_t result_spd = node.readHoldingRegisters(0x0002, 2);
  if (result_spd == node.ku8MBSuccess) {
    uint16_t high_word = node.getResponseBuffer(0);
    uint16_t low_word  = node.getResponseBuffer(1);

    // 正確順序：低位在前
    union {
      uint32_t i;
      float f;
    } u;
    u.i = ((uint32_t)low_word << 16) | high_word;
```

```
    spd = u.f;
  }

  // ====== 防呆與輸出 JSON ======
  if (!isnan(spd) && !isnan(dir)) {
    Serial.printf("{\"spd\":%.2f,\"dir\":%.1f}\n", spd, dir);  // 標準
JSON
  } else {
    Serial.println("{\"spd\":0,\"dir\":0}");  // 若讀取失敗則回傳 0
  }

  delay(1000);  // 每秒輸出一次
}
```

2. Raspberry Pi 風速風向資料處理與 Flask 伺服器程式碼

程式碼：

```
# ================================
# 風速風向 Flask Server（完整穩定版）
# ================================
from flask import Flask, render_template, jsonify
import serial, json, time, csv, os, threading, socket
from datetime import datetime

app = Flask(__name__)

# ---------- 串口設定 ----------
# 依你目前環境：ESP32 用 /dev/ttyUSB0，115200
# 若不同可改成 /dev/ttyACM0 或 Windows 的 COMx
SERIAL_PORT = os.environ.get("WIND_SERIAL", "/dev/ttyUSB0")
```

```
BAUD = 115200
ser = None

def _open_serial():
    global ser
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD, timeout=1)
        print(f"☑ Serial opened on {SERIAL_PORT} @ {BAUD}")
    except Exception as e:
        ser = None
        print(f"❌ Serial open failed: {e}")

_open_serial()

# ---------- 全域資料 ----------
last_data = {"spd": 0.0, "dir": 0.0}
samples = []
data_lock = threading.Lock()

# ---------- 工具：取得本機 IP ----------
def get_ip():
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except:
        return "127.0.0.1"

# ---------- 感測資料讀取（單筆） ----------
def read_one():
    global last_data, ser
```

```
    if ser is None or not ser.is_open:
        _open_serial()
        return

    try:
        line = ser.readline().decode("utf-8", errors="ignore").strip()
        if not line:
            return

        # 僅接受完整 JSON
        if not (line.startswith("{") and line.endswith("}")):
            # print("⚠ 忽略不完整資料:", line)
            return

        d = json.loads(line)
        if "spd" in d and "dir" in d:
            with data_lock:
                last_data = {"spd": float(d["spd"]), "dir": float(d["dir"])}
                samples.append(last_data)
        # else:
        #       print("⚠ JSON 缺少欄位:", d)

    except json.JSONDecodeError:
        # print("❌ JSON decode failed:", line)
        pass
    except Exception as e:
        print("❌ read_one error:", e)

# ---------- 每分鐘平均存檔 ----------
def save_average():
    global samples
    with data_lock:
        if not samples:
```

```
            return
        avg_spd = sum(d["spd"] for d in samples) / len(samples)
        avg_dir = sum(d["dir"] for d in samples) / len(samples)
        samples = []

    if not os.path.exists("data"):
        os.makedirs("data")
    filename = f"data/wind_{datetime.now().strftime('%Y-%m-%d')}.csv"
    write_header = not os.path.exists(filename)

    with open(filename, "a", newline="") as f:
        w = csv.writer(f)
        if write_header:
            w.writerow(["timestamp", "avg_speed(m/s)",
"avg_direction(deg)"])
        w.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                    round(avg_spd, 2), round(avg_dir, 1)])
    print(f"📦 Saved avg: {avg_spd:.2f} m/s, {avg_dir:.1f}°")

# ---------- 感測讀取背景執行緒 ----------
def sensor_loop():
    last_save = time.time()
    while True:
        read_one()
        if time.time() - last_save > 60:
            save_average()
            last_save = time.time()
        time.sleep(0.02)  # 50Hz 輕量輪詢

# ---------- Flask 路由 ----------
@app.route("/")
def index():
    return render_template("index.html")  # templates/index.html
```

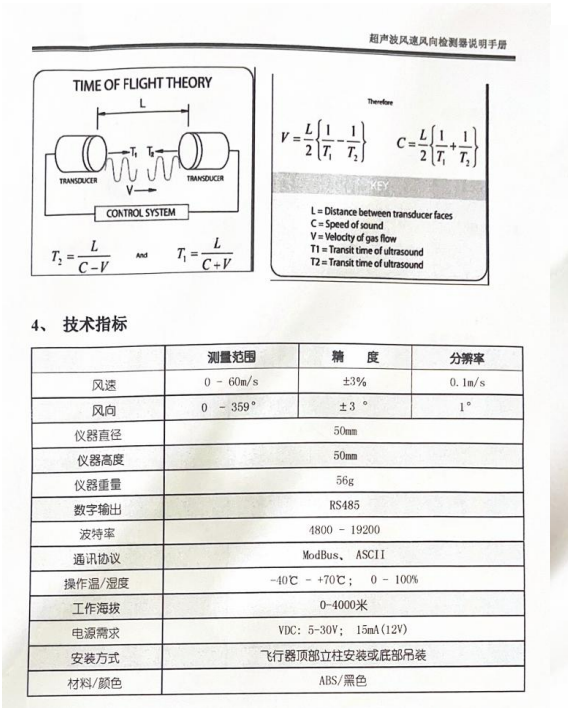
```
@app.route("/data")
def data():
    with data_lock:
        d = dict(last_data)
    return jsonify(d)


# ---------- 入口 ----------
if __name__ == "__main__":
    ip = get_ip()
    print(f"🌐 Open http://{ip}:5000 （你的 IP 是 {ip}，你提供的是 172.20.10.6 也可以）")

    # 啟動感測背景執行緒
    threading.Thread(target=sensor_loop, daemon=True).start()

    # 啟動 Flask（對外可訪問）
    app.run(host="0.0.0.0", port=5000, debug=False)
```

3. 超聲波風速風向儀風戶使用手冊重點截圖：



超声波风速风向检测器说明手册

TIME OF FLIGHT THEORY

$T_2 = \frac{L}{C-V}$ And $T_1 = \frac{L}{C+V}$

Therefore

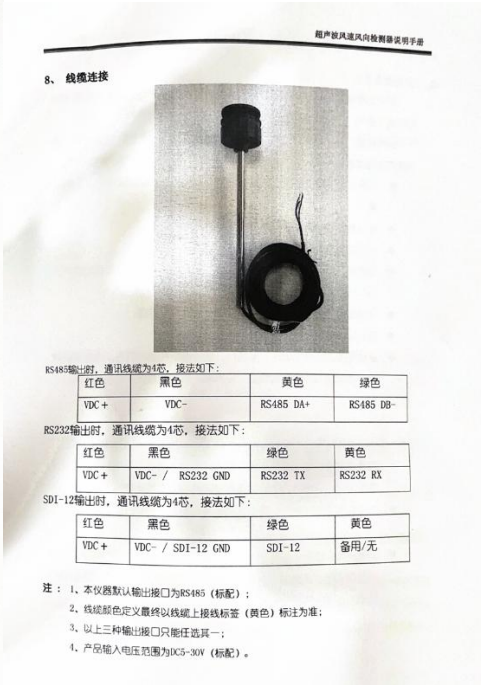
$V = \frac{L}{2}\left\{\frac{1}{T_1} - \frac{1}{T_2}\right\}$ $C = \frac{L}{2}\left\{\frac{1}{T_1} + \frac{1}{T_2}\right\}$

KEY

L = Distance between transducer faces
C = Speed of sound
V = Velocity of gas flow
T1 = Transit time of ultrasound
T2 = Transit time of ultrasound

4、 技术指标

	测量范围	精 度	分辨率
风速	0 - 60m/s	±3%	0.1m/s
风向	0 - 359°	±3 °	1°
仪器直径	50mm		
仪器高度	50mm		
仪器重量	56g		
数字输出	RS485		
波特率	4800 - 19200		
通讯协议	ModBus、 ASCII		
操作温/湿度	-40℃ - +70℃； 0 - 100%		
工作海拔	0-4000米		
电源需求	VDC：5-30V； 15mA(12V)		
安装方式	飞行器顶部立柱安装或底部吊装		
材料/颜色	ABS/黑色		



超声波风速风向检测器说明手册

8、 线缆连接

RS485输出时，通讯线缆为4芯，接法如下：

红色	黑色	黄色	绿色
VDC+	VDC-	RS485 DA+	RS485 DB-

RS232输出时，通讯线缆为4芯，接法如下：

红色	黑色	绿色	黄色
VDC+	VDC- / RS232 GND	RS232 TX	RS232 RX

SDI-12输出时，通讯线缆为4芯，接法如下：

红色	黑色	绿色	黄色
VDC+	VDC- / SDI-12 GND	SDI-12	备用/无

注： 1、本仪器默认输出接口为RS485（标配）；
2、线缆颜色定义最终以线缆上接线标签（黄色）标注为准；
3、以上三种输出接口只能任选其一；
4、产品输入电压范围为DC5-30V（标配）。

MODBUS 协议-RTU 模式

一、 通讯端口定义

起始位	1 bit
数据位	8 bit
奇偶校验位	EVEN(偶校验)
停止位	1 bit
波特率	9600

二、 通讯方式

仪器与外部设备通讯方式为 RS485 或 RS232，默认通讯方式为 *RS485*。

三、 通讯协议

MODBUS 协议-RTU 模式。

四、 协议简介

MODBUS 是一个请求/应答协议，并且提供功能码规定的服务。MODBUS 功能码是 MODBUS 请求/应答 PDU 的元素。它是一项应用层报文传输协议，用于在通过不同类型的总线或网络连接的设备之间的客户机/服务器通信。

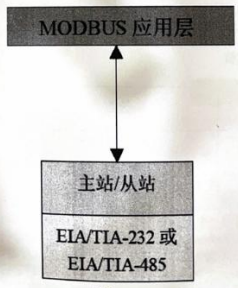


图 1

五、 协议描述

MODBUS 协议定义了一个与基础通信层无关的简单协议数据单元（PDU）。特定总线或网络上的MODBUS 协议映射能够在应用数据单元（ADU）上引入一些附加域。

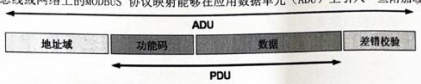


图 2. 通用 MODBUS 帧

MODBUS 有两种传输模式，**RTU** 和 **ASCII**。*本仪器采用 RTU 传输模式。*

(一) RTU传输模式

当设备使用RTU (Remote Terminal Unit) 模式在Modbus 串行链路通信， 报文中每个8位字节含有两个4 位十六进制字符。这种模式的主要优点是较高的数据密度，在相同的波特率下比ASCII 模式有更高的吞吐率。每个报文必须以连续的字符流传送。

- **RTU 模式串行位系列**



图 3. RTU 模式串行位序列

- **Modbus RTU 报文帧**

子节点地址	功能代码	数据	CRC
1 字节	1 字节	0 到 252 字节	2 字节 CRC 低 | CRC 高

图 4. ASCII 报文帧

- **CRC 校验**

在RTU 模式包含一个对全部报文内容执行的，基于循环冗余校验 (CRC - Cyclical Redundancy Checking) 算法的错误检验域。CRC 域检验整个报文的内容。不管报文有无奇偶校验，均执行此检验。CRC 包含由两个8 位字节组成的一个16 位值。CRC 域作为报文的最后的域附加在报文之后。计算后，首先附加低字节，然后是高字节。CRC高字节为报文发送的最后一个子节。

附加在报文后面的CRC 的值由发送设备计算。接收设备在接收报文时

重新计算CRC 的值，并将计算结果于实际接收到的CRC 值相比较。如果两个值不相等，则为错误。

附录含有LRC 生成的详细示例。

(二) MODBUS通讯模型

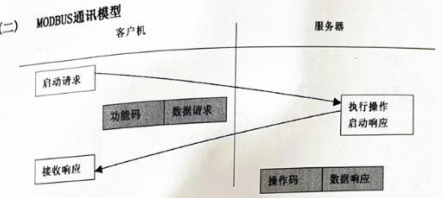


图 5. MODBUS 事务处理（无差错）

图 5 中，此处的客户机指的就是要查询仪器信息的主机。而服务器就是仪器。

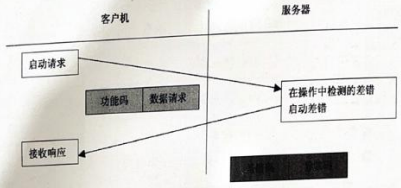


图 6. MODBUS 事务处理（异常响应）

数据编码

MODBUS 使用一个" big-Endian" 表示地址和数据项。这意味着当发射多个字节时，首先发送最高有效位。

例如：

寄存器大小值	值
16bit	0x1234

发送的第一字节为 0x12 然后 0x34

注：具体 MODBUS 协议，请参照 MODBUS 标准。

(三)

- 本仪器协议支持
- 支持的功能码

功能码类型	字长	功能码 (HEX)	描述
数据访问	16 bit	03	内部寄存器读出
数据访问	16 bit	10	写多个寄存器

- 支持的异常码

异常码	描述
01	功能码错误
02	寄存器地址错误
03	寄存器值错误
06	设备正在忙

- 仪器内部寄存器描述：

寄存器	字长	值数据类型	值定义	值范围
寄存器 1	16 bit	16 bit 整形	保留	/
寄存器 2	16 bit	16 bit 整形	风向	0 到 359 °
寄存器 3	16 bit	32 bit 浮点型	风速	0 m/s 到 +60 m/s
寄存器 4	16 bit			
寄存器 5	16 bit	32 bit 浮点型	温度	-40 到 +120 ℃
寄存器 6	16 bit			
寄存器 7	16 bit	32 bit 浮点型	湿度	0 到 100 %
寄存器 8	16 bit			
寄存器 9	16 bit	32 bit 浮点型	气压	150 到 1020 hPa
寄存器 10	16 bit			
寄存器 11	16 bit	16 bit 整形	电子罗盘角度	0 到 359 °
寄存器 12	16 bit	16 bit 整形	降落物状态	0x0000 ~ 0x000F
寄存器 13	16 bit	32 bit 浮点型	降雨强度	单精度值
寄存器 14	16 bit			
寄存器 15	16 bit	32 bit 浮点型	累积降雨量	单精度值
寄存器 16	16 bit			
寄存器 17	16 bit	16 bit 整形	降雨强度单位	见附表
寄存器 18	16 bit	16 bit 整形	定位状态	0.未定位 1.已定位
寄存器 19	16 bit	32 bit 浮点型	航速	Km/h
寄存器 20	16 bit			
寄存器 21	16 bit	16 bit 整形	航向	0 到 359 °

寄存器 22	16 bit	32 bit 浮点型	经度	东经为正值 西经为负值
寄存器 23	16 bit			
寄存器 24	16 bit	32 bit 浮点型	纬度	北纬为正值 南纬为负值
寄存器 25	16 bit			
寄存器 26	16 bit	32 bit 浮点型	粉尘浓度	ug/m^3
寄存器 27	16 bit			
寄存器 28	16 bit	32 bit 浮点型	能见度	m
寄存器 29	16 bit			
寄存器 30	16 bit	32 bit 浮点型	太阳辐射/ 光照度	Lux
寄存器 31	16 bit			
寄存器 32	16 bit	32 bit 浮点型	日辐射量	KJ
寄存器 33	16 bit			
寄存器 34	16 bit	32 bit 浮点型	太阳辐射 功率	W
寄存器 35	16 bit			
寄存器 36	16 bit	32 bit 浮点型	真实风向 （真北方）	0 到 359 °
寄存器 37	16 bit			
寄存器 38	16 bit	32 bit 浮点型	海拔高度	m
寄存器 39	16 bit			
寄存器 40	16 bit	32 bit 浮点型	真实风速	m/s
寄存器 41	16 bit			

注意：寄存器地址是从 0 开始编址的，如寄存器 1（地址 0x0000）。

◆ **功能码(0x03)描述-读保持寄存器**

在一个远程设备中，使用该功能码读取保持寄存器连续块的内容。请求PDU说明了起始寄存器地址和寄存器数量。从零地址开始寻址寄存器。因此，寻址寄存器1-3 对应地址为0-2。将响应报文中的寄存器数据分成每个寄存器有两字节，在每个字节中直接地调整二进制内容。对于每个寄存器，第一个字节包括高位比特，并且第二个字节包括低位比特。

请求

功能码	1 个字节	0x03
起始地址	2 个字节	0x0000 至 0x0028
寄存器数量	2 个字节	1 至 41

响应

功能码	1 个字节	0x03
字节数	1 个字节	N*2
寄存器值	N*2 个字节	

注：N 表示寄存器个数

错误响应

差错码	1 个字节	0x83
异常码	1 个字节	01 或 02 或 03 或 06

		寄存器 37 值-高字节	41	
		寄存器 37 值-低字节	48	
		寄存器 38 值-高字节	33	海拔高度 (50.3)
		寄存器 38 值-低字节	33	
		寄存器 39 值-高字节	42	
		寄存器 39 值-低字节	49	
		寄存器 40 值-高字节	A2	真实风速 (2.82)
		寄存器 40 值-低字节	E1	
		寄存器 41 值-高字节	40	
		寄存器 41 值-低字节	34	

如上，实际通讯系列为：

- **请求系列：**

  (HEX) 01 03 00 00 00 29 84 14

- **应答系列：**

  (HEX) 01 03 52 BF FF 00 38 C2 BF 3C F5 33 33 41 E7 66 66 42 74 3E 14 44 68 00 0C 00 01 66 66 42 56 00 00 42 F7 02 01 00 01 99 9A 41 21 01 43 32 46 41 25 A5 9C 42 CF 00 00 42 70 00 00 42 C8 40 00 45 9C 00 00 43 48 39 9A 45 59 14 08 41 48 33 33 42 49 A2 E1 40 34 D2 5D

- **整型解析**

以风向为例

D1	D0
寄存器 2 高字节	寄存器 2 低字节
00	38
高字节	低字节

转换为整形后，值为： 0x0038 => 56°

- **浮点型解析(IEEE754 标准)**

以温度为例

D3	D2	D1	D0
寄存器 6 高字节	寄存器 6 低字节	寄存器 5 高字节	寄存器 5 低字节
41	E7	33	33
高字节	中间字节 1	中间字节 2	低字节

转换为浮点数后，值为： 0x41E73333 => 28.9 ℃

- **Device State(设备状态表)**

BIT15	BIT14	BIT13	BIT12	BIT11	BIT10	BIT9	BIT8
1	0	1	1/0	1/0	1/0	1/0	1/0
			真实风速	真实风向	海拔高度	能见度	光照度
BIT7	**BIT6**	**BIT5**	**BIT4**	**BIT3**	**BIT2**	**BIT1**	**BIT0**
1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0
太阳辐射功率/日辐射量	粉尘浓度	定位状态 航速 航向 经度 纬度	降落物状态 降雨强度 累积降雨量 降雨单位	罗盘角度	气压	风速 风向	温度 湿度

注：当设备状态表字节中相应位为 1 时，相应数据才有效；为 0，则数值无效。

- **降落物状态（HEX）**

BIT15	BIT14	BIT13	BIT12	BIT11	BIT10	BIT9	BIT8
0	0	0	0	0	0	0	0
BIT7	**BIT6**	**BIT5**	**BIT4**	**BIT3**	**BIT2**	**BIT1**	**BIT0**
0	0	0	0	0/1	固体 0/1	雪 0/1	雨 0/1

注：只是用了低 3 位。相应位为 1 时，状态有效。

◆ **功能码(0x10)描述-写多个寄存器**

在一个远程设备中，使用该功能码写连续寄存器块。

在请求数据域中说明了请求写入的值，每个寄存器将数据分成两字节。

正常响应返回功能码、起始地址和被写入寄存器的数量。

注意：这个功能只用于清除“累积雨量寄存器”，且只能写入0，否者报错。

请求

功能码	1 个字节	0x10
起始地址	2 个字节	0x000F
寄存器数量	2 个字节	2
字节数	1 个字节	4
寄存器值	4 个字节	00000000

注：这里 起始地址、寄存器数量、字节数和寄存器值为固定值。

响应

功能码	1 个字节	0x10
起始地址	2 个字节	0x000F
寄存器数量	2 个字节	2

错误响应

差错码	1 个字节	0x90
异常码	1 个字节	01 或 02 或 03 或 06

通讯实例：

写 2 个寄存器（清除累积雨量寄存器 15、16）的实例

注意：这里的累积雨量寄存器定义的地址是 15, 16, 和前面只读的累积雨量寄存器定义的地址（14, 15）有错位，这是由于版本兼容造成的，所以就把以下格式当做一个特定的通讯系列。

请求		响应	
域名	(十六进制)	域名	(十六进制)
功能码	10	功能码	10
起始地址-高字节	00	起始地址-高字节	00
起始地址-低字节	0F	起始地址-低字节	0F
寄存器数-高字节	00	寄存器数-高字节	00
寄存器数-低字节	02	寄存器数-低字节	02
字节数	04		
寄存器值-高字节	00		
寄存器值-低字节	00		
寄存器值-高字节	00		
寄存器值-低字节	00		

如上，实际通讯系列为：

- **请求系列：**
  (HEX) 01 10 00 0F 00 02 04 00 00 00 00 B3 EF
- **应答系列：**
  (HEX) 01 10 00 0F 00 02 71 CB

附属协议 - 仪器参数设置

通过与仪器的通讯连接，可以配置仪器的一些参数，如修改通讯地址，修改通讯串口波特率。

一、指令集

指令 1（进入设置模式指令）

发送
（ASCII ） >*\r\n
（16 进制） 3E 2A 0D 0A

仪器返回
（ASCII） \n>CONFIGURE MODE\r\n
（16 进制） 0A 3E 43 4F 4E 46 49 47 55 52 45 20 4D 4F 44 45 0D 0A

指令 2 （设置串口配置指令）

发送
（ASCII） >CUS 9600 8-N-1\r\n
（16 进制） 3E 43 55 53 20 39 36 30 30 20 38 2D 4E 2D 31 0D 0A
说明：CUS 后里就是需要设置的串口参数，如果后面不跟参数 则此条指令变为查询通讯地址如（>CUS\r\n），仪器会返回相应数据，如 \n>COM USART SET : 9600 N-8-1\r\n

仪器返回
（ASCII） >CMD IS SET\r\n
（16 进制） 3E 43 4D 44 20 49 53 20 53 45 54 0D 0A

指令 3 （设置地址指令）

发送
（ASCII） >ID 2\r\n
（16 进制） 3E 49 44 20 32 0D 0A
说明：ID 后里的 2 就是需要设置的地址（1-255），必须是十进制格式，如果后面不跟参数则此条指令变为查询通讯地址如（>ID\r\n），仪器会返回相应数据，如>ID(HEX) : 02\r\n

仪器返回
（ASCII） >CMD IS SET\r\n
（16 进制） 3E 43 4D 44 20 49 53 20 53 45 54 0D 0A

指令 4 （复位指令）

发送
（ASCII） >RESET\r\n
（16 进制） 3E 52 45 53 45 54 0D 0A

仪器成功接收到此指令后，会进行软复位。

指令 5 （手动退出设置模式指令）

发送
（ASCII） >!\r\n
（16 进制） 3E 21 0D 0A

仪器返回
（ASCII） \n>NORMAL MODE\r\n
（16 进制） 0A 3E 4E 4F 52 4D 41 4C 20 4D 4F 44 45 0D 0A
说明：成功接收此指令后，仪器退出设置模式

二、操作步骤

1. 设置通讯地址
   步骤：**“指令 1” => “指令 3” => “指令 5” => “指令 4”**
2. 设置串口参数
   步骤：**“指令 1” => “指令 2” => “指令 5” => “指令 4”**

备注：

1. CUS 9600 8-N-1 字段中有 2 个空格需要注意。8-N-1 以‘-’隔开，无空格。

波特率	数据位	校验位	停止位
2400~115200	7 位（部分协议支持） 8 位	N 无校验 O 奇校验 E 偶校验	1 2

2. 任何设置指令（2，3），必须先让仪器进入设置模式才生效，且设置模式有 15 秒无操作自动关闭功能，所以设置指令必须在此时间内输入，成功输入后，15 倒计时重置。
3. 对仪器进行设置后，必须发送“指令 4”让仪器软复位后，新的设置才能生效。
4. **“\r\n”** 为回车换行符，对应 HEX (0x0D,0x0A)

附录1. - CRC 的生成

循环冗余校验(CRC) 域为两个字节，包含一个二进制16 位值。附加在报文后面的CRC 的值由发送设备计算。接收设备在接收报文时重新计算CRC 的值，并将计算结果于实际接收到的CRC值相比较，如果两个值不相等，则为错误。

CRC 的计算，开始对一个16位寄存器预装全1. 然后将报文中的连续的8位子节对其进行后续的计算。只有字符中的8个数据位参与生成CRC 的运算，起始位，停止位和校验位不参与CRC计算。

CRC 的生成过程中， 每个 8 - 位字符与寄存器中的值异或。然后结果向最低有效位(LSB)方向移动(Shift) 1位，面最高有效位(MSB) 位置充零。然后提取并检查LSB；如果LSB 为1，则寄存器中的值与一个固定的预置值异或；如果LSB 为 0， 则不进行异或操作。

这个过程将重复直到执行完8 次移位。完成最后一次（第8 次）移位及相关操作后，下一个8位字节与寄存器的当前值异或，然后又同上面描述过的一样重复8 次。当所有报文中子节都运算之后得到的寄存器中的最终值，就是CRC.

生成CRC 的过程为：

1. 将一个16 位寄存器装入十六进制FFFF（全1）. 将之称作CRC 寄存器.
2. 将报文的第一个8位字节与16 位CRC 寄存器的低字节异或，结果置于CRC 寄存器.
3. 将CRC 寄存器右移1位(向LSB 方向)， MSB 充零. 提取并检测LSB.
4. （如果LSB 为0）：重复步骤3（另一次移位）.
   （如果LSB 为1）：对CRC 寄存器异或多项式值0xA001（1010 0000 0000 0001）.
5. 重复步骤3 和 4，直到完成8 次移位。当做完此操作后，将完成对8位字节的完整操作。
6. 对报文中的下一个字节重复步骤2 到5，继续此操作直至所有报文被处理完毕。
7. CRC 寄存器中的最终内容为CRC 值.
8. 当放置CRC 值于报文时，如下面描述的那样，高低字节必须交换。

将 CRC 放置于报文

当16 位CRC（2 个 8 位字节）在报文中传送时，低位字节首先发送，然后是高位字节。例如， 如果 CRC 值为十六进制1241（0001 0010 0100 0001）：

地址	功能	数据计数	数据	数据	数据	数据	CRC 低	CRC 高
							0x41	0x12

例

执行CRC 生成的C 语言的函数在下面示出，所有的可能的CRC 值都被预装在两个数组中，当计算报文内容时可以简单的索引即可。一个数组含有16 位CRC 域的所有256个可能的高位字节，另一个数组含有地位字节的值。这种索引访问CRC 的方式提供了比对报文缓冲区的每个新字符都计算新的CRC 更快的方法。

注意：此函数内部执行高/低CRC 字节的交换。此函数返回的是已经经过交换的CRC 值。也就是说，从该函数返回的CRC 值可以直接放置于报文用于发送。（低字节首先发送，接着发送高字节）

函数使用两个参数：

```
unsigned char *puchMsg;      指向含有用于生成CRC 的二进制数据报文缓冲区的指针
unsigned short usDataLen;    报文缓冲区的字节数.
```

CRC 生成函数

```
unsigned short CRC16 ( puchMsg,  usDataLen )   /* 函数以unsigned short 类型返回CRC */
unsigned char *puchMsg ;                       /* 用于计算CRC 的报文*/
unsigned short usDataLen ;                     /* 报文中的字节数*/
{
    unsigned char uchCRCHi = 0xFF ;            /* CRC 的高字节的初始化*/
    unsigned char uchCRCLo = 0xFF ;            /* CRC 的低字节的初始化*/
    unsigned uIndex ;                          /* CRC 查询表索引*/
    while (usDataLen--)                        /* 传输整个报文缓冲区*/
    {
        uIndex = uchCRCLo ^ * puchMsg++ ;      /* 计算CRC */
        uchCRCLo = uchCRCHi ^ auchCRCHi[uIndex] ;
        uchCRCHi = auchCRCLo[uIndex] ;
    }
    return (uchCRCHi << 8 | uchCRCLo) ;
}
```

高字节表

```
/* 高位字节的CRC 值*/
static unsigned char auchCRCHi[] =
{
  0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,0x00, 0xC1, 0x81,0x40, 0x01,
  0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81,0x40, 0x01, 0xC0,0x80, 0x41, 0x01, 0xC0,
  0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1,0x81, 0x40, 0x01,0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
  0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,0xC0, 0x80, 0x41,0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
  0x00, 0xC1, 0x81, 0x40,0x00, 0xC1, 0x81,0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,
  0xC0, 0x80,0x41, 0x01, 0xC0,0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,
  0xC0,0x80, 0x41, 0x01,0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,0x00,0xC1,
  0x81, 0x40,0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,0x00, 0xC1,
  0x81,0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,0x40, 0x01, 0xC0,0x80,
  0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1,0x81, 0x40, 0x01,0xC0, 0x80, 0x41,
  0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01,0xC0, 0x80, 0x41,0x00, 0xC1, 0x81, 0x40, 0x00,
  0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
  0x00, 0xC1, 0x81,0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,0x40, 0x01,
  0xC0,0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,0x80, 0x41, 0x01,0xC0,
  0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,0xC0, 0x80, 0x41,
  0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,0x00, 0xC1, 0x81,0x40
};
```


附录2. 4个字节转换为浮点数。（基于C语言）

```
union                                   //共用体
{
    float   TestData_Float;             //浮点数（4 个字节）
    unsigned char TestArray[4];         //数值
}TData;
```

注：在共用体中，上例中的浮点数和四个字节的字符数组共用一段存储空间。

解析：

D3	D2	D1	D0
寄存器 2 高字节	寄存器 2 低字节	寄存器 1 高字节	寄存器 1 低字节
40	AC	19	DF
高字节	中间字节 1	中间字节 2	低字节

转换为浮点数后，值为： 5.378

程序：

```
float Tempfloat;

TData.TestArray [3]= 0x40;              //输入高字节
TData.TestArray [2]= 0xac;              //
TData.TestArray [1]= 0x19;              //
TData.TestArray [0]= 0xdf;              //低字节

Tempfloat = TData.TestData_Float;       //得到浮点数 5.378
```

注：以上实例在 VC6.0 测试通过，因 PC 是基于小端模式存储，故低地址字节存储数据低字节，高地址字节存储数据高字节。在实际应用中，根据实际存储模式，决定存储浮点型高低字节的位置。